Lecture notes. Lecture 10

Reinforcement learning
Full lecture notes

Marcin Płodzień
Deep Learning in Classical and Quantum Physics
Institute of Theoretical Physics, Jagiellonian University in Kraków

These notes introduce reinforcement learning, the branch of machine learning in which an agent learns a strategy from rewards, and apply it to the control of quantum systems. The order and notation of the general theory follow the chapter on reinforcement learning in Ref. [1,2], and Ref. [2] is the standard textbook of the field. Appendix B lists the notation. The derivations state the assumptions behind the algorithms and distinguish their mathematical objectives from the recorded numerical experiments.

Part I
Foundations

1 Learning from rewards

1.1 Three ways to learn

Machine learning distinguishes three broad ways to learn from experience. In unsupervised learning a model receives data without labels and finds structure in them, such as clusters, a low-dimensional description or a probability distribution from which new samples can be drawn. In supervised learning a model learns from examples labelled by a teacher. A network that classifies images is shown many images together with their correct classes, and it learns to give the answers of the teacher. Its training signal is therefore limited to the labelled examples supplied to it [1,2].

Reinforcement learning removes the teacher. A learner, now called the agent, acts, observes the consequences of its actions and receives a number, the reward, that rates how good the outcome was. Its task is to find the strategy that collects the most reward. The reward rates the outcome and leaves open which action would have been right. In chess, a supervised program would be trained on recorded games of strong players, and for every position it would predict the move that such a player made. A position often has several good moves, and the right one depends on the plan that the player follows over the next moves. A model that copies single moves from many different players may therefore mix their plans and follow none of them. A reinforcement learning program plays games against other programs or against itself, knowing only the rules, and learns from the outcomes. The outcome, a win or a loss, arrives only at the end of the game, so the agent has to find out by itself which of its many moves were responsible for it [1,2].

1.2 Machines that learned to beat humans

The board game Go shows how far this approach reaches. Two players place black and white stones on the 361 points of a 19×19 board. The number of legal positions is about 2.1×10^{170} [5], so no program can store a good move for every position. The value of a move often becomes clear only

hundreds of moves later. AlphaGo first learned from games of human experts by supervised learning and then improved by playing against itself. It defeated the European Go champion by five games to zero, the first win of a program over a human professional player on the full board [4]. AlphaGo Zero started from the rules alone, learned only from games against itself, and defeated the earlier version [3]. MuZero, which was not given the rules, learned a model that predicts the consequences of its moves and planned inside this learned model. In Go, chess and shogi it matched the playing strength of a predecessor that had been given the rules, and it also played Atari video games [6].

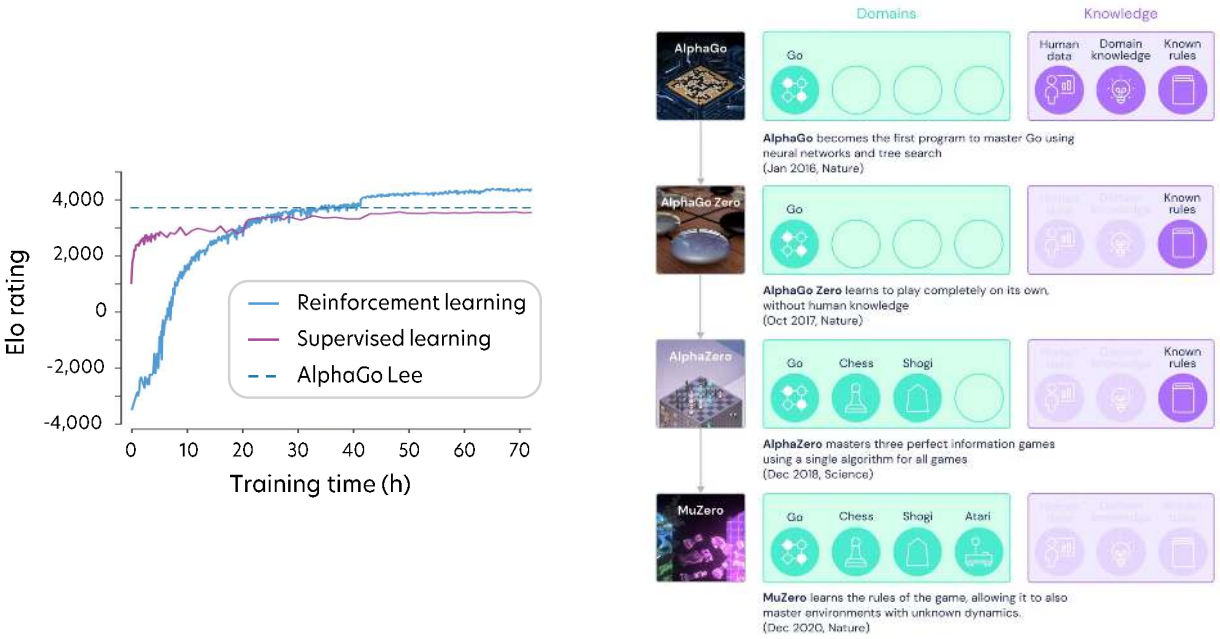


Figure 1: Learning Go. Left, the Elo rating, a measure of playing strength computed from game results, against the training time in hours. The curves show AlphaGo Zero trained only by playing against itself (reinforcement learning) and the same network trained by supervised learning on expert games. The dashed line marks the AlphaGo version that defeated the world champion Lee Sedol in 2016. The supervised start leads early and is then overtaken. Source [3,1]. Right, AlphaGo, AlphaGo Zero, AlphaZero and MuZero, the games each one masters and the knowledge it is given, human games, domain knowledge and the rules. Image by DeepMind.

Video games gave further milestones. A single deep network, the deep Q-network, learned to play 49 Atari games from the pixels of the screen and the game score alone, with the same architecture and settings for every game. It reached a level comparable to a professional human games tester on many of them [7]. AlphaStar learned the real-time strategy game StarCraft II, in which the player sees only part of the map and controls many units at once. It was ranked above 99.8% of the officially ranked human players [8]. In a game of hide and seek, two teams of simulated agents were rewarded only for staying hidden or for finding the hiders. The agents learned one after another to build shelters from boxes, to climb walls with ramps and to lock the ramps away, although nothing in the reward mentions boxes or ramps (Figure 2) [9]. Robots learn in the same way, mostly in simulation, because a simulated robot can fall many thousands of times at no cost. A policy for the legged robot ANYmal trained in simulation ran on the real robot, followed velocity commands, ran faster than the previous controllers and recovered from falls [11]. Boston Dynamics trains a locomotion policy for its quadruped robot Spot with reinforcement learning in over a million simulations and runs it on the real robots [10] (Figure 3).

The general recipe is to formulate a task as a game whose score rewards the goal, and to let an agent discover a strategy by playing it. Strategies found this way include new quantum optics experiments [12] and faster algorithms for matrix multiplication [13] and for sorting [14]. Physics laboratories are full of control problems of this kind. A policy network trained in a simulator of the TCV tokamak sets the voltages of its magnetic coils in real time and shapes the plasma [15].

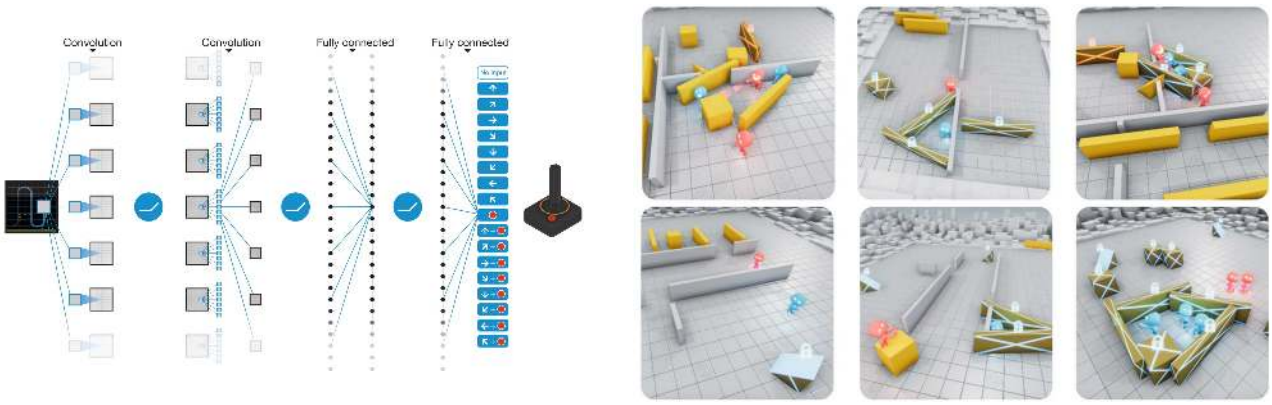


Figure 2: Left, the deep Q-network that learned Atari games. The screen image enters convolutional layers and fully connected layers, drawn schematically. The outputs rate the joystick actions, up to 18 depending on the game, each drawn as a button. Source [7]. Right, stages of the strategies found by the hiders (blue) and seekers (red) in the hide-and-seek game, from running around to building shelters from boxes and using ramps. Image by OpenAI [9].

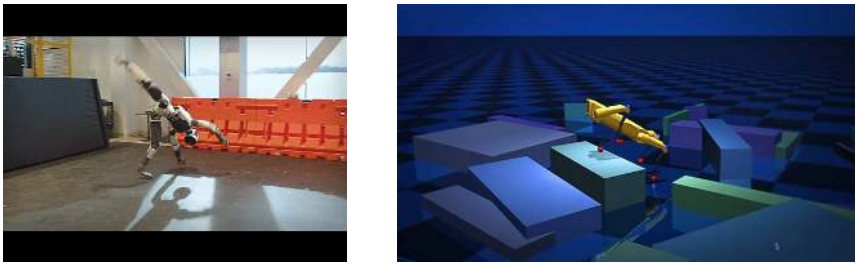


Figure 3: Robots of Boston Dynamics. Left, the humanoid Atlas in the video "Walk, Run, Crawl, RL Fun" (https://www.youtube.com/watch?v=I44_zbEwz_w), with motions learned by reinforcement learning. Right, the quadruped Spot in one of the simulations used to train its locomotion policy (https://bostondynamics.com/video/reinforcement-learning-with-spot/). Images by Boston Dynamics.

Model-free reinforcement learning steers particle beams in accelerator lines at CERN [16], and an agent learned to move single atoms on a surface with the tip of a scanning tunnelling microscope [17]. Figure 4 shows both setups. In quantum technologies the agent chooses pulses, gates or feedback from measurements, and the reward measures how close the device came to the desired quantum state.

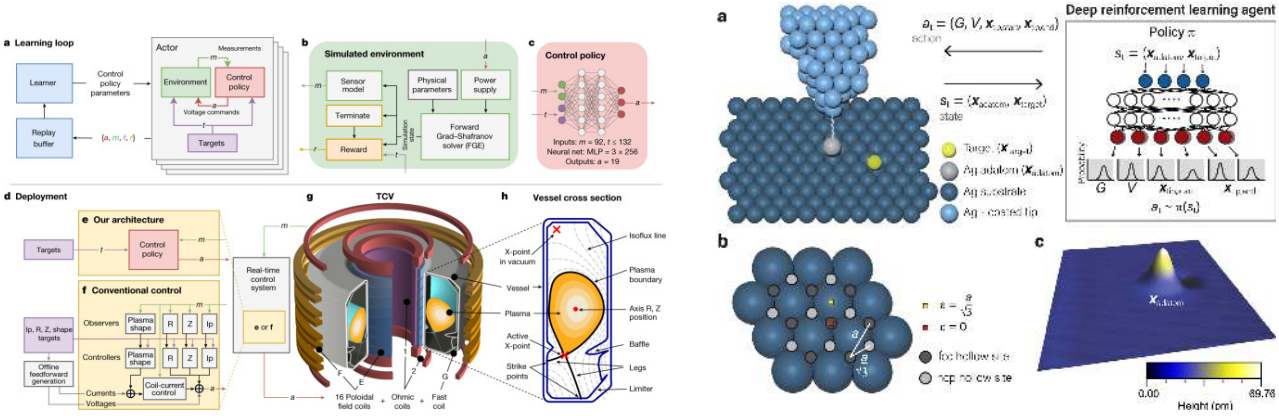


Figure 4: Reinforcement learning in the laboratory. Left, control of the plasma in the TCV tokamak. The control policy, a neural network, is trained in a simulated environment (top). In deployment it sets the voltages of the control coils of the tokamak in real time (bottom). Source [15], Fig. 1. Right, manipulation of single atoms. The state is the position of a silver adatom and of its target. The action sets the bias, the current and the start and end positions of the microscope tip, and a policy network chooses the action. Source [17], Fig. 1.

1.3 Three guiding points

A single reward rates only one step, whereas the agent cares about the total reward that it collects over many steps. A move that earns nothing now can therefore be the best one when it prepares a large reward later. Without labelled examples, the objective is defined by the rewards alone, as an average total reward over the random course of events, and the agent produces its own data by acting. Two families of methods optimise this objective. One learns how much total reward to expect from each situation and acts on these expectations, and the other adjusts the strategy itself along the gradient of the expected total reward.

2 Agent and environment

2.1 Agent and environment

Every reinforcement learning problem has two parts, an agent and an environment [1,2]. The agent is the learner and decision maker, the program that plays Go, the controller of a robot or the controller of a qubit. The environment, everything the agent interacts with, contains all that defines the task, such as the rules of a game together with the opponent, or an apparatus together with the laws of physics that govern it. Time runs in discrete steps $t = 0, 1, 2, \ldots$. At step t the agent observes the state s_t of the environment and chooses an action a_t. The action changes the state to s_{t+1}, and the environment returns a single number, the reward r_{t+1} (Figure 5). The reward carries the index $t + 1$ because it arrives together with the new state. The reward is the only information about the goal that the agent ever gets.

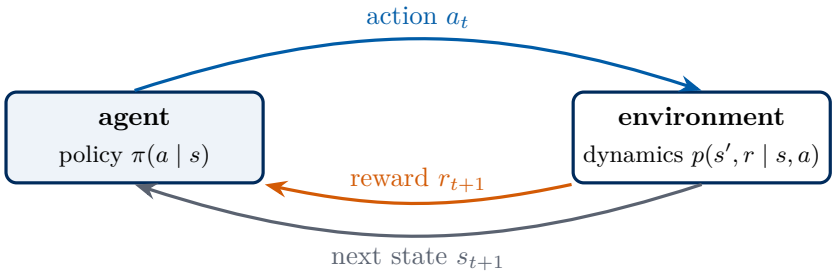


Figure 5: The loop of reinforcement learning. The agent observes the state s_t and chooses an action a_t with its policy $\pi(a|s)$. The environment answers with the next state s_{t+1} and the reward r_{t+1}, a single number, drawn from its dynamics $p(s', r|s, a)$, and the loop repeats.

2.2 States and actions

The state space $\mathcal{S}$ is the set of all states the environment can be in. For Go a state is a position of the stones on the board. For a robot arm it is the list of the angles and angular velocities of its joints, and for a qubit it is its quantum state $|\psi\rangle$. The state space can be finite, as for Go, or a continuous space of many dimensions, as for the robot arm. When the agent receives only part of the state, it receives an observation, and the problem is called partially observed.

The action space $\mathcal{A}(s)$ is the set of actions the agent can take in the state s. A Go player chooses one of the legal moves, a set that depends on the position. The controller of the robot arm sets the torques of the joint motors, and the controller of a qubit chooses the next microwave pulse or the next quantum gate. When every state offers the same actions we write $\mathcal{A}$. An action can have a deterministic or a random outcome. A move in Go always puts the stone where it was placed, while a pulse on a real qubit sometimes fails.

Figure 6 draws a small abstract environment that we use throughout this part. It has three states, s_A, s_B and s_C, drawn as circles, and the actions available in each state are drawn as small dots attached to it. The state s_A offers two actions and the state s_B one. Each action leads to one

of several next states, and the arrows carry the probability of each outcome and the reward that comes with it. The state s_C ends the run and is called terminal. The numbers in the figure are an illustration and stand for no particular system. We name these states with letters, so that a numerical subscript, as in s_t, always denotes the time step.

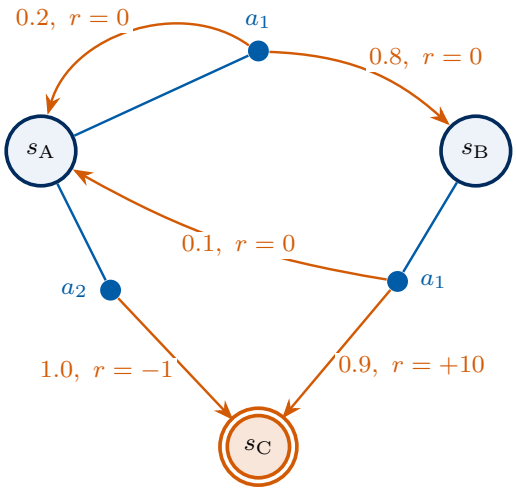


Figure 6: A small abstract decision process. Circles are the states s_A, s_B and the terminal state s_C (double circle). Dots are the actions, a_1 and a_2 in s_A and a_1 in s_B. Each arrow leaving an action is one possible outcome, labelled with its probability $p(s', r|s, a)$ and its reward r. For example, the action a_1 in s_B leads to s_C with the reward $+10$ with probability 0.9, and back to s_A with the reward 0 with probability 0.1. The numbers are an illustration.

2.3 The reward

After every action the environment returns the reward $r_{t+1} \in \mathbb{R}$. In Figure 6 the reward is written on each arrow. Reaching s_C from s_B pays $+10$, reaching it directly from s_A costs -1, and every other outcome pays 0. In Go the reward is $+1$ for a win and -1 for a loss at the end of the game and zero for every move before it. For a qubit that has to be prepared in a target state the reward can be the fidelity with the target when the protocol ends. One run from the start to a terminal state is an episode.

2.4 The total reward as the goal

The agent cares about the sum of the rewards of an episode, the return

$$G \coloneqq r_1 + r_2 + \cdots + r_T, \tag{2.1}$$

where T is the step at which the episode ends (Figure 7). The goal is to choose actions that make G as large as possible, on average over the random outcomes of the actions. Maximising the next reward alone can mislead the agent, as Figure 6 shows. In s_A the action a_1 earns 0 and the action a_2 earns -1. An agent that judged each action by its immediate reward would prefer a_1 for a reason unrelated to the large reward $+10$. A move in Go that earns nothing now can decide the game a hundred moves later. The agent must judge an action by everything that follows it.

2.5 The policy

The agent chooses its actions according to a policy, a rule that assigns to every state a probability for each action (Figure 8). A deterministic policy gives one action per state. In Figure 6 there are two deterministic policies, since only s_A offers a choice, a_1 or a_2. The random policy chooses each available action with equal probability, whatever happened before. The policy is the strategy of the agent, and reinforcement learning is the search for a policy with a large expected return.

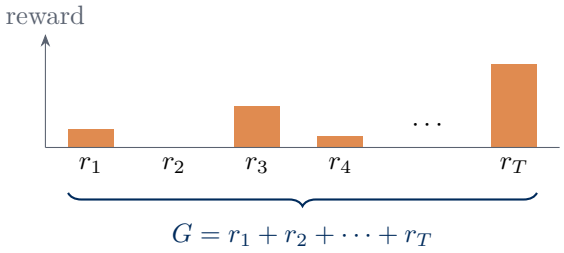


Figure 7: The rewards $r_1, \ldots, r_T$ of one episode as bars, and the return G of equation (2.1), their sum. The heights are an illustration. Small rewards early and a large reward at the end are typical of tasks with a goal.

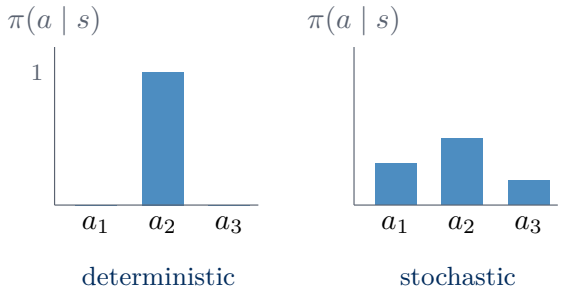


Figure 8: A policy in one state s with three actions, as the probabilities $\pi(a|s)$ of the actions. Left, a deterministic policy, which always takes a_2. Right, a stochastic policy, which takes every action with some probability. The heights are an illustration.

2.6 Episodes and trajectories

The interaction of agent and environment produces the sequence

$$s_0,\ a_0,\ r_1,\ s_1,\ a_1,\ r_2,\ s_2,\ \ldots,\ s_T, \tag{2.2}$$

called a trajectory, which lists the states, the actions and the rewards (Figure 9). An episode is a trajectory from the start to a terminal state. Its length T is random, since the choices of the policy and the outcomes of the actions change its course. Starting from s_A in Figure 6, the action a_1 can return to s_A several times before the agent reaches s_B, so episodes of every length occur. The agent records a list of transitions $(s_t, a_t, r_{t+1}, s_{t+1})$, and nothing in this list tells which of the earlier actions were good. A Go program that wins after two hundred moves receives a single reward $+1$ at the end, and inferring which decisions were responsible is the credit assignment problem.

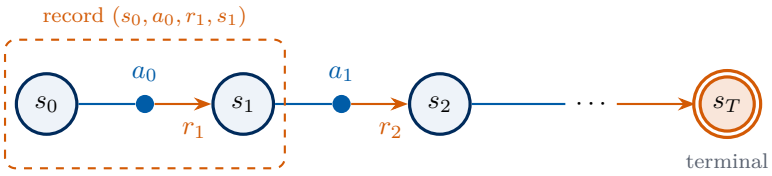


Figure 9: One episode as a chain. Circles are states, dots the actions a_t chosen in them, and the reward r_{t+1} (orange) arrives with the next state. The double circle s_T is the terminal state. The dashed box marks one recorded transition (s_0, a_0, r_1, s_1).

3 The Markov decision process

3.1 The Markov property

The return depends on the future rewards, and these depend on how the environment responds to the actions. Reinforcement learning describes this response with environments that have the Markov property [1,2].

Markov property

The next state depends on the history only through the present state and action,

$$p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1} \mid s_t, a_t). \tag{3.1}$$

The present state is a sufficient statistic. It contains all the information about the past that matters for the future.

The graph of Figure 6 has this property by construction, since the probabilities on its arrows depend only on the state and the action they start from. The position of a thrown ball alone is not a Markov state, since it does not predict the next position, while position and velocity together are (Figure 10). Whether a description has the Markov property therefore depends on what is included in the state.

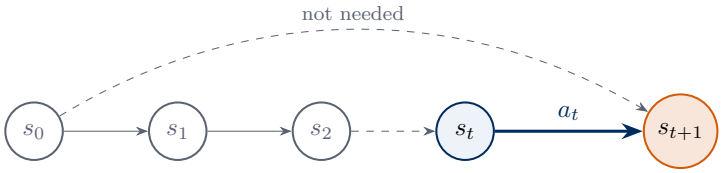


Figure 10: The Markov property. The next state s_{t+1} depends on the present state s_t and the action a_t. The earlier states $s_0, s_1, s_2, \ldots$ (grey) add no information about it.

3.2 The dynamics and the model of the environment

The environment answers an action with a next state and a reward. The joint probability of the two is the central object of the theory [1,2].

Dynamics and its marginals

The dynamics is the probability that the action a in the state s leads to the next state s' with the reward r,

$$p(s', r \mid s, a) \coloneqq \Pr\left(s_{t+1} = s',\ r_{t+1} = r \mid s_t = s,\ a_t = a\right), \qquad \sum_{s',r} p(s', r \mid s, a) = 1. \tag{3.2}$$

Summing out the reward gives the state-transition probability, where the agent lands,

$$p(s' \mid s, a) = \sum_r p(s', r \mid s, a), \tag{3.3}$$

and averaging the reward gives the expected reward of an action and of a transition that ends in s',

$$r(s, a) \coloneqq \mathbb{E}[r_{t+1} \mid s_t = s, a_t = a] = \sum_r \sum_{s'} r\, p(s', r \mid s, a), \tag{3.4}$$

$$r(s, a, s') \coloneqq \mathbb{E}[r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'] = \sum_r r \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \tag{3.5}$$

The dynamics contains everything the environment does, and it is called the model of the environment.

For a deterministic environment the dynamics is zero or one. The ratio in equation (3.5) is the probability of the reward r given that the next state is s'. In Figure 6 all four objects can be read off. Take the state s_B and the action a_1,

$$p(s_\mathrm{C}, +10 \mid s_\mathrm{B}, a_1) = 0.9, \qquad p(s_\mathrm{A}, 0 \mid s_\mathrm{B}, a_1) = 0.1, \tag{3.6}$$

and all other entries vanish. The marginals are $p(s_\mathrm{C}|s_\mathrm{B}, a_1) = 0.9$ and $p(s_\mathrm{A}|s_\mathrm{B}, a_1) = 0.1$, the expected reward is $r(s_\mathrm{B}, a_1) = 0.9 \cdot 10 + 0.1 \cdot 0 = 9$, and the transition rewards are $r(s_\mathrm{B}, a_1, s_\mathrm{C}) = 10$ and $r(s_\mathrm{B}, a_1, s_\mathrm{A}) = 0$. In this graph the reward is fixed once the next state is known, and $p(s'|s, a)$ together with $r(s, a, s')$ carries the same information as $p(s', r|s, a)$.

The reward can also be random for a fixed next state. In the card game blackjack, a player who decides to stop with a given hand ends the game, and the outcome depends on the hidden cards of the dealer. The next state is always the end of the game, $p(\mathrm{end}\,|\,s, \mathrm{stop}) = 1$, and the whole randomness is in the reward. The reward is $+1$ with the probability $w(s)$ of winning with the hand s and -1 otherwise, so that $r(s, \mathrm{stop}) = 2w(s) - 1$. The joint distribution $p(s', r|s, a)$ is the general object that covers both cases.

3.3 The discounted return

The return (2.1) has two defects. In a task without end it can be infinite, and it does not reward speed. The discounted return weighs a reward collected k steps later with the factor γ^k [1,2].

Discounted return

The discounted return from the step t is

$$G_t \coloneqq \sum_{k=0}^{T-t-1} \gamma^k \, r_{t+k+1} = r_{t+1} + \gamma \, r_{t+2} + \gamma^2 r_{t+3} + \dots, \tag{3.7}$$

where T is the final step of the episode and $\gamma \in [0, 1]$ is the discount factor. It obeys the recursion

$$G_t = r_{t+1} + \gamma \, G_{t+1}. \tag{3.8}$$

The return from now is the next reward plus the discounted return from the next step. A reward k steps ahead counts with the weight γ^k.

For $\gamma < 1$ the sum (3.7) stays finite even for an endless episode, $T = \infty$, since bounded rewards give $|G_t| \le r_\mathrm{max}/(1 - \gamma)$ with r_max the largest absolute reward. Almost every method of these notes is built on the recursion (3.8).

In a task whose only reward is 1 for reaching a goal, an episode that reaches the goal with its T-th action has the return $G_0 = \gamma^{T-1}$. Every extra step multiplies the return by γ, and the discount turns the goal "reach the goal" into the goal "reach the goal with as few steps as possible". With $\gamma = 1$ every episode that eventually succeeds has the return 1, and speed no longer matters. The discount, or an explicit cost per step, makes speed part of the task.

The two limits of γ show what the discount does (Figure 11). For $\gamma \to 0$ only the next reward counts, and the agent becomes myopic. It grabs the reward it can get now and ignores everything after. For $\gamma \to 1$ all rewards count equally, and the agent may accept poor rewards early in exchange for larger ones later. The weights γ^k sum to $1/(1 - \gamma)$, which sets a horizon of about $1/(1 - \gamma)$ steps, ten steps for $\gamma = 0.9$ and a hundred for $\gamma = 0.99$. Choosing γ changes the task itself, and in practice it is often chosen by trial and error [1,2].

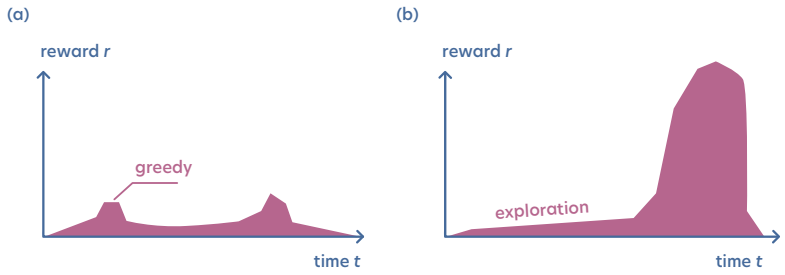


Figure 11: The effect of the discount factor, the reward r collected against the time t of an episode. (a) A myopic agent, $\gamma \to 0$, greedily takes early small rewards. (b) A long-term agent, $\gamma \to 1$, explores with poor early rewards and collects a much larger reward later. Source [1,2].

3.4 The decision process

Markov decision process

A Markov decision process (MDP) is the tuple

$$(\mathcal{S},\ \mathcal{A},\ p,\ r,\ \gamma) \tag{3.9}$$

of the state space, the action space, the dynamics (3.2), the reward and the discount factor [1,2]. The dynamics describes the environment, and the reward together with the discount fixes the objective, the discounted return (3.7).

The word decision refers to the actions of the agent, which change the probabilities of what comes next. Figure 6 is a complete Markov decision process once a discount factor is chosen. All reinforcement learning problems of these notes are formulated as Markov decision processes.

3.5 Policy and transition matrix

Policy

A policy is the probability of each action in each state,

$$\pi(a\,|\,s) \coloneqq \Pr(a_t = a\,|\,s_t = s), \qquad \sum_a \pi(a\,|\,s) = 1. \tag{3.10}$$

It is the complete strategy of the agent. A fixed policy turns the decision process into a Markov chain on the states with the transition matrix

$$P^\pi_{ss'} \coloneqq \sum_a \pi(a\,|\,s)\, p(s'\,|\,s,a), \tag{3.11}$$

the probability to go from s to s' in one step when the action is drawn from the policy.

A deterministic policy puts probability one on a single action in every state, and the random policy, also called uniform, has $\pi(a|s) = 1/|\mathcal{A}|$, where $|\mathcal{A}|$ is the number of actions. The expected reward per step in the state s becomes $r_\pi(s) \coloneqq \sum_a \pi(a|s)\, r(s,a)$. For the random policy of Figure 6, $\pi(a_1|s_\mathrm{A}) = \pi(a_2|s_\mathrm{A}) = \frac{1}{2}$ and $\pi(a_1|s_\mathrm{B}) = 1$. The row of s_A is then $P^\pi_{s_\mathrm{A}s_\mathrm{A}} = \frac{1}{2}\cdot 0.2 = 0.1$, $P^\pi_{s_\mathrm{A}s_\mathrm{B}} = \frac{1}{2}\cdot 0.8 = 0.4$ and $P^\pi_{s_\mathrm{A}s_\mathrm{C}} = \frac{1}{2}\cdot 1.0 = 0.5$, and $r_\pi(s_\mathrm{A}) = \frac{1}{2}\cdot 0 + \frac{1}{2}\cdot(-1) = -\frac{1}{2}$. The last entry leads into the terminal state and drops out of the matrix of non-terminal states.

3.6 Trajectories and their probability

In a trajectory $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$ all states, actions and rewards are random variables. The agent chooses, the environment answers, and the two alternate (Figure 12).

Probability of a trajectory

By the Markov property the probability of a trajectory, for a given initial state s_0, is a product of one policy factor and one dynamics factor per step [1,2],

$$p(\tau) = \prod_{t=0}^{T-1} \pi(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t), \tag{3.12}$$

and its discounted return is $G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$. The policy factors are what the agent controls, the dynamics factors what the environment does.

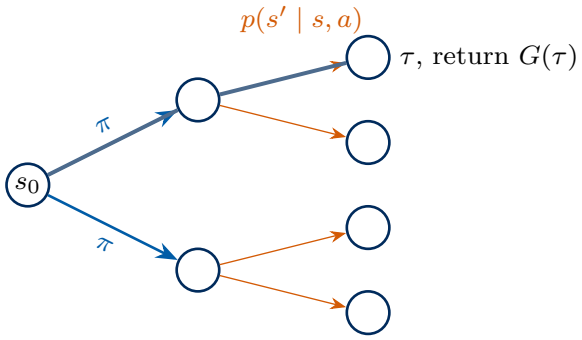


Figure 12: Trajectories as a tree. From s_0 the policy π chooses an action (blue), and the environment answers with a next state drawn from $p(s'|s,a)$ (orange). One path through the tree (dark) is a trajectory τ with its return $G(\tau)$, and its probability is the product (3.12) of the factors along the path.

When the reward is fixed by the next state, as in Figure 6, the dynamics factor reduces to $p(s_{t+1}|s_t, a_t)$. Under the random policy the trajectory $s_\mathrm{A} \xrightarrow{a_1} s_\mathrm{B} \xrightarrow{a_1} s_\mathrm{C}$ has the probability

$$p(\tau) = \underbrace{\tfrac{1}{2}}_{\pi(a_1|s_\mathrm{A})} \cdot \underbrace{0.8}_{p(s_\mathrm{B}|s_\mathrm{A},a_1)} \cdot \underbrace{1}_{\pi(a_1|s_\mathrm{B})} \cdot \underbrace{0.9}_{p(s_\mathrm{C}|s_\mathrm{B},a_1)} = 0.36, \tag{3.13}$$

and the return $G(\tau) = 0 + \gamma \cdot 10 = 10\gamma$. A trajectory that first returns to s_A once carries an extra factor $\frac{1}{2} \cdot 0.2 = 0.1$ and the return $10\gamma^2$. Long trajectories are individually rare, since every step multiplies the probability by a factor smaller than one, although long episodes as a whole can be common. A change of the policy changes only the first factor of every step.

3.7 Memory and enlarged states

The Markov property is a property of the chosen state, and a description that violates it can often be repaired. Suppose that the motors of a robot arm heat up. The response of a joint to a given torque then depends on how hard the arm worked in the last minute. The angles and velocities no longer determine the next state, and the dynamics on them has memory. Adding the temperatures of the motors to the state restores the Markov property on the enlarged state space. In general the past can always be included in the state, at the cost of a larger state space. Conversely, Markovian dynamics, even deterministic dynamics, on a large state space looks random and non-Markovian when only a part of the state is watched [1,2].

Open quantum systems offer a close analogy. A quantum system together with its environment, the bath, evolves unitarily, and nothing is lost or random in the global evolution. The system alone evolves non-unitarily, and when the bath remembers the past interaction, its reduced dynamics is non-Markovian. In the same way an agent that sees only part of the state sees a process with memory. A window of its recent observations, given as input, is an enlarged observation that carries part of this memory.

4 Exploration and exploitation

An agent that learns from its own experience faces a dilemma [1,2]. To collect reward it should exploit what it already knows and choose the action that has worked best so far. To learn something new it has to explore, to try actions whose consequences it does not yet know, which costs reward now and may pay off later. Pure exploration is a random search and never uses what it has learned. Pure exploitation commits to the first strategy that gives any reward, however poor. A learning agent needs both, and the exploration should gradually concentrate on the promising parts of the state and action spaces. An agent that has once reached a reward along one route and only exploits keeps to this route and never finds out whether another one pays more.

The simplest rule that combines both is the ε-greedy policy (Figure 13). With probability $1-\varepsilon$ the agent takes the action it currently believes best, and with probability ε it takes an action drawn uniformly from all actions,

$$\pi(a \mid s) = \begin{cases} 1-\varepsilon+\varepsilon/|\mathcal{A}| & a = \text{best action so far}, \\ \varepsilon/|\mathcal{A}| & \text{otherwise}. \end{cases} \tag{4.1}$$

The value $\varepsilon = 1$ is pure exploration and $\varepsilon = 0$ pure exploitation. A common schedule starts with a large ε and lowers it during training. A second rule acts through the initial beliefs of the agent. An optimistic start assumes that every untried action is better than any outcome the agent can achieve. Every action that has been tried then looks worse than the untried ones, so even an agent that always takes the action it believes best tries every action at least once.

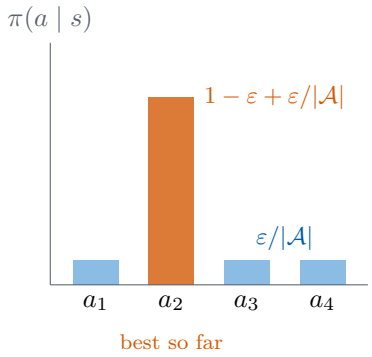


Figure 13: The ε-greedy policy (4.1) in one state with four actions. The action a_2 has the largest estimated value and is taken with probability $1-\varepsilon+\varepsilon/|\mathcal{A}|$. Every other action keeps the probability $\varepsilon/|\mathcal{A}|$, so it is still tried from time to time.

5 Model-free and model-based learning

There are two ways to learn, and they differ in what the agent knows about the dynamics $p(s', r|s, a)$, the model of the environment [1,2] (Figure 14). In model-free reinforcement learning the agent has no access to the model and learns only from the transitions it experiences, by trial and error. In model-based reinforcement learning the agent has the model, or learns one from its experience, and uses it to plan, predicting the outcome of a sequence of actions before taking any of them. A learned model can be a network trained by supervised learning to predict (s_{t+1}, r_{t+1}) from (s_t, a_t). For discrete states the planning often takes the form of a search tree, as in AlphaGo and MuZero. A model is valuable when interactions with the environment are expensive, as in an experiment, because simulated experience can replace part of the real one. It is helpful only when it is accurate, and building it costs effort and is specific to the problem. Model-free methods need no knowledge of the environment and apply without change to many different tasks.

With the model the values of a policy and the best policy can be computed without any learning. Such answers are the reference against which model-free methods are measured.

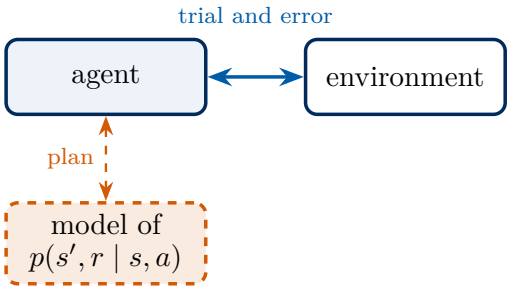


Figure 14: Model-free and model-based learning. A model-free agent learns only from trial and error with the environment (blue). A model-based agent also plans with a model of the dynamics $p(s', r|s, a)$ (orange, dashed), given or learned.

6 Value functions and the Bellman equation

6.1 State value, action value and advantage

A state is good for an agent when the return that follows it is large on average. In Figure 6 the state s_{B} is one step away from the reward $+10$, while s_{A} is at least two steps away and risks the penalty -1. In s_{A} the action a_1 is better than a_2, which ends the episode at once with -1. Since the return depends on the actions taken later, the value of a state is defined for a given policy [1,2].

Value functions

The state-value function of the policy π is the expected return from the state s when π is followed,

$$V_\pi(s) \coloneqq \mathbb{E}_\pi\big[G_t \,\big|\, s_t = s\big] = \mathbb{E}_\pi\Big[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \,\Big|\, s_t = s\Big]. \tag{6.1}$$

It says how good it is to be in s under π. The action-value function is the expected return when the action a is taken in s and π is followed afterwards,

$$Q_\pi(s, a) \coloneqq \mathbb{E}_\pi\big[G_t \,\big|\, s_t = s,\ a_t = a\big]. \tag{6.2}$$

The action value rates the choice of a in s. The advantage measures how much better the action a is than the average action of the policy,

$$A_\pi(s, a) \coloneqq Q_\pi(s, a) - V_\pi(s). \tag{6.3}$$

The expectation $\mathbb{E}_\pi$ runs over all trajectories that start in s, weighted with their probability (3.12), and so averages over the random choices of the policy and over the random outcomes of the environment. A terminal state has the value zero, since no reward follows it. The simplest estimate of a value runs many episodes from s (Figure 15), and the mean of their returns approaches $V_\pi(s)$ as their number grows.

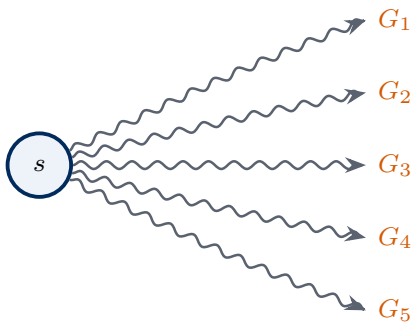


Figure 15: Five sampled episodes from the state s under the policy π (wavy lines), each with its return $G_1, \dots, G_5$. The mean of the returns of many such episodes estimates $V_\pi(s)$. When the first action is fixed to a and π is followed afterwards, the same mean estimates $Q_\pi(s,a)$.

6.2 The Bellman equation

The Bellman equation is a consistency condition on the unknown values, inherited from the recursion (3.8) of the return. The expected next reward plus the expected continuation must agree with the value assigned to the current state. Because the continuation value already averages all later rewards under the policy, a relation over one transition encodes the whole future. Starting from the definition and inserting $G_t = r_{t+1} + \gamma G_{t+1}$,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi\big[G_t \,\big|\, s_t = s\big] = \mathbb{E}_\pi\big[r_{t+1} + \gamma G_{t+1} \,\big|\, s_t = s\big] \\ &= \underbrace{\sum_a \pi(a \,|\, s)}_{\text{policy average}} \; \underbrace{\sum_{s',r} p(s', r \,|\, s, a)}_{\text{environment average}} \Big(r + \gamma \underbrace{\mathbb{E}_\pi\big[G_{t+1} \,\big|\, s_{t+1} = s'\big]}_{= V_\pi(s') \text{ by the Markov property}} \Big) \\ &= \sum_a \pi(a \,|\, s) \sum_{s',r} p(s', r \,|\, s, a) \big[r + \gamma\, V_\pi(s') \big] = \mathbb{E}_\pi\big[r_{t+1} + \gamma V_\pi(s_{t+1}) \,\big|\, s_t = s\big]. \end{aligned} \tag{6.4}$$

The second line splits the expectation into an average over the first step and an average over everything after it (the law of total expectation). In the first step the policy chooses a with probability $\pi(a|s)$ and the environment responds with (s', r) with probability $p(s', r|s, a)$. The later return is conditioned on the whole first step, $s_t = s$, $a_t = a$, $s_{t+1} = s'$ and $r_{t+1} = r$. By the Markov property the future after $s_{t+1} = s'$ depends on none of s, a and r, and only the condition $s_{t+1} = s'$ survives. The third line recognises this expectation as $V_\pi(s')$, a step that also requires the policy and the dynamics to be independent of the time step. When the remaining time affects the best action, we include it in the state and the derivation goes through unchanged. Therefore

$$V_\pi(s) = \sum_a \pi(a \,|\, s) \sum_{s',r} p(s', r \,|\, s, a) \big[r + \gamma\, V_\pi(s') \big] \tag{6.5}$$

The Bellman equation of the policy π [19] relates the value of a state to the values of its successors one step ahead. Dropping the average over the first action, we obtain

$$\begin{aligned} Q_\pi(s,a) &= \mathbb{E}_\pi\big[r_{t+1} + \gamma G_{t+1} \,\big|\, s_t = s, a_t = a\big] = \sum_{s',r} p(s', r \,|\, s, a) \big[r + \gamma\, V_\pi(s') \big] \\ &= r(s,a) + \gamma \sum_{s'} p(s' \,|\, s, a)\, V_\pi(s'), \end{aligned} \tag{6.6}$$

where the second line uses the marginals (3.3) and (3.4). Comparing the two equations shows $V_\pi(s) = \sum_a \pi(a|s)\, Q_\pi(s,a)$. The value is the action value averaged over the actions of the policy, so the advantage averages to zero, $\sum_a \pi(a|s) A_\pi(s,a) = 0$. Figure 16 draws the structure of equation (6.5) as a backup diagram in the convention of Ref. [2].

For a finite set of states the Bellman equation is a linear system. In matrix form, with the transition matrix (3.11) and the expected reward r_π written as a vector, it reads $V_\pi = r_\pi + \gamma P^\pi V_\pi$, or

$$(\mathbb{1} - \gamma P^\pi) V_\pi = r_\pi. \tag{6.7}$$

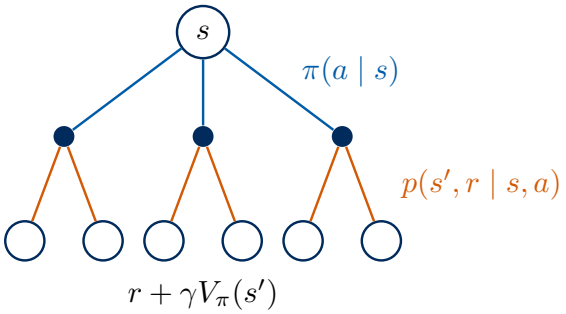


Figure 16: Backup diagram of the Bellman equation (6.5). Open circles are states and filled dots actions. From the state s the policy chooses an action with probability $\pi(a|s)$ (blue), and the environment answers with a next state s' and a reward r with probability $p(s', r|s, a)$ (orange). The value $V_\pi(s)$ is the average of $r + \gamma V_\pi(s')$ over both branchings.

The matrix runs over the non-terminal states only, since the value of a terminal state is zero, and transitions into a terminal state drop out of the rows. Each row of P^π therefore has non-negative entries that sum to at most one. For any vector v the largest component of $P^\pi v$ is at most the largest component of v in absolute value, $\|P^\pi v\|_\infty \le \|v\|_\infty$. Hence $\|\gamma P^\pi v\|_\infty \le \gamma \|v\|_\infty$, the series $\sum_k (\gamma P^\pi)^k$ converges for $\gamma < 1$, and its sum is the inverse of $\mathbb{1} - \gamma P^\pi$. The value of a policy is the unique solution of a linear system. For $\gamma = 1$ the argument fails in general. It survives when every episode ends with probability one, because then the probability of still running after k steps, the entries of $(P^\pi)^k$, decays to zero and the series converges again. Episodes of a fixed finite length belong to this case.

The decision process of Figure 6 shows the equation at work. Take the policy that chooses a_1 in s_A, and $\gamma = 0.9$. The Bellman equation (6.5) gives one equation for each of the two non-terminal states,

$$V_\pi(s_\mathrm{A}) = 0.8\,\gamma\, V_\pi(s_\mathrm{B}) + 0.2\,\gamma\, V_\pi(s_\mathrm{A}), \qquad V_\pi(s_\mathrm{B}) = 0.9 \cdot 10 + 0.1\,\gamma\, V_\pi(s_\mathrm{A}), \tag{6.8}$$

where V_π is the value of this policy. The first equation gives $V_\pi(s_\mathrm{A}) = 0.72\, V_\pi(s_\mathrm{B})/0.82$, and inserting it into the second gives $V_\pi(s_\mathrm{B}) = 9.77$ and $V_\pi(s_\mathrm{A}) = 8.58$. The random policy, which also takes a_2 in s_A with probability one half, has $V_\pi(s_\mathrm{A}) = 3.12$ and $V_\pi(s_\mathrm{B}) = 9.28$, lower in both states. The penalty -1 of a_2 ends half of the episodes from s_A without the large reward. By equation (6.6), the action values in s_A under the first policy are $Q_\pi(s_\mathrm{A}, a_2) = -1$ and $Q_\pi(s_\mathrm{A}, a_1) = V_\pi(s_\mathrm{A}) = 8.58$, so the advantages are $A_\pi(s_\mathrm{A}, a_1) = 0$ for the action the policy takes and $A_\pi(s_\mathrm{A}, a_2) = -1 - 8.58 = -9.58$ for the alternative. The negative advantage says how much worse it is to take a_2 once and then follow the policy.

7 Optimality and value iteration

7.1 Optimal policies

Values order policies. A policy π is at least as good as π', written $\pi \ge \pi'$, when $V_\pi(s) \ge V_{\pi'}(s)$ in every state. The order is partial, because one policy may be better in some states and worse in others, and two such policies are not comparable. A finite Markov decision process with bounded rewards and a discount below one nevertheless has an optimal policy π^* [2], at least as good as every other policy in every state. Two optimal policies are each at least as good as the other, so their values coincide, and the optimal values

$$V_*(s) \coloneqq V_{\pi^*}(s) = \max_\pi V_\pi(s), \qquad Q_*(s, a) \coloneqq Q_{\pi^*}(s, a) = \max_\pi Q_\pi(s, a) \tag{7.1}$$

are unique, whichever optimal policy defines them.

A policy can be improved with its own action values. Let π' be the greedy policy with respect to Q_π, which takes in every state the action $\pi'(s) = \arg\max_a Q_\pi(s, a)$. Since a maximum is at least the average, $V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s, a) \le Q_\pi(s, \pi'(s))$. Writing Q_π with equation (6.6) and applying the

same inequality to the value of the next state gives

$$\begin{aligned} V_\pi(s) &\le \mathbb{E}\big[r_{t+1} + \gamma V_\pi(s_{t+1}) \,\big|\, s_t = s, a_t = \pi'(s)\big] \\ &\le \mathbb{E}_{\pi'}\big[r_{t+1} + \gamma r_{t+2} + \gamma^2 V_\pi(s_{t+2}) \,\big|\, s_t = s\big] \le \cdots \le V_{\pi'}(s), \end{aligned} \tag{7.2}$$

where every further step follows π' for one more action. The chain of inequalities, the policy improvement theorem [2], states that the greedy policy with respect to Q_π is at least as good as π in every state. Alternating the evaluation of a policy, the linear system (6.7), with this greedy improvement is called policy iteration. A finite process has finitely many deterministic policies, and every strict improvement raises the value in at least one state. With ties resolved by retaining the current action, the alternation therefore stops after finitely many steps. When greedy improvement no longer changes the policy, $V_\pi(s) = \max_a Q_\pi(s,a)$ in every state. The values of π then satisfy the Bellman optimality equation, whose only solution is the optimal value, so the final policy is optimal. In Figure 6 one improvement step from the random policy already gives the policy that always takes a_1, since $Q(s_\mathrm{A}, a_1) > Q(s_\mathrm{A}, a_2)$, and no other policy does better.

7.2 The Bellman optimality equations

An optimal policy can always be chosen to put all its probability on a best action, so the average over actions in the Bellman equation becomes a maximum,

$$V_*(s) = \max_a \mathbb{E}\big[G_t \,\big|\, s_t = s, a_t = a, \pi^*\big] = \max_a \mathbb{E}\big[r_{t+1} + \gamma V_*(s_{t+1}) \,\big|\, s_t = s, a_t = a\big] = \max_a Q_*(s,a). \tag{7.3}$$

Inserting this into equation (6.6) for π^* gives the Bellman optimality equations [1,19].

Bellman optimality equations

$$\begin{aligned} V_*(s) &= \max_a \sum_{s',r} p(s', r \,|\, s, a)\big[r + \gamma\, V_*(s')\big], \\ Q_*(s,a) &= \sum_{s',r} p(s', r \,|\, s, a)\big[r + \gamma \max_{a'} Q_*(s', a')\big]. \end{aligned} \tag{7.4}$$

The best value of a state is reached by the best first action followed by optimal behaviour. Once Q_* is known, the optimal policy is greedy with respect to it, $a^*(s) = \arg\max_a Q_*(s,a)$.

A greedy choice is enough, because Q_* already contains the long-term consequences of each action. The optimality equations, one per state, are nonlinear because of the maximum, and solving them directly needs the model $p(s', r|s, a)$. Figure 17 shows how the maximum replaces the average over the actions in the backup diagram.

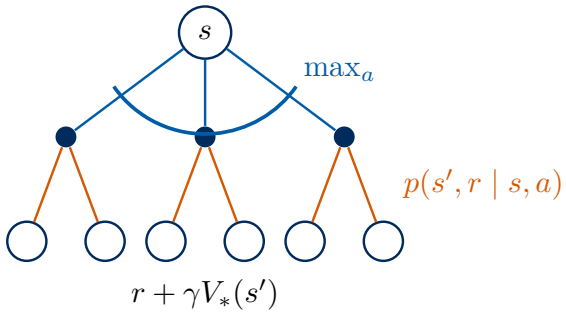


Figure 17: Backup diagram of the Bellman optimality equation for V_*, equation (7.4). The arc marks the maximum over the actions (blue), which replaces the average over the policy of Figure 16. The environment still averages over the next states and rewards (orange).

7.3 Value iteration and its contraction

Value iteration solves the optimality equation by applying its right-hand side repeatedly. Write $(\mathcal{T}V)(s) \coloneqq \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma V(s')]$ for the Bellman operator acting on a vector V of values. Starting from $V_0 = 0$ and for $\gamma < 1$, the iteration $V_{k+1} = \mathcal{T}V_k$ converges to V_*, the fixed point $\mathcal{T}V_* = V_*$. For two arbitrary vectors V and V' and any state s,

$$|(\mathcal{T}V)(s) - (\mathcal{T}V')(s)| \le \max_a \left|\gamma \sum_{s'} p(s'|s, a)\,[V(s') - V'(s')]\right| \le \gamma \max_{s'} |V(s') - V'(s')|. \tag{7.5}$$

The first inequality uses $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$, the rewards cancel in the difference, and the second inequality holds because the probabilities $p(s'|s, a)$ sum to at most one. With the maximum norm $\|V\|_\infty = \max_s |V(s)|$ this reads

$$\|\mathcal{T}V - \mathcal{T}V'\|_\infty \le \gamma \,\|V - V'\|_\infty, \qquad \text{hence} \qquad \|V_k - V_*\|_\infty \le \gamma^k \,\|V_0 - V_*\|_\infty, \tag{7.6}$$

where the second statement follows by applying the first with $V' = V_*$ k times. The Bellman operator is therefore a contraction with factor γ. Its fixed point is unique, and the error of value iteration shrinks at least by γ per iteration. The same argument without the maximum shows that the iteration $V_{k+1} = r_\pi + \gamma P^\pi V_k$ converges to V_π, which gives a second way to solve the linear system (6.7). Appendix G gives a stopping criterion for value iteration based on the measurable Bellman residual.

The factor γ is a bound, and the actual speed can be much higher. Suppose that after some iteration the greedy action in every state no longer changes. From then on the maximum in $\mathcal{T}$ always picks the action of π^*, and the error $e_k = V_k - V_*$ obeys the linear recursion $e_{k+1} = \gamma P^{\pi^*} e_k$, so that $e_k = (\gamma P^{\pi^*})^k e_0$. The error then decreases per iteration by $\gamma\Lambda$, where Λ is the largest eigenvalue of P^{π^*} restricted to the non-terminal states. This eigenvalue is the rate at which the probability of not yet having ended decays under the optimal policy. When the optimal policy ends episodes quickly, Λ is small and the convergence is much faster than the bound. Before its error starts to fall, value iteration also needs about as many iterations as there are steps between a state and the reward. Each iteration carries the information about the reward one step further back (Figure 18).

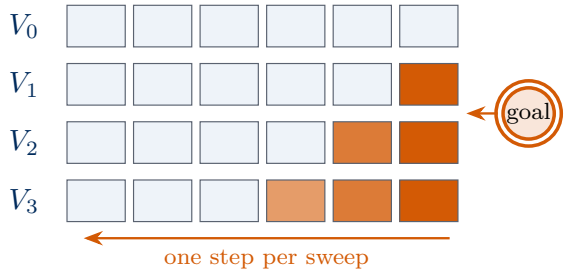


Figure 18: Value iteration on a chain of six states whose right end leads to the goal, starting from $V_0 = 0$. Each row is one iterate V_k, and a coloured cell is a state whose value already contains the reward of the goal. The k-th sweep reaches the state k steps from the goal, so the information travels one step per iteration. The chain is an illustration.

8 Learning values from experience

8.1 Two sampled targets

Value iteration used the model. Without it the agent must estimate $V_\pi(s) = \mathbb{E}_\pi[G_t|s_t = s]$ from sampled trajectories alone. From here on V denotes such a learned estimate of V_π, and V_t its current value after t updates. A learning rule moves the estimate of a visited state towards a target, a number computed from experience that stands in for the unknown $V_\pi(s_t)$, and the rules below differ in this target.

The Monte Carlo target is the observed return G_t of the rest of the episode. It is an unbiased sample of $V_\pi(s_t)$, but it becomes available only when the episode ends, and its noise collects the randomness of every later action and outcome. The temporal-difference (TD) target $r_{t+1} + \gamma V(s_{t+1})$ adds one observed reward to the current estimate of the continuation. After one transition it is already available and carries the noise of one step only, while any error of the current estimate $V(s_{t+1})$ passes into it. Figure 19 compares the two.

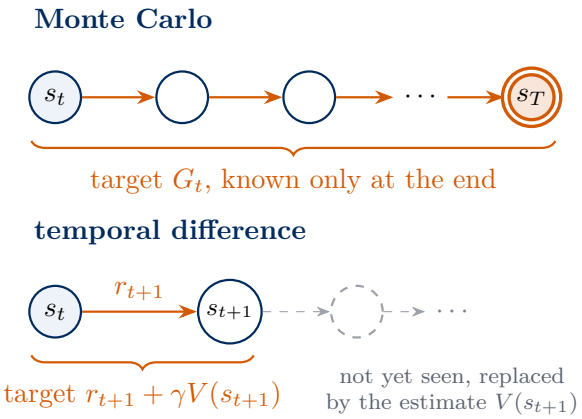


Figure 19: The targets of the two updates of the value $V(s_t)$. Top, Monte Carlo waits for the end of the episode and uses the observed return G_t. Bottom, temporal difference uses one step, the reward r_{t+1}, and replaces the unseen rest of the episode by the current estimate $V(s_{t+1})$.

8.2 Monte Carlo updates

After each episode, Monte Carlo evaluation moves the estimate of every visited state a fraction α, the learning rate, towards the observed return [1,2],

$$V(s_t) \leftarrow V(s_t) + \alpha[G_t - V(s_t)]. \tag{8.1}$$

A state visited several times in one episode can be updated at its first visit only or at every visit. With the step size $\alpha = 1/n$ at the n-th update of a state, the estimate is the sample average of the observed returns, and both versions converge to V_π [2]. Decreasing step sizes with $\sum_n \alpha_n = \infty$ and $\sum_n \alpha_n^2 < \infty$ also give convergence, while a constant α leaves the estimate fluctuating around V_π. The update can be made only at the end of an episode, which is slow for long or endless episodes.

8.3 Temporal-difference updates and the Bellman residual

The TD target is built from the Bellman equation (6.5). Write $\mathcal{T}_\pi$ for the fixed-policy Bellman operator,

$$(\mathcal{T}_\pi V)(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]. \tag{8.2}$$

The operator takes a proposed set of values V and returns the expected one-step target computed from them, and the true policy value satisfies $V_\pi = \mathcal{T}_\pi V_\pi$. With a known model we can evaluate the sums, and without it an observed transition supplies one sample,

$$Y_t = r_{t+1} + \gamma V_t(s_{t+1}), \qquad \delta_t = Y_t - V_t(s_t). \tag{8.3}$$

Here V_t is the current learned estimate. Holding it fixed and averaging over the action chosen by the policy and the outcome generated by the environment gives

$$\mathbb{E}_\pi[Y_t \mid s_t = s, V_t] = (\mathcal{T}_\pi V_t)(s), \tag{8.4}$$

$$\mathbb{E}_\pi[\delta_t \mid s_t = s, V_t] = (\mathcal{T}_\pi V_t)(s) - V_t(s). \tag{8.5}$$

The second expression is the Bellman residual at the visited state. Temporal-difference learning uses a noisy sample of this residual to change the estimate,

$$V_{t+1}(s_t) = V_t(s_t) + \alpha_t \delta_t. \tag{8.6}$$

All other table entries remain unchanged on that transition. The visited entry moves towards its Bellman target in expectation, although a single update can move in the wrong direction because the observed outcome is random.

8.4 A transition in the running decision process

In the decision process of Figure 6, the state s_{B} has one action. With probability 0.9 it terminates with reward 10, and with probability 0.1 it returns to s_{A} with reward zero. At $\gamma = 0.9$, the Bellman equation for a fixed policy is

$$V_\pi(s_{\mathrm{B}}) = 0.9 \cdot 10 + 0.1[0 + 0.9 V_\pi(s_{\mathrm{A}})]. \tag{8.7}$$

The learner does not observe this average on any single visit. Its sampled target is

$$Y_t = \begin{cases} 10, & \text{with probability } 0.9, \\ 0.9 V_t(s_{\mathrm{A}}), & \text{with probability } 0.1. \end{cases} \tag{8.8}$$

The mean of these targets is $9 + 0.09 V_t(s_{\mathrm{A}})$, the right-hand side of the Bellman equation evaluated with the current estimate. Repeated observations estimate that average. Updating $V(s_{\mathrm{A}})$ changes the continuation target used at s_{B}, so both entries gradually become mutually consistent.

At the solution individual temporal-difference errors remain nonzero, because the terminal outcome and the return to s_{A} still give different targets, and only their conditional mean is zero. Bellman consistency is therefore a statement about expected targets, which is assessed with averages or with the residual of a known model.

The estimate of the next state is a provisional value inside the target. Learning revises these provisional values until their expected one-step predictions agree with one another and with the observed rewards. Appendix G shows why the averaged update converges and what sampling adds.

8.5 Returns, estimation errors and a finite horizon

Monte Carlo and temporal difference estimate the same fixed-policy value with different targets. Let $e(s) = V(s) - V_\pi(s)$ be the current value error, with the current estimates held fixed while sampling a transition. The Bellman equation gives

$$\mathbb{E}_\pi[r_{t+1} + \gamma V(s_{t+1}) \mid s_t = s] - V_\pi(s) = \gamma \sum_{s'} P^\pi(s, s') e(s'), \tag{8.9}$$

$$\mathbb{E}_\pi[\delta_t \mid s_t = s] = \gamma \sum_{s'} P^\pi(s, s') e(s') - e(s). \tag{8.10}$$

The first expression is the bias of the temporal-difference target and the second its mean correction. The Monte Carlo return has zero target bias under the fixed policy, provided it is integrable.

For a realised trajectory we keep V frozen and set $V(s_T) = 0$. Expanding and cancelling the intermediate values gives

$$\sum_{k=t}^{T-1} \gamma^{k-t} \delta_k = \sum_{k=t}^{T-1} \gamma^{k-t} [r_{k+1} + \gamma V(s_{k+1}) - V(s_k)] \tag{8.11}$$

$$= G_t - V(s_t). \tag{8.12}$$

The Monte Carlo prediction error is the sum of discounted temporal-difference errors along that trajectory. The equality requires the same frozen value function in every term, and online changes to V introduce additional terms.

Repeated substitution in the return recursion also gives an n-step target,

$$Y_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j r_{t+j+1} + \gamma^n V(s_{t+n}). \tag{8.13}$$

At termination the continuation is zero and the reward sum stops. One-step temporal difference uses $n = 1$, and a target extending to termination is the Monte Carlo return. These targets therefore connect the two methods through the amount of sampled experience used before bootstrapping [2].

8.6 The remaining budget as a state variable

In a finite-budget task, identical physical states can have different values when they have different numbers of actions left. With x the physical state and h the remaining action budget, the Markov state is (x, h). With zero reward after budget exhaustion, backward induction obeys

$$V_{*,0}(x) = 0, \tag{8.14}$$

$$Q_{*,h}(x, a) = \sum_{x',r} p(x', r \mid x, a)[r + \gamma V_{*,h-1}(x')], \qquad h \geq 1, \tag{8.15}$$

$$V_{*,h}(x) = \max_a Q_{*,h}(x, a). \tag{8.16}$$

Success termination also has zero continuation. The optimal policy can depend on h, even when the physical transition law is time independent, and the input of a network can include h next to the physical features. A sampling cutoff in a continuing task differs, since the task continues after collection stops and its last transition keeps a continuation estimate.

8.7 SARSA and Q-learning

To choose actions without a model the agent needs action values, since V alone does not say which action leads to a good next state. The same TD update for action values, with the next action a_{t+1} chosen by the policy, is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big], \tag{8.17}$$

called SARSA after the five quantities $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ it uses [2]. It learns the action values of the policy that generates the data, for example an ε-greedy policy, including the cost of its random actions. Methods of this kind are called on-policy.

Q-learning learns the optimal action values directly [26]. It replaces the expectation over s' in the optimality equation (7.4) by the sampled next state and moves the estimate towards the sampled target after every transition (s, a, r, s'),

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\Big], \tag{8.18}$$

with the target r alone when s' is terminal. The values are kept in a table with one entry per state and action. A behaviour policy, usually ε-greedy with respect to the current table, chooses the actions, while the target uses the best next action whatever the agent actually takes next (Figure 20). Under the standard tabular conditions, repeated visits to every state–action pair and suitably decreasing learning rates, the table therefore approaches Q_* even when the behaviour policy explores, and methods of this kind are called off-policy.

Q-learning

1. Initialise $Q(s, a)$ for all states and actions, with $Q = 0$ in terminal states.
2. Start every episode in s_0. At every step choose a ε-greedily from $Q(s, \cdot)$, observe r and s',

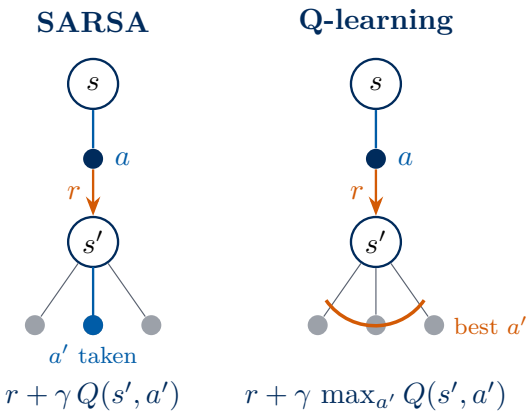


Figure 20: The targets of SARSA, equation (8.17), and Q-learning, equation (8.18), after the same transition from s with the action a and the reward r to s'. SARSA uses the value of the next action a' that the agent actually takes. Q-learning uses the largest value over all next actions (arc), whatever the agent does next.

apply the update (8.18) and continue from s' until the episode ends.

3. Return the greedy policy $a^*(s) = \arg\max_a Q(s, a)$.

The agent learns from the outcomes of its own actions alone and never uses the probabilities $p(s', r|s, a)$.

In the running decision process of Figure 6 the rows of the table are the states s_A and s_B and the columns the actions a_1 and a_2. The state s_B offers only a_1, and the terminal state s_C needs no row because its values are zero. Suppose that learning has produced the entries on the left of Table 1, and take $\gamma = 0.9$ and $\alpha = 0.2$. The agent in s_B takes a_1, receives $r = 10$ and reaches the terminal state. The target is $y = 10$ with no continuation, and the entry moves to $Q(s_B, a_1) = 3 + 0.2\,(10 - 3) = 4.4$. Later the agent in s_A takes a_1, receives $r = 0$ and arrives in s_B. The target now bootstraps from the updated row, $y = 0 + 0.9 \max_{a'} Q(s_B, a') = 0.9 \cdot 4.4 = 3.96$, and the entry moves to $Q(s_A, a_1) = 2 + 0.2\,(3.96 - 2) = 2.392$. Each transition changes one entry, and the reward of the terminal transition reaches s_A through the second update.

	before		after $(s_B, a_1, 10, s_C)$		after $(s_A, a_1, 0, s_B)$	
state	a_1	a_2	a_1	a_2	a_1	a_2
s_A	2	-1	2	-1	**2.392**	-1
s_B	3	–	**4.4**	–	4.4	–

Table 1: A Q-table of the running decision process with states as rows and actions as columns, before and after two Q-learning updates with $\gamma = 0.9$ and $\alpha = 0.2$. The changed entry is in bold. The state s_B has no action a_2. The entries before the updates are an illustration.

8.8 Q-learning as a sampled Bellman optimality update

For control, the unknown function is Q_*. Its Bellman operator is

$$(\mathcal{T}_*^Q Q)(s, a) = \sum_{s', r} p(s', r \mid s, a)[r + \gamma \max_{a'} Q(s', a')]. \tag{8.19}$$

The maximum states the continuation objective. After the first action the agent should choose an action with the largest return estimate, while the outcomes of the environment are still averaged.

For one sampled transition, the Q-learning target and error are

$$Y_t = r_{t+1} + \gamma \max_{a'} Q_t(s_{t+1}, a'), \tag{8.20}$$

$$\delta_t^Q = Y_t - Q_t(s_t, a_t). \tag{8.21}$$

Terminal continuation is zero. Holding Q_t and the visited pair fixed while averaging over environment outcomes gives

$$\mathbb{E}[Y_t \mid s_t = s, a_t = a, Q_t] = (\mathcal{T}_*^Q Q_t)(s,a), \tag{8.22}$$

$$\mathbb{E}[\delta_t^Q \mid s_t = s, a_t = a, Q_t] = (\mathcal{T}_*^Q Q_t)(s,a) - Q_t(s,a). \tag{8.23}$$

The Q-learning update therefore follows the sampled residual of the Bellman optimality equation.

The behaviour policy decides which pairs supply data and can choose exploratory actions while the target continues to use the greedy maximum. Q-learning is off-policy in this sense, since data collection is separated from the policy described by the target.

The same argument as in equation (7.5) shows that $\mathcal{T}_*^Q$ contracts with factor $\gamma < 1$ in the maximum norm. For a finite table with bounded rewards, every relevant pair visited infinitely often, and per-pair step sizes satisfying

$$\sum_n \alpha_n(s,a) = \infty, \qquad \sum_n \alpha_n(s,a)^2 < \infty, \tag{8.24}$$

stochastic Q-learning converges to this fixed point under the standard tabular assumptions [26,2]. The conditions retain enough total correction to overcome initial errors while reducing the accumulated noise. They demand more than applying the update many times.

A neural network changes this argument. One parameter update can change many state-action estimates, and fitting a sampled target need not preserve the Bellman contraction.

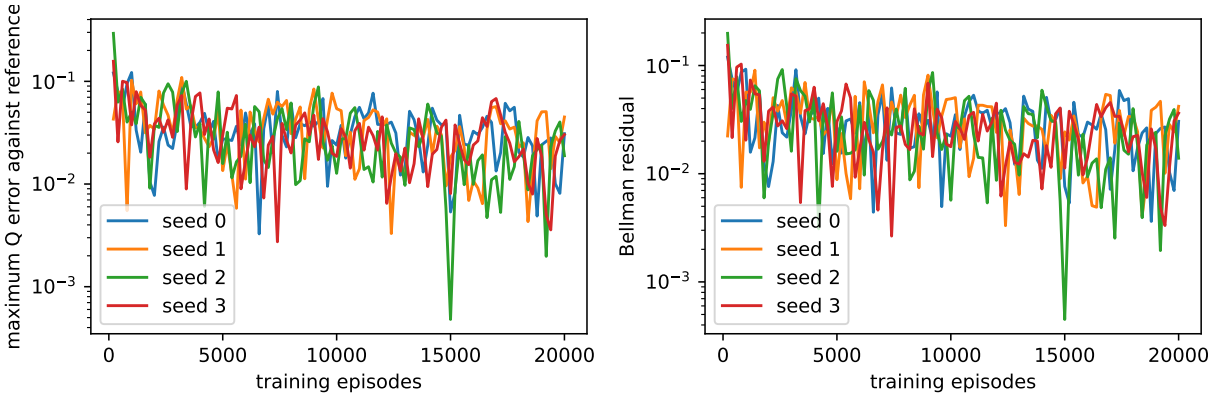


Figure 21: Learning the running three-state process from sampled transitions. Left, the maximum error against the Bellman reference. Right, the known-model Bellman residual of the current Q-table. Curves show independent seeds. The model is used to measure error and residual, while the learner updates its table using only sampled transitions. Finite-run fluctuations remain.

8.9 Double Q-learning

The maximum in the Q-learning target is taken over estimates that contain noise, and the maximum of noisy numbers is biased upwards [1,2,27,28]. Let two actions have the same true value 0, and let their estimates X_1 and X_2 be independent Gaussian numbers with mean 0 and standard deviation σ_e. The true maximum is 0, and the expected maximum of the estimates is positive. With $\max(X_1, X_2) = \frac{1}{2}(X_1 + X_2) + \frac{1}{2}|X_1 - X_2|$ and the difference $\Delta X = X_1 - X_2$, a Gaussian with variance $2\sigma_e^2$,

$$\mathbb{E}\max(X_1, X_2) = \tfrac{1}{2}\mathbb{E}|\Delta X| = \frac{1}{2} \cdot \sqrt{2}\,\sigma_e \sqrt{\frac{2}{\pi}} = \frac{\sigma_e}{\sqrt{\pi}} \approx 0.56\,\sigma_e, \tag{8.25}$$

where $\mathbb{E}|\Delta X| = \sqrt{2/\pi} \cdot (\text{standard deviation of } \Delta X)$ for a Gaussian of zero mean. With more actions the bias grows. In Q-learning the biased maximum enters the target and is stored in $Q(s,a)$. It is then passed on to the states that lead to s, which can make the agent prefer actions whose values are merely uncertain.

Double Q-learning separates the choice of the best next action from the evaluation of its value [28]. The agent keeps two tables Q^A and Q^B. At every step one of them, chosen at random, is updated, and its target uses the other table to evaluate the action that it considers best,

$$Q^A(s,a) \leftarrow Q^A(s,a) + \alpha\Big[r + \gamma\, Q^B\big(s', \arg\max_{a'} Q^A(s',a')\big) - Q^A(s,a)\Big], \tag{8.26}$$

and the same rule with A and B exchanged. The behaviour policy uses both tables, for example their sum. When Q^B is unbiased and its errors are independent of the choice made with Q^A, the expected value of Q^B at the chosen action equals the true value of that action. The choice then no longer selects positive errors, and in the example of two actions with equal true values the double estimate has mean zero. For four such actions with unit noise a simulation gives the single maximum at 1.03 and the double estimate at zero within the sampling error. When the true values differ, the double estimate can lie slightly below the largest true value, since a noisy Q^A sometimes chooses an action that is not the best one [28]. The method needs twice the memory, and each table sees about half of the experience.

The textbook example of the maximisation bias shows the effect [2, Example 6.7]. It is a small abstract decision process with two states, s_{start} and s_{detour}, and an end. In s_{start} the agent chooses between a safe route, which ends the episode with the reward 0, and a detour, which leads to s_{detour} with the reward 0. In s_{detour} the agent chooses one of ten actions, and each ends the episode with a Gaussian random reward of mean -0.1 and standard deviation 1. With $\gamma = 1$ the true action values in s_{start} are 0 for the safe route and -0.1 for the detour. An ε-greedy agent with the right values and $\varepsilon = 0.1$ takes the detour only through its random choice, with probability $\varepsilon/2 = 0.05$. The value of the detour is learned from $\max_{a'} Q(s_{\text{detour}}, a')$, the maximum over ten noisy estimates of actions whose true values are all -0.1. Early in learning this maximum is positive. In our reimplementation with $\alpha = 0.1$, over 10 000 seeds and 300 episodes (Figure 22), Q-learning takes the detour in up to a fraction 0.94 of the runs. It does so in a fraction 0.86 on average over the first 50 episodes, and still in 0.126 over the last 50. Both algorithms start at 0.5, since all values start at zero and ties are broken at random. Double Q-learning takes the detour in 0.29 of the runs over the first 50 episodes and in 0.07 over the last 50, close to the optimal level.

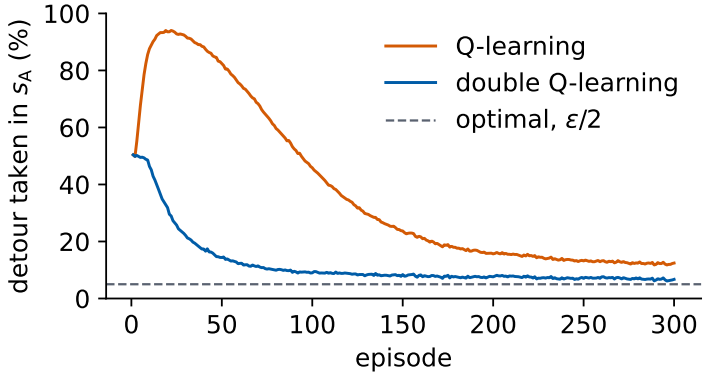


Figure 22: The detour task of the maximisation bias. The percentage of 10 000 runs that take the detour in s_{start}, against the episode, for Q-learning and for double Q-learning. The axis labels the start state s_{A}, as in the textbook. The dashed line is the level $\varepsilon/2$ of an ε-greedy agent with the true values.

9 Deep Q-learning

9.1 The Q-network as a regression

A table with one entry per state and action is possible for a few thousand states and impossible for a state that takes a continuum of values, such as the state vector of several qubits. Deep Q-learning replaces the table by a neural network $Q(s,a;\theta)$ with parameters θ, the deep Q-network (DQN) [7].

The network takes a numerical representation of the state as input and has one output per action, so one evaluation gives the action values of all actions. Learning the action values becomes a regression in which, for a transition (s, a, r, s'), the Q-learning target $y = r + \gamma \max_{a'} Q(s', a'; \theta)$ plays the role of the label. The network is fitted by gradient descent on the mean squared error $\frac{1}{2}[y - Q(s, a; \theta)]^2$, with the target held fixed when the gradient is taken,

$$\theta \leftarrow \theta + \alpha \big[y - Q(s, a; \theta)\big] \nabla_\theta Q(s, a; \theta). \tag{9.1}$$

For a table, where $Q(s, a; \theta)$ is a single parameter, the update reduces to the Q-learning rule (8.18).

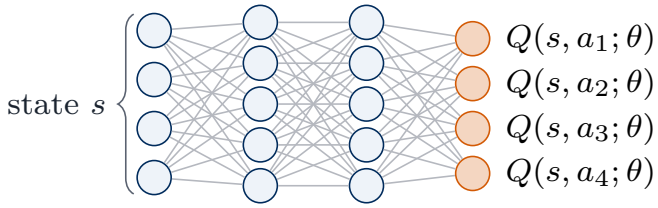


Figure 23: A Q-network. The state s enters as a vector of numbers (left), followed by two hidden layers. The network has one output $Q(s, a_k; \theta)$ for each action a_k (orange), so one evaluation gives the action values of all actions. The greedy action is the output with the largest value.

Applied directly, this regression is unstable for three reasons [1,2]. Consecutive transitions of one trajectory are strongly correlated, which can make stochastic gradient updates inefficient and unstable. The target depends on the same parameters θ that are being fitted, so every update also moves the label. Finally, a small change of θ can change which action has the largest value, and with it the actions of the ε-greedy agent and the distribution of the data. Two devices address these problems (Figure 24). The network that is trained and that chooses the actions is called the online network, with parameters θ. Experience replay stores the transitions (s, a, r, s') in a memory and trains the network on random batches drawn from it, which reduces the correlations between consecutive training samples and reuses each experience several times [34]. A target network computes the targets, so that the label stays fixed while the network fits it [7]. Its parameters θ^- are a copy of θ, refreshed periodically, every 10^4 updates in the Atari agent. Both devices soften the third problem, the shift of the data when the greedy action changes. A large memory mixes data of many earlier policies, and the slowly updated targets change the greedy action less abruptly. The shift persists, however, and the training of a deep Q-network can still oscillate.

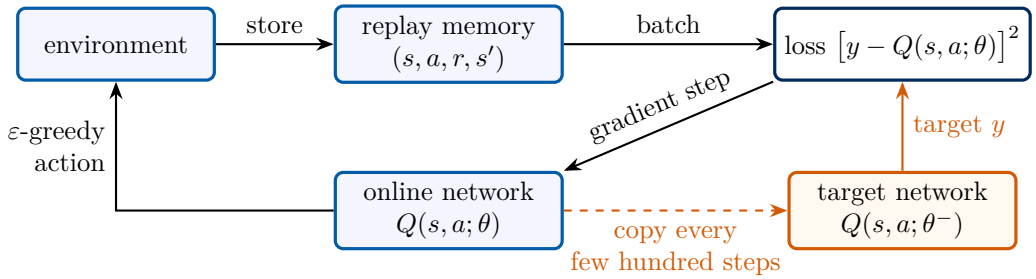


Figure 24: Deep Q-learning. The agent acts ε-greedily with the online network and stores its transitions in a replay memory. It fits the online network on random batches to targets computed with a target network, an older copy of the online network.

9.2 Overestimation and the Double DQN loss

The overestimation bias of Section 8.9 is present in deep Q-learning as well, and the target network offers a second estimator without extra cost. Double DQN (DDQN) chooses the best next action with the online network and evaluates it with the target network [35]. For a batch of B transitions

(s_i, a_i, r_i, s_i') drawn from the memory the loss is [1,2]

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B}\Big[r_i + \gamma\,(1-d_i)\,Q\big(s_i', \arg\max_{a'} Q(s_i', a'; \theta); \theta^-\big) - Q(s_i, a_i; \theta)\Big]^2, \tag{9.2}$$

where $d_i = 1$ when s_i' is terminal and $d_i = 0$ otherwise. Plain DQN uses $\max_{a'} Q(s_i', a'; \theta^-)$ in place of the evaluated term, so its target is never smaller than the DDQN target computed with the same networks. The two networks of DDQN are only partly independent, since one is an older copy of the other, so the overestimation is reduced and part of it can remain. Many implementations replace the square by the Huber function, quadratic for small and linear for large errors, which limits the effect of rare large errors.

Deep Q-learning and Double DQN

1. Initialise the online network $Q(s, a; \theta)$, copy it to the target network, $\theta^- \leftarrow \theta$, and start with an empty replay memory.
2. In the current state s choose a ε-greedily from $Q(s, \cdot\,; \theta)$, apply it, observe r, s' and whether s' is terminal ($d = 1$ or 0), and store (s, a, r, s', d) in the memory.
3. Draw a random batch of B stored transitions and compute the targets $y_i = r_i + \gamma(1 - d_i)\max_{a'} Q(s_i', a'; \theta^-)$ for DQN, or $y_i = r_i + \gamma(1 - d_i)\,Q(s_i', \arg\max_{a'} Q(s_i', a'; \theta); \theta^-)$ for Double DQN.
4. Take one gradient step on $\frac{1}{B}\sum_i [y_i - Q(s_i, a_i; \theta)]^2$ with respect to θ, the targets held fixed.
5. Every few hundred steps copy $\theta^- \leftarrow \theta$, lower ε according to its schedule, start a new episode when s' is terminal, and return to step 2.

9.3 The gradient of a replay update

For a stored transition i, let $q_i = Q(s_i, a_i; \theta)$ and let y_i be the DQN or Double DQN target computed before the update. Define the regression loss with a factor of one half,

$$L(\theta) = \frac{1}{2B}\sum_i [q_i - y_i]^2, \tag{9.3}$$

$$\nabla_\theta L = \frac{1}{B}\sum_i (q_i - y_i)\nabla_\theta Q(s_i, a_i; \theta), \tag{9.4}$$

$$\theta_{\text{new}} = \theta + \frac{\alpha}{B}\sum_i (y_i - q_i)\nabla_\theta Q(s_i, a_i; \theta). \tag{9.5}$$

The observed action selects one output of the network for each transition. The targets are held fixed, including the selection performed by the online network in Double DQN, which automatic differentiation implements by stopping the gradient through y_i. Differentiating through the target as well would minimise the squared Bellman residual itself, a different optimisation procedure with different fixed points and stability.

For the same next state, let u_a and v_a denote the outputs of the online and target networks. Writing $a_u = \arg\max_a u_a$ gives

$$y_{\text{DQN}} = r + \gamma(1 - d)\max_a v_a, \tag{9.6}$$

$$y_{\text{DDQN}} = r + \gamma(1 - d)v_{a_u}, \tag{9.7}$$

$$y_{\text{DQN}} - y_{\text{DDQN}} = \gamma(1 - d)[\max_a v_a - v_{a_u}] \geq 0. \tag{9.8}$$

This ordering holds for targets formed from these particular networks and transition and implies no ordering of their eventual learned policies. A terminal transition has $d = 1$, so both targets reduce to the observed reward.

Replay changes the sampling distribution of regression updates, and a target network changes the rate at which the labels move. The combination of function approximation, bootstrapped targets and off-policy data can make the iteration diverge even where each ingredient alone is stable [2]. Evaluation therefore uses a frozen policy and independent evaluation episodes. A small regression loss by itself establishes only agreement with the current labels, which can themselves be wrong.

10 Policy gradient

10.1 A parametrised policy

Value-based methods learn values and derive the policy from them as the greedy choice, a maximum over actions. Policy-gradient methods learn the policy directly [1,2]. The policy $\pi_\theta(a|s)$ is a differentiable function of parameters θ, for example a neural network, in the same spirit as a variational wave function whose parameters are optimised. Finding the optimal policy becomes finding parameters θ^* with $\pi_{\theta^*} \approx \pi^*$. A policy of this kind is stochastic, so it explores by itself, and it can become nearly deterministic as it learns. It also handles continuous actions, for which the maximum over actions of a value-based method would be an optimisation problem of its own at every step. The parameters are improved by gradient ascent on the expected return

$$\mathcal{J}(\theta) \coloneqq \mathbb{E}\big[G \,\big|\, \pi_\theta\big], \tag{10.1}$$

averaged over the trajectories that the policy produces from a given initial state or distribution.

10.2 The policy gradient theorem

The expected return depends on θ through the trajectories that the policy produces, and these trajectories depend on the unknown dynamics. The policy gradient theorem shows that the gradient nevertheless can be estimated from sampled trajectories alone [31,32]. We derive it for the return of a whole episode [1,2]. With the trajectory probability (3.12) written for the parametrised policy,

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \,|\, s_t)\, p(s_{t+1}, r_{t+1} \,|\, s_t, a_t), \tag{10.2}$$

where $p_0(s_0)$ is a fixed initial-state distribution, independent of θ, with a prescribed initial state as its point-mass case. The environment dynamics and the reward rule do not depend on the policy parameters either. In the expected return $\mathcal{J}(\theta) = \sum_\tau p_\theta(\tau)\, G(\tau)$ the return of a given trajectory is independent of θ, and the gradient acts only on the probability. For any positive differentiable function, $\nabla_\theta p_\theta = p_\theta \nabla_\theta \ln p_\theta$, the log-derivative identity, and therefore

$$\nabla_\theta \mathcal{J} = \sum_\tau G(\tau)\, \nabla_\theta p_\theta(\tau) = \sum_\tau p_\theta(\tau)\, G(\tau)\, \nabla_\theta \ln p_\theta(\tau) = \mathbb{E}\big[G(\tau)\, \nabla_\theta \ln p_\theta(\tau)\big]. \tag{10.3}$$

The logarithm turns the product (10.2) into a sum,

$$\ln p_\theta(\tau) = \ln p_0(s_0) + \sum_{t=0}^{T-1} \ln \pi_\theta(a_t \,|\, s_t) + \sum_{t=0}^{T-1} \ln p(s_{t+1}, r_{t+1} \,|\, s_t, a_t). \tag{10.4}$$

The gradients of the initial-distribution term and of the environment sum vanish, since neither depends on θ, and only the policy terms remain,

$$\nabla_\theta \mathcal{J} = \mathbb{E}\Big[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \ln \pi_\theta(a_t \,|\, s_t)\Big] \tag{10.5}$$

Only the choices of the agent remain (Figure 25). The vector $\nabla_\theta \ln \pi_\theta(a|s)$ is called the score function, and the gradient of the expected return is the expected return weighted by the scores of the actions

that produced it. We estimate this expectation by sampling trajectories with the current policy, in a simulation or in an experiment, without a model of the environment and without a derivative of the reward. For continuous actions the sums over actions become integrals and the probability of an action becomes its density.

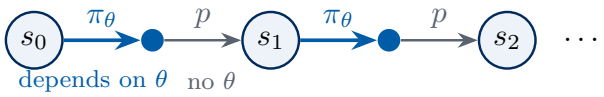


Figure 25: A trajectory as alternating choices of the agent (blue, the policy π_θ) and answers of the environment (grey, the dynamics p). Only the blue factors of the trajectory probability (10.2) depend on the parameters θ, so only they survive in the gradient (10.5).

10.3 Softmax policies and the fluctuation identity

A policy over discrete actions is commonly written as a softmax of action preferences $z(s,a)$, also called logits,

$$\pi_\theta(a \mid s) = \frac{e^{z_\theta(s,a)}}{\sum_{a'} e^{z_\theta(s,a')}}, \tag{10.6}$$

where the preferences are computed from the parameters [1,2]. In the simplest case they are linear in a feature vector of the state and action, and in general a network with one output per action computes them. The score of the softmax policy is

$$\nabla_\theta \ln \pi_\theta(a|s) = \nabla_\theta z(s,a) - \sum_{a'} \pi_\theta(a'|s) \nabla_\theta z(s,a'), \tag{10.7}$$

the deviation of the gradient of the preference from its average over the policy.

The identity behind equation (10.3), which equates the derivative of an average with its correlation with the derivative of the log-probability, is the fluctuation identity of statistical mechanics. In the canonical ensemble with $p_\beta(E) = e^{-\beta E}/\mathcal{Z}_\beta$ and the partition function $\mathcal{Z}_\beta = \sum e^{-\beta E}$, the derivative of the log-probability is $\partial_\beta \ln p_\beta = -E - \partial_\beta \ln \mathcal{Z}_\beta = -(E - \langle E \rangle)$, because $\partial_\beta \ln \mathcal{Z}_\beta = -\langle E \rangle$. The same steps as in equation (10.3) then give

$$\partial_\beta \langle E \rangle = \langle E \, \partial_\beta \ln p_\beta \rangle = -\big(\langle E^2 \rangle - \langle E \rangle^2\big) = -\operatorname{Var} E. \tag{10.8}$$

The correspondence is exact, since the softmax policy (10.6) is a Boltzmann distribution over the actions with the energies $-z(s,a)$ at unit temperature. Its score has the same form as $-(E - \langle E \rangle)$, a deviation from the average.

10.4 REINFORCE and the reward-to-go

REINFORCE estimates the gradient (10.5) from a batch of B sampled trajectories and takes a step of gradient ascent [32],

$$\theta \leftarrow \theta + \alpha \, \frac{1}{B} \sum_{i=1}^{B} G(\tau_i) \sum_{t=0}^{T_i - 1} \nabla_\theta \ln \pi_\theta(a_t^{(i)} \mid s_t^{(i)}). \tag{10.9}$$

Actions that occurred in trajectories with a high return become more likely, and those in trajectories with a low return less likely (Figure 26). In automatic differentiation the update is written as a step of gradient descent on the loss $-\frac{1}{B} \sum_i G(\tau_i) \sum_t \ln \pi_\theta(a_t^{(i)}|s_t^{(i)})$, whose negative gradient is the estimate (10.9). This loss is a cross-entropy in which the taken action is the label and the return its weight [1,2].

The estimate (10.9) multiplies every score by the return of the whole episode, including the rewards that came before the action. These earlier rewards contribute nothing on average, and dropping them makes the noisy estimate less noisy without changing its expectation. For a reward

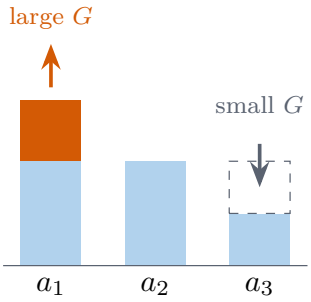


Figure 26: One REINFORCE step for a policy over three actions. The light bars are the probabilities before the step. The action a_1 appeared in an episode with a large return and becomes more probable (orange). The action a_3 appeared in an episode with a small return and becomes less probable (dashed).

$r_{t'+1}$ and a later action, $t > t'$, we condition on the beginning of the trajectory up to the state s_t, $\tau_{\le t} = (s_0, a_0, r_1, \dots, r_t, s_t)$, which contains $r_{t'+1}$,

$$\mathbb{E}\big[r_{t'+1}\nabla_\theta \ln \pi_\theta(a_t|s_t) \,\big|\, \tau_{\le t}\big] = r_{t'+1}\sum_a \pi_\theta(a|s_t)\,\nabla_\theta \ln \pi_\theta(a|s_t) = r_{t'+1}\,\nabla_\theta \sum_a \pi_\theta(a|s_t) = 0, \tag{10.10}$$

because the probabilities sum to one for every θ. Removing these terms from $G(\tau)\sum_t \nabla \ln \pi_\theta(a_t|s_t) = \sum_{t,t'} \gamma^{t'} r_{t'+1} \nabla \ln \pi_\theta(a_t|s_t)$ leaves only the terms with $t' \ge t$. In the plane of the two indices these terms fill a triangle, and summing it first over t' at fixed t gives $\sum_{t' \ge t} \gamma^{t'} r_{t'+1} = \gamma^t G_t$,

$$\nabla_\theta \mathcal{J} = \mathbb{E}\Big[\sum_{t=0}^{T-1} \gamma^t\, G_t\, \nabla_\theta \ln \pi_\theta(a_t \,|\, s_t)\Big]. \tag{10.11}$$

Each action is now weighted only with the reward that follows it, the reward-to-go G_t, and the estimate has fewer noisy terms at an unchanged expectation. Past rewards drop out much as the past drops out of the future of a Markov process. Since $Q_\pi(s_t, a_t) = \mathbb{E}[G_t|s_t, a_t]$, the reward-to-go is a sample of the action value, and equation (10.11) can equally be written with $Q_\pi(s_t, a_t)$ in place of G_t [2].

10.5 The baseline

Returns vary strongly between episodes, and so does the estimate (10.11). A baseline $b(s_t)$, any function of the state that does not depend on the action, can be subtracted from the reward-to-go,

$$\nabla_\theta \mathcal{J} = \mathbb{E}\Big[\sum_{t=0}^{T-1} \gamma^t (G_t - b(s_t))\nabla_\theta \ln \pi_\theta(a_t \,|\, s_t)\Big], \tag{10.12}$$

without changing the expectation [1,2], because the added term vanishes on average. We split the expectation into the part of the trajectory up to s_t, written $\mathbb{E}_{\tau_{\le t}}$, and the rest. Summing first over the action a_t, then over the next state and finally over the remaining trajectory gives

$$\begin{aligned}
&\mathbb{E}\big[b(s_t)\nabla_\theta \ln \pi_\theta(a_t|s_t)\big] \\
&\quad= \mathbb{E}_{\tau_{\le t}}\Big[b(s_t)\sum_{a_t} \pi_\theta(a_t|s_t)\,\nabla_\theta \ln \pi_\theta(a_t|s_t) \underbrace{\sum_{s_{t+1},r_{t+1}} p(s_{t+1}, r_{t+1}|s_t, a_t)}_{=1} \underbrace{\sum_{\tau_{>t+1}} p_\theta(\tau_{>t+1})}_{=1}\Big] \\
&\quad= \mathbb{E}_{\tau_{\le t}}\Big[b(s_t)\,\nabla_\theta \underbrace{\sum_{a_t} \pi_\theta(a_t|s_t)}_{=1}\Big] = 0,
\end{aligned} \tag{10.13}$$

where $\tau_{>t+1}$ is the part of the trajectory after s_{t+1} and the second line uses the log-derivative identity. An action-independent baseline therefore leaves the expected gradient unchanged and acts only on its

variance. A baseline near the typical return centres the weights $G_t - b$, so that better-than-usual actions are reinforced and worse ones suppressed (Figure 27). The gradient also multiplies these weights by a score, so the variance can increase or decrease. Appendix F derives the variance-minimising baseline and gives an example in which the exact value baseline increases the variance. With $b(s_t) = V_\pi(s_t)$ the weight is on average the advantage, $\mathbb{E}[G_t|s_t, a_t] - V_\pi(s_t) = A_\pi(s_t, a_t)$. In practice the baseline can be a learned value function or an average of the returns of other, independently sampled episodes at the same step, a leave-one-out average, whose finite-batch properties Appendix F derives. With a constant baseline in a stateless problem with one action per episode the gradient takes the form of a covariance, $\nabla_\theta \mathcal{J} = \mathbb{E}[(G - \mathbb{E}G)\nabla_\theta \ln \pi_\theta] = \mathrm{Cov}(G, \nabla_\theta \ln \pi_\theta)$, the analogue of $\partial_\beta \langle E \rangle = -\mathrm{Cov}(E, E)$ in the fluctuation identity.

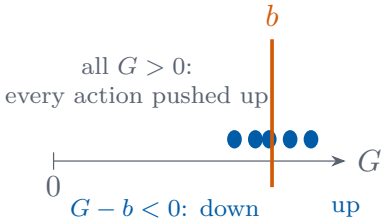


Figure 27: Returns G of five episodes (dots), all positive. Without a baseline each sampled score has a positive return weight. With the baseline b near the mean return (orange line), the weights $G - b$ have both signs. The combined probability change also depends on the policy scores and their average over episodes.

The box collects REINFORCE with both refinements. The factor γ^t in the update comes from the derivation. Many implementations leave it out and so weight late steps more strongly, with an update that is no longer the gradient of the discounted return. For $\gamma = 1$ the two coincide.

REINFORCE with reward-to-go and baseline

1. Initialise the policy $\pi_\theta(a|s)$, for example a network with a softmax or Gaussian output.
2. Sample a batch of B episodes with the current policy and record the states, actions and rewards.
3. For every episode i and step t compute the reward-to-go $G_t^{(i)} = \sum_{t' \geq t} \gamma^{t'-t} r_{t'+1}^{(i)}$ and the baseline b_t. The baseline can be a leave-one-out average of G_t over the other episodes of the batch that are still running at step t.
4. Take one step of gradient ascent, $\theta \leftarrow \theta + \alpha \frac{1}{B} \sum_i \sum_t \gamma^t (G_t^{(i)} - b_t) \nabla_\theta \ln \pi_\theta(a_t^{(i)}|s_t^{(i)})$, and return to step 2.

10.6 The gradient in the running decision process

The running decision process of Figure 6 has one choice, in s_A. Let the policy take a_1 there with the sigmoid probability $\pi_\theta(a_1|s_\mathrm{A}) = 1/(1 + e^{-\theta})$, and let $\mathcal{J}(\theta) = V_{\pi_\theta}(s_\mathrm{A})$ with $\gamma = 0.9$. Solving the two Bellman equations (6.5) for every θ gives $\mathcal{J}(0) = 3.12$, the value of the random policy, and $\mathcal{J} \to 8.58$ for $\theta \to \infty$, the value of the policy that always takes a_1. The derivative at the random policy, $\partial_\theta \mathcal{J}(0) = 2.35$, is positive, and making a_1 more probable raises the expected return. Equation (10.5) gives the same number as an average over sampled episodes, each weighted with its return and the sum of its scores $\partial_\theta \ln \pi_\theta(a_1|s_\mathrm{A}) = 1 - \pi_\theta(a_1|s_\mathrm{A})$ and $\partial_\theta \ln \pi_\theta(a_2|s_\mathrm{A}) = -\pi_\theta(a_1|s_\mathrm{A})$, without solving any equation. The episodes that take a_2 end with -1 and receive negative scores, and episodes that take a_1 and reach the reward $+10$ receive positive ones.

10.7 A walker with two actions

A stateless example can be solved on paper [1,2]. A walker on a line moves up or down at each of T steps and receives $r = +1$ for a step up and $r = -1$ for a step down. With $\gamma = 1$ the return is the final height, $G = n_{\mathrm{up}} - n_{\mathrm{down}} = 2n_{\mathrm{up}} - T$, where n_{up} is the number of steps up, and the optimal policy always moves up. The walker sees no state, and its policy is the sigmoid of one parameter θ,

$$\pi_\theta(\mathrm{up}) = \frac{1}{1+e^{-\theta}}, \qquad \pi_\theta(\mathrm{down}) = \frac{1}{1+e^{\theta}}, \tag{10.14}$$

with the scores $\partial_\theta \ln \pi_\theta(\mathrm{up}) = \pi_\theta(\mathrm{down})$ and $\partial_\theta \ln \pi_\theta(\mathrm{down}) = -\pi_\theta(\mathrm{up})$. Write π_{up} for $\pi_\theta(\mathrm{up})$. The sum of the scores of an episode is $n_{\mathrm{up}}(1-\pi_{\mathrm{up}}) - n_{\mathrm{down}}\pi_{\mathrm{up}} = n_{\mathrm{up}} - T\pi_{\mathrm{up}} = n_{\mathrm{up}} - \mathbb{E}n_{\mathrm{up}}$, since n_{up} is binomial with mean $T\pi_{\mathrm{up}}$. The policy gradient (10.5) is therefore

$$\partial_\theta \mathcal{J} = \mathbb{E}\big[(2n_{\mathrm{up}} - T)(n_{\mathrm{up}} - \mathbb{E}n_{\mathrm{up}})\big] = 2\,\mathrm{Var}\, n_{\mathrm{up}} = 2T\pi_{\mathrm{up}}(1-\pi_{\mathrm{up}}), \tag{10.15}$$

where the constant $-T$ drops out because $n_{\mathrm{up}} - \mathbb{E}n_{\mathrm{up}}$ has mean zero, the baseline argument in its simplest form. The result agrees with the direct derivative of $\mathcal{J} = \mathbb{E}G = T(2\pi_{\mathrm{up}} - 1)$. The gradient is largest at $\pi_{\mathrm{up}} = \frac{1}{2}$ and small at both ends. A walker that already goes up has little left to learn, and one that almost never goes up rarely sees what going up earns. The initial policy therefore matters, and a policy that starts close to deterministic in the wrong direction learns slowly.

10.8 Gaussian policies for continuous actions

Many controls are continuous, a rotation angle or a drive amplitude. Discretising them costs precision and multiplies the number of actions, and the maximum over actions that a value-based method needs in every update becomes an optimisation over a continuum. A policy-gradient agent can instead sample the action from a Gaussian with the mean $\mu_\theta(s)$ and the standard deviation $\sigma_\theta(s)$ [1,2], $a = \mu + \sigma\xi$ with a standard normal ξ,

$$\pi_\theta(a \mid s) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big[-\frac{(a-\mu)^2}{2\sigma^2}\Big], \tag{10.16}$$

where the mean μ is computed by a network from the state and the width $\sigma = e^{\nu}$ is either another output or a free parameter. The log-width ν is trained in place of σ so that σ stays positive. The scores follow from $\ln \pi = -(a-\mu)^2/2\sigma^2 - \nu - \frac{1}{2}\ln 2\pi$,

$$\frac{\partial \ln \pi}{\partial \mu} = \frac{a-\mu}{\sigma^2}, \qquad \frac{\partial \ln \pi}{\partial \nu} = \frac{(a-\mu)^2}{\sigma^2} - 1. \tag{10.17}$$

The mean moves towards actions that gave a return above the baseline (Figure 28 shows the policy and the actions it samples). The width shrinks when the better actions lie within one standard deviation of the mean and grows when they lie farther out. When the reward has a single narrow maximum, the policy can therefore become nearly deterministic as learning proceeds.

A quadratic reward shows both effects without approximation. Let a single action be rewarded with $G = -(a-a_0)^2$, largest at the unknown point a_0, and write $c = \mu - a_0$. With $a = \mu + \sigma\xi$, $\mathcal{J} = -\mathbb{E}(c+\sigma\xi)^2 = -c^2 - \sigma^2$, so $\partial_\mu \mathcal{J} = -2c$ and $\partial_\nu \mathcal{J} = \sigma\,\partial_\sigma \mathcal{J} = -2\sigma^2$. The score estimates give the same numbers from the rewards alone. With $\mathbb{E}\xi = \mathbb{E}\xi^3 = 0$, $\mathbb{E}\xi^2 = 1$ and $\mathbb{E}\xi^4 = 3$,

$$\mathbb{E}\Big[G\,\frac{a-\mu}{\sigma^2}\Big] = -\frac{1}{\sigma}\mathbb{E}\big[(c+\sigma\xi)^2\xi\big] = -2c, \qquad \mathbb{E}\Big[G\Big(\frac{(a-\mu)^2}{\sigma^2} - 1\Big)\Big] = -\mathbb{E}\big[(c+\sigma\xi)^2(\xi^2-1)\big] = -2\sigma^2. \tag{10.18}$$

The mean moves towards a_0, and the width keeps shrinking, so the best policy is the deterministic choice $a = a_0$.

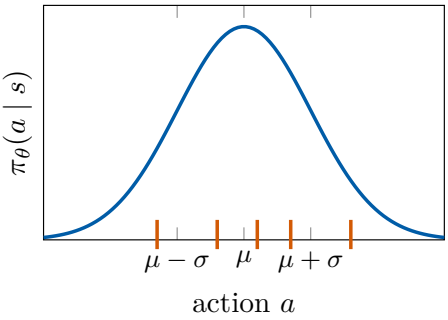


Figure 28: A Gaussian policy $\pi_\theta(a|s)$ over a continuous action a with the mean μ and the width σ. The orange ticks are sampled actions, which explore around the mean.

10.9 Policy networks

The policy gradient needs no change when the table of preferences is replaced by a network. A policy network takes the state as input. For discrete actions it has one output per action, the preferences $z_\theta(s, a)$ of the softmax (10.6) (Figure 29). For continuous actions it has two outputs per action, the mean μ and the log-width ν of the Gaussian of Section 10.8. Training minimises the weighted cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i - 1} \gamma^t (G_t^{(i)} - b_t) \ln \pi_\theta(a_t^{(i)} \mid s_t^{(i)}), \tag{10.19}$$

whose negative gradient is the estimate of the REINFORCE box [1,2]. The episodes must be sampled with the current policy, since equation (10.12) is an average over the trajectories of π_θ itself, so REINFORCE is on-policy and uses no replay memory. The weights $G_t - b_t$ are often divided by their standard deviation in the batch. The division keeps the size of the step independent of the scale of the returns and, unlike a baseline, introduces a small bias [1,2]. The norm of the gradient is also often clipped to a maximum, which protects the network from a single batch with unusually large weights.

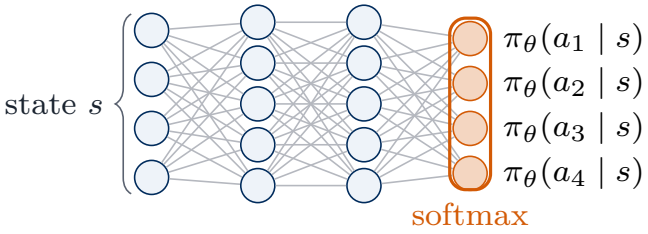


Figure 29: A policy network for discrete actions. The state s enters on the left, and the outputs, the preferences $z_\theta(s, a_k)$, pass through a softmax (orange frame), which turns them into the probabilities $\pi_\theta(a_k|s)$ of equation (10.6). The action is sampled from these probabilities.

10.10 The actor and the critic

The actor chooses actions with $\pi_\theta(a \mid s)$, and the critic $V_\phi(s)$ estimates the return of continuing with that policy. For an exact critic, the conditional expectation of its temporal-difference error is

$$\mathbb{E}[\delta_t \mid s_t = s, a_t = a] = \mathbb{E}[r_{t+1} + \gamma V_\pi(s_{t+1}) \mid s, a] - V_\pi(s) \tag{10.20}$$

$$= Q_\pi(s, a) - V_\pi(s) = A_\pi(s, a). \tag{10.21}$$

If the critic has error $e_\phi(s) = V_\phi(s) - V_\pi(s)$, the same calculation gives

$$\mathbb{E}[\delta_t \mid s, a] = A_\pi(s, a) + \gamma \sum_{s'} p(s' \mid s, a) e_\phi(s') - e_\phi(s). \tag{10.22}$$

The actor thus obtains an estimate of the advantage after every transition, at the cost of depending on the critic's accuracy. The critic moves towards the observed reward plus the estimated continuation,

while the actor moves in the direction $\gamma^t \delta_t \nabla_\theta \ln \pi_\theta(a_t \mid s_t)$ for the discounted episodic objective derived above. Terminal continuations are zero, and the actor treats the advantage estimate as fixed during its update.

An actor–critic update from one transition

1. Observe s_t and sample $a_t \sim \pi_\theta(\cdot \mid s_t)$ with the actor.
2. Apply the action and observe r_{t+1}, s_{t+1} and the terminal flag d_{t+1}.
3. Before changing either network, compute $y_t = r_{t+1} + \gamma(1 - d_{t+1}) V_\phi(s_{t+1})$ and $\delta_t = y_t - V_\phi(s_t)$.
4. Update the critic by descending $\frac{1}{2}[y_t - V_\phi(s_t)]^2$, holding y_t fixed.
5. Update the actor by descending $-\gamma^t \delta_t \ln \pi_\theta(a_t \mid s_t)$, holding the sampled transition and δ_t fixed.
6. Continue from the next state, or reset after termination.

The actor's loss has a minus sign because maximising the return uses gradient ascent [1,2]. The critic descends its squared-error loss with the target held fixed, even though the next-state value was evaluated with the same critic.

The quantum-preparation experiments use DQN, Double DQN and REINFORCE. Proximal policy optimisation (PPO) is an actor–critic method that limits each policy update with a clipped objective.

Part II
Classical control

11 Balancing a pole on a moving cart

A cart moves along a horizontal track with a pole hinged to it. Gravity makes the upright pole unstable. In a simplified model with the pole mass at distance ℓ from the hinge, the tilt ϑ from the vertical obeys

$$\ell\,\ddot{\vartheta} = g \sin\vartheta - \ddot{x}\cos\vartheta, \tag{11.1}$$

where g is the gravitational acceleration and $\ddot{x}$ the acceleration of the cart. Without a push a small tilt grows as $e^{t\sqrt{g/\ell}}$, and accelerating the cart towards the side of the tilt, $\ddot{x}$ with the sign of ϑ, reduces the angular acceleration. We can push the cart left or right, moving the support beneath the pole to keep it balanced. The controller must learn a sequence of pushes that keeps both the cart and the pole inside their permitted ranges. Figure 30 shows the environment used in the simulation.

11.1 Observations and pushes

The observation is $s = (x, v, \vartheta, \omega)$. Here x is the cart position, v its velocity, ϑ the pole angle measured from the upright position, and ω its angular velocity. These numbers distinguish configurations that look similar in a still image. A pole tilted slightly to the right can be falling further right or already moving back towards upright, and its angular velocity changes the appropriate response.

The environment integrates the mechanical response after each force choice. The actor learns neither a label for the correct push nor the equations of motion. During evaluation we freeze the actor and measure how long the resulting feedback rule maintains balance.

The physical dynamics are Markov in the four mechanical variables. Survival for 500 steps means reaching the evaluation limit and establishes no claim of indefinite balancing.

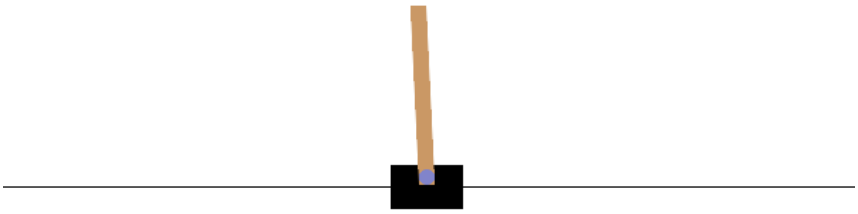


Figure 30: A rendered CartPole-v1 environment at reset. The image illustrates the apparatus. The policy receives four numerical observations of the cart and the pole.

quantity	CartPole-v1 definition
observation	cart position and velocity, pole angle and angular velocity
action	$a = 0$ applies a force of 10 N to the left, $a = 1$ the same force to the right
decision interval	0.02 seconds
reward	$+1$ for each step, including the failure transition
physical termination	cart outside $\lvert x \rvert \leq 2.4$ or pole tilt outside $\lvert \vartheta \rvert \leq 12^\circ$
evaluation cutoff	wrapper limit of 500 steps

Table 2: The default CartPole-v1 task used here [52]. The allowed observation-space bounds extend beyond the physical termination thresholds.

11.2 Survival reward and return

Under the default convention every reward equals one, so the undiscounted return of an episode of T steps is its length. With $\gamma < 1$,

$$G_0 = 1 + \gamma + \cdots + \gamma^{T-1}, \tag{11.2}$$

$$(1 - \gamma)G_0 = 1 - \gamma^T, \tag{11.3}$$

$$G_0 = \frac{1 - \gamma^T}{1 - \gamma}. \tag{11.4}$$

Longer survival increases both objectives, while discounting reduces the contribution of later survival. The two pushes earn the same immediate reward and still have different values, because they lead to different later states and chances of failure. This separates immediate reward from the expected continuation in the Bellman equation.

11.3 A policy update and a recorded controller

We use a policy network with four inputs, two hidden layers and two output logits. A softmax converts its outputs to probabilities for left and right. We record random-action trajectories and evaluate a previously trained actor, which takes its most probable action, on a new set of reset seeds. Figure 31 pairs individual trajectories with their realised episode lengths.

The new evaluation uses 20 reset seeds. The random controller has mean episode length 21.3 and loses the pole after about twenty steps. The saved actor has mean length 500.0 and reaches the 500-step evaluation limit in 20 of these episodes, balancing the pole for the whole evaluation window from every tested start.

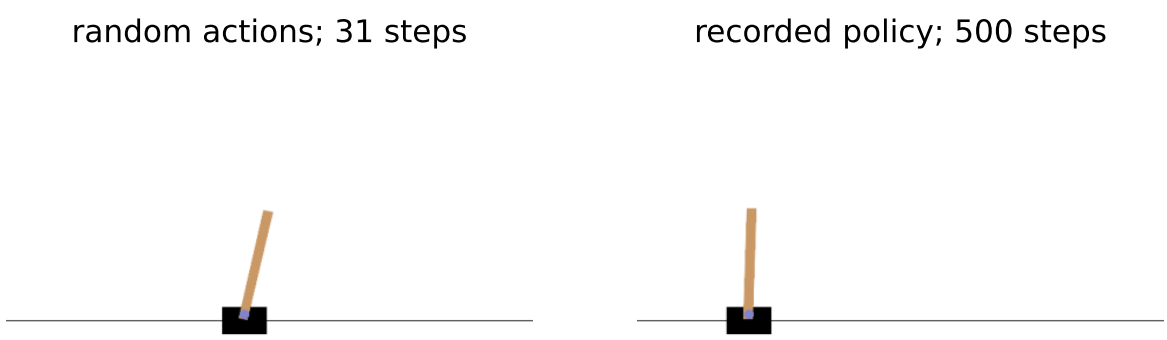


Figure 31: Final rendered states of one random-action trajectory and one trajectory controlled by the recorded CartPole policy. The displayed episode lengths describe these individual runs. Aggregate evaluation uses separate reset seeds.

We also initialise a fresh actor and perform one REINFORCE update from eight sampled episodes. For episode i we compute each discounted reward-to-go $G_t^{(i)}$ and form

$$\widetilde{\nabla_\theta \mathcal{J}} = \frac{1}{B} \sum_{i=1}^{B} \sum_{t} \gamma^t G_t^{(i)} \nabla_\theta \ln \pi_\theta(a_t^{(i)} \mid s_t^{(i)}). \tag{11.5}$$

The sampled states, actions and returns are held fixed while differentiating the log probabilities. The probabilities before and after the update show how the score estimator changes the policy.

12 Landing a vehicle with discrete engine commands

A lander begins above a pad between two flags, with a random initial disturbance. Its controller can fire a main engine or either orientation engine, or leave all engines off. Gravity, momentum and rotation couple the choices over time. The agent must reduce displacement, speed and tilt while managing the cost of firing engines. Figure 32 shows the rendered environment.

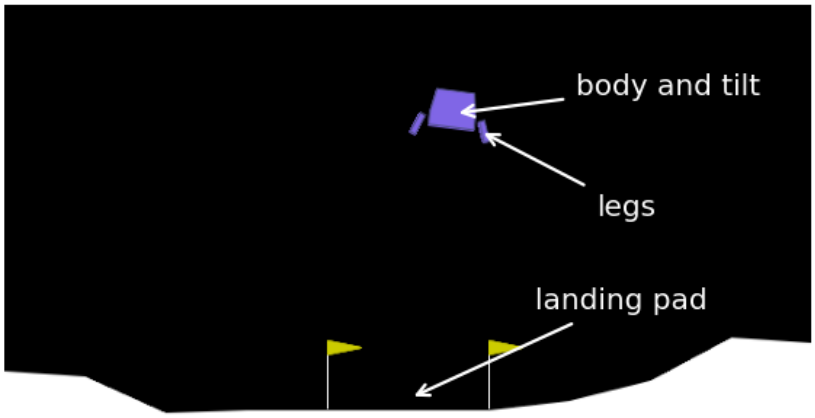


Figure 32: LunarLander-v3 after 60 no-engine steps from reset, with the complete vehicle in view. The flags mark the nominal landing pad. The network receives numerical kinematic observations and leg-contact flags.

12.1 Kinematic observations and engine actions

The observation has eight entries,

$$s = (x, y, v_x, v_y, \vartheta, \omega, c_L, c_R). \tag{12.1}$$

The position is measured relative to the pad in normalised environment coordinates, and the velocity features are scaled observations. The angle ϑ and the angular-velocity feature ω describe orientation and rotation, while the binary variables c_L and c_R indicate contact of the left and right legs with the ground. The policy receives these numbers without a rendered image.

action	engine command	physical interpretation
0	all engines off	free evolution under gravity and contact forces
1	left orientation engine	a side-engine impulse changes rotation and horizontal motion
2	main engine	thrust acts along the lander's current orientation
3	right orientation engine	the opposite side-engine command

Table 3: The four actions of the discrete LunarLander-v3 environment [53]. Engine names identify the fired engine. They specify neither a desired position nor a requested direction of travel.

Momentum makes the current velocity important, since a lander directly above the pad can still move away from it. Thrust also depends on orientation, so firing the main engine while tilted can change both vertical and horizontal motion. The controller must learn those delayed consequences from transitions and rewards.

12.2 Shaping reward and terminal outcomes

The installed environment computes a shaping potential,

$$\Phi(s) = -100\sqrt{x^2 + y^2} - 100\sqrt{v_x^2 + v_y^2} - 100|\vartheta| + 10c_L + 10c_R. \tag{12.2}$$

For an ordinary nonterminal transition, its reward is

$$r_{t+1} = \Phi(s_{t+1}) - \Phi(s_t) - 0.3\,\mathbf{1}\{a_t = 2\} - 0.03\,\mathbf{1}\{a_t \in \{1, 3\}\}. \tag{12.3}$$

Moving closer to the pad, slowing down or reducing tilt can raise the potential. Making leg contact adds a contribution that is removed if contact is lost. Engine charges penalise firing.

At physical termination the implementation replaces that calculation by a terminal reward. A crash or a departure beyond the horizontal bounds gives -100, and a lander whose physics body comes to rest and sleeps gives $+100$. The wrapper imposes a 1000-step cutoff separately. A rest event, a return above a chosen threshold and landing at a particular geometric location are distinct evaluation criteria.

The difference $\Phi(s') - \Phi(s)$ belongs to this environment's reward definition. With discount $\gamma < 1$ this difference is not of the potential-based form $\gamma\Phi(s') - \Phi(s)$, so it can change which policy is optimal.

12.3 Action values and actor–critic control

The recorded DQN maps the eight observations to four action values. Its deterministic evaluation selects

$$a_t = \arg\max_a Q(s_t, a; \theta). \tag{12.4}$$

Each output estimates a discounted return, measured in reward units.

The recorded PPO controller has an actor, which produces four logits and their softmax probabilities, and a critic. The critic estimates $V_\phi(s)$, the return expected under the collecting policy, with

zero continuation at a physical terminal state. With an accurate critic, the conditional expectation of its temporal-difference error gives the advantage, as derived above.

During PPO training, a batch is collected with policy π_{old}. For a sampled state and action, define the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}. \tag{12.5}$$

For a fixed state this ratio reweights the action distribution. The states in the batch still come from the old policy, so the clipped objective is a surrogate for policy improvement,

$$L_{\text{clip}}(\theta) = \mathbb{E}_{\text{batch}}\left[\min\left(r_t(\theta)\tilde{A}_t, \text{clip}(r_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\tilde{A}_t\right)\right]. \tag{12.6}$$

Here $\tilde{A}_t$ is the sampled advantage estimate and $\epsilon_{\text{clip}} > 0$ sets the clipping interval. For positive advantage, the surrogate stops rewarding an increase in the ratio beyond the upper edge. For negative advantage, its corresponding flat region lies below the lower edge. Clipping modifies this incentive and imposes no hard bound on the actual policy change. The critic is fitted to return estimates, and a separate entropy term can encourage exploration [54].

12.4 Evaluation of the recorded agents

Both saved agents are evaluated deterministically on the same 20 reset seeds, with wind disabled and the same wrapper limit. Figure 33 shows individual end states. The new mean returns are -166.2 for random actions, 252.6 for the saved DQN and 200.0 for the saved PPO actor. The standard error is about 16 for each of the two trained agents. Counted by the terminal rest event, the DQN lands in 16 of 20 episodes, crashes in 2 and reaches the time limit in 2. The PPO actor lands in 15, crashes in 1 and reaches the time limit in 4. Every random-action episode ends in a crash.



Figure 33: One random-action trajectory and trajectories controlled by the saved DQN and PPO actors, using the same reset seed. Titles give their realised returns. These screenshots describe individual episodes. The mean returns in the text cover the full set of reset seeds.

The DQN was trained with 200 000 interactions and discount 0.99. The PPO run used 1 998 848 interactions with discount 0.999, together with a different update procedure. These differences prevent an algorithm-wide ranking from this comparison.

12.5 The connection to quantum preparation

CartPole and LunarLander make the control loop visible. The actor receives an observation, chooses an allowed operation and receives a reward from the resulting transition. In quantum preparation the operation becomes a gate or Hamiltonian evolution, and the reward describes fidelity and resource costs, while the value and policy algorithms stay the same.

The classical agents read kinematic features, while the quantum-compilation simulator supplies complex amplitudes that a laboratory controller cannot read directly.

Part III
Quantum applications

13 The state-preparation problem

We are given an initial state $|\psi_0\rangle$ and a target $|\psi_{\text{target}}\rangle$. We construct an allowed unitary $\hat{U}$ to maximise

$$F = |\langle\psi_{\text{target}}|\hat{U}|\psi_0\rangle|^2, \qquad \hat{U} = \hat{U}_{a_{m-1}} \cdots \hat{U}_{a_0}.$$

Each action chooses a gate from the available set or an evolution $\hat{U}_a = e^{-i\hat{\mathcal{H}}_a \Delta t_a}$ under an allowed Hamiltonian generator for an allowed duration, in units with $\hbar = 1$. A gate or time budget makes the length and cost of the sequence part of the task. The current prepared state is the observation in the simulated compilation problem, and a policy chooses its next operation. We apply the values and learning algorithms developed above to this problem and compare learned circuits with reference constructions and small-system searches.

13.1 Primitive gates and their action

A gate acts on the complex amplitudes of the state. For one qubit, $|0\rangle = (1,0)^\top$, $|1\rangle = (0,1)^\top$ and $|\psi\rangle = a|0\rangle + b|1\rangle$, with $|a|^2 + |b|^2 = 1$. The transformed vector is $\hat{U}|\psi\rangle$, where $\hat{U}^\dagger\hat{U} = \hat{I}$. The Pauli operators are

$$\hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \hat{Y} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \hat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The Hadamard and phase gates are

$$\hat{H} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \hat{S} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad \hat{T} = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}. \tag{13.1}$$

Hadamard maps $|0\rangle$ to $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ to $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. The phase gates preserve the two computational-basis probabilities and change the phase between the amplitudes. For example, $\hat{T}|+\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$. A relative phase can affect a later measurement after another gate causes interference.

With $c = \cos(\theta/2)$ and $s = \sin(\theta/2)$, rotations have the matrices

$$\hat{R}_x(\theta) = \begin{pmatrix} c & -is \\ -is & c \end{pmatrix}, \quad \hat{R}_y(\theta) = \begin{pmatrix} c & -s \\ s & c \end{pmatrix}, \quad \hat{R}_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

They obey $\hat{R}_\alpha(\theta) = \exp[-i\theta\hat{\sigma}_\alpha/2]$. A Hamiltonian $\hat{\mathcal{H}} = (\Omega/2)\hat{\sigma}_\alpha$ applied for a duration Δt implements the angle $\theta = \Omega\Delta t$. The factor $1/2$ in the exponent makes θ the physical rotation angle. Both $\hat{R}_x(\theta)|0\rangle = c|0\rangle - is|1\rangle$ and $\hat{R}_y(\theta)|0\rangle = c|0\rangle + s|1\rangle$ give probability $\sin^2(\theta/2)$ of finding one, with different phases. A z rotation preserves the populations and changes their relative phase by θ. The phase gate satisfies $\hat{T} = e^{i\pi/8}\hat{R}_z(\pi/4)$.

The controlled-NOT (CNOT) gate has a control and a target. In the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, with control first,

$$\widehat{\text{CNOT}}_{0\to1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

It preserves $|00\rangle$ and $|01\rangle$ and exchanges $|10\rangle$ with $|11\rangle$. By linearity this action applies to each component of a superposition. After $\hat{H}$ on the first qubit and CNOT on $|00\rangle$ the result is the entangled state $(|00\rangle + |11\rangle)/\sqrt{2}$. It cannot be expressed as the product of a state of the first qubit and a state of the second.

13.2 Gates as Hamiltonian evolutions

The Hadamard gate satisfies $\hat{H} = (\hat{X} + \hat{Z})/\sqrt{2}$ and $\hat{H}^2 = \hat{I}$. Its exact exponential forms are

$$\hat{H} = i \exp\left[-\frac{i\pi}{2\sqrt{2}}(\hat{X} + \hat{Z})\right] = \exp\left[-\frac{i\pi}{2}\left(\hat{I} - \frac{\hat{X} + \hat{Z}}{\sqrt{2}}\right)\right].$$

The first exponential is a rotation through π about the axis halfway between x and z, and the prefactor i is its global phase. The identity term in the second form moves that phase into the exponential. A generator $\hat{\mathcal{H}}_H = \pi(\hat{I} - \hat{H})/(2\Delta t)$ implements the exact matrix in duration Δt.

The control projectors $\hat{P}_0 = (\hat{I} + \hat{Z})/2$ and $\hat{P}_1 = (\hat{I} - \hat{Z})/2$ give $\widehat{\text{CNOT}} = \hat{P}_0 \otimes \hat{I} + \hat{P}_1 \otimes \hat{X}$. An exact exponential representation is

$$\widehat{\text{CNOT}} = \exp\left[-\frac{i\pi}{4}(\hat{I} - \hat{Z}) \otimes (\hat{I} - \hat{X})\right] = \exp\left[-\frac{i\pi}{4}(\hat{I}_4 - \hat{Z} \otimes \hat{I} - \hat{I} \otimes \hat{X} + \hat{Z} \otimes \hat{X})\right].$$

The generator vanishes for control zero and acts on the target as $\pi(\hat{I} - \hat{X})/2$ for control one, whose exponential is $\hat{X}$. The four terms of the expanded generator commute, and the expression gives the full CNOT matrix with all its phases.

13.3 Two-qubit rotations

For $\alpha = x, y, z$, define $\hat{R}_{\alpha\alpha}(\theta) = \exp[-i\theta\hat{\sigma}_\alpha \otimes \hat{\sigma}_\alpha/2]$. Since the Pauli product squares to $\hat{I}_4$, it equals $c\hat{I}_4 - is\hat{\sigma}_\alpha \otimes \hat{\sigma}_\alpha$. In the same ordered two-qubit basis,

$$\hat{R}_{xx}(\theta) = \begin{pmatrix} c & 0 & 0 & -is \\ 0 & c & -is & 0 \\ 0 & -is & c & 0 \\ -is & 0 & 0 & c \end{pmatrix}, \quad \hat{R}_{yy}(\theta) = \begin{pmatrix} c & 0 & 0 & is \\ 0 & c & -is & 0 \\ 0 & -is & c & 0 \\ is & 0 & 0 & c \end{pmatrix},$$

$$\hat{R}_{zz}(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 & 0 & 0 \\ 0 & e^{i\theta/2} & 0 & 0 \\ 0 & 0 & e^{i\theta/2} & 0 \\ 0 & 0 & 0 & e^{-i\theta/2} \end{pmatrix}.$$

The XX and YY rotations mix $|00\rangle$ with $|11\rangle$ and $|01\rangle$ with $|10\rangle$. For example, $\hat{R}_{xx}(\theta)|00\rangle = c|00\rangle - is|11\rangle$, while YY gives the opposite sign of the imaginary amplitude. At $\theta = \pi/2$ these states are maximally entangled. The ZZ rotation preserves computational-basis probabilities and assigns phase $e^{-i\theta/2}$ to even parity and $e^{i\theta/2}$ to odd parity. Acting on $|++\rangle$ it can create entanglement through the relative phase. For each of these interactions, a Hamiltonian $(\Omega/2)\hat{\sigma}_\alpha \otimes \hat{\sigma}_\alpha$ implements $\theta = \Omega\Delta t$.

Combining XX and YY rotations gives an excitation-transfer gate. With the phase gate $\hat{S}$ of equation (13.1) on the second qubit,

$$\hat{G}(\theta) = (\hat{I} \otimes \hat{S})\, \hat{R}_{xx}(\theta)\hat{R}_{yy}(\theta)\, (\hat{I} \otimes \hat{S})^\dagger = \exp\Big[-\frac{i\theta}{2}(\hat{X} \otimes \hat{Y} - \hat{Y} \otimes \hat{X})\Big], \tag{13.2}$$

which leaves $|00\rangle$ and $|11\rangle$ unchanged and acts as

$$\hat{G}(\theta)|10\rangle = \cos\theta|10\rangle + \sin\theta|01\rangle, \qquad \hat{G}(\theta)|01\rangle = -\sin\theta|10\rangle + \cos\theta|01\rangle.$$

It moves an excitation between the two qubits without changing the number of excitations.

13.4 GHZ, W and Dicke reference states

We consider three families of target states of several qubits and describe their quantum properties before their preparation,

$$|\mathrm{GHZ}_N\rangle = \frac{|0\ldots0\rangle + |1\ldots1\rangle}{\sqrt{2}}, \qquad |\mathrm{W}_N\rangle = \frac{1}{\sqrt{N}}\sum_j |0\ldots1_j\ldots0\rangle, \qquad |D_N^{(k)}\rangle = \frac{1}{\sqrt{\binom{N}{k}}}\sum_{|x|=k}|x\rangle.$$

The GHZ state combines two aligned configurations with a definite relative phase. The W state shares one excitation coherently, and a Dicke state shares a fixed number k of excitations. All are permutation-symmetric. For four qubits and two excitations all components have the same amplitude,

$$|D_4^{(2)}\rangle = \frac{|1100\rangle + |1010\rangle + |1001\rangle + |0110\rangle + |0101\rangle + |0011\rangle}{\sqrt{6}}.$$

Every component contains two excitations, and each placement has probability 1/6 when all four bits are measured.

The six-qubit three-excitation Dicke state has twenty equal positive amplitudes. The coherent amplitudes distinguish these pure states from classical mixtures of the same bit strings.

GHZ and W entanglement differ in what remains of each target when part of the register is ignored. The Dicke reference has no trained-agent result in these studies.

13.5 A bipartition and its reduced state

Divide the qubits into two parts A and B. Every state can be written as a Schmidt decomposition

$$|\psi\rangle = \sum_k \mu_k\,|u_k\rangle_A|v_k\rangle_B, \qquad \mu_k > 0, \quad \sum_k \mu_k^2 = 1, \tag{13.3}$$

with orthonormal states $|u_k\rangle$ of A and $|v_k\rangle$ of B and the Schmidt coefficients μ_k. They are the singular values of the amplitudes arranged as a matrix whose rows are labelled by the configurations of A and whose columns by those of B. The half-chain entanglement entropy is

$$S_{N/2} \coloneqq -\sum_k \mu_k^2 \log_2 \mu_k^2. \tag{13.4}$$

A product state has a single term and $S_{N/2} = 0$, and the Bell and GHZ states have two equal terms and $S_{N/2} = 1$ bit.

Discarding part B means retaining predictions for measurements on A while summing over the unobserved basis index of B. Write $|\psi\rangle = \sum_{i,b}\psi_{ib}|i\rangle_A|b\rangle_B$. The reduced density matrix is

$$(\rho_A)_{ij} = \sum_b \psi_{ib}\psi_{jb}^*. \tag{13.5}$$

This operation is the partial trace, and its eigenvalues are the squared Schmidt coefficients. A pure joint state can have a reduced state that needs a density matrix. More than one nonzero eigenvalue means that no single vector accounts for all retained measurement statistics. For a pure joint state the reduced entropy measures entanglement across the partition, whereas for a mixed joint state it can also reflect classical uncertainty.

13.6 Reduced states and entanglement of the targets

Divide the register into nonempty parts A and B, with m and $N-m$ qubits. The GHZ state already has a Schmidt decomposition,

$$|\mathrm{GHZ}_N\rangle = \frac{|0_A\rangle|0_B\rangle + |1_A\rangle|1_B\rangle}{\sqrt{2}}. \tag{13.6}$$

The two states of each part are orthogonal, so the reduced state has eigenvalues $1/2, 1/2$. Tracing out one qubit removes the off-diagonal terms because $\langle 0|1\rangle = 0$, and the remaining mixture of two product states is separable.

The W state divides according to which part contains its single excitation,

$$|\mathrm{W}_N\rangle = \sqrt{\frac{m}{N}}|\mathrm{W}_m\rangle|0_B\rangle + \sqrt{\frac{N-m}{N}}|0_A\rangle|\mathrm{W}_{N-m}\rangle. \tag{13.7}$$

Both coefficients are nonzero. The reduced state has the eigenvalues m/N and $(N-m)/N$, and its entropy is the binary entropy of m/N, equal to one for an equal split. After one qubit is lost,

$$\hat{\rho}_{N-1} = \frac{N-1}{N}|\mathrm{W}_{N-1}\rangle\langle\mathrm{W}_{N-1}| + \frac{1}{N}|0\ldots0\rangle\langle 0\ldots0|. \tag{13.8}$$

Entanglement entropy diagnoses a pure bipartite state, while for this mixed state the reduced-state entropy alone also includes classical uncertainty. An entanglement test for mixed states uses the partial transpose, which transposes the indices of one part only. A separable state always has a partial transpose without negative eigenvalues, so a negative eigenvalue certifies entanglement across the tested cut. For $N = 3$ the two-qubit state left after losing one qubit has the eigenvalue -0.206, so the W state keeps entanglement after the loss.

For a Dicke target with k excitations, group basis strings by the j excitations in A. There are $\binom{m}{j}\binom{N-m}{k-j}$ such strings. Normalising each group gives

$$|D_N^{(k)}\rangle = \sum_j \sqrt{p_j}|D_m^{(j)}\rangle|D_{N-m}^{(k-j)}\rangle, \tag{13.9}$$

$$p_j = \frac{\binom{m}{j}\binom{N-m}{k-j}}{\binom{N}{k}}, \qquad \max(0, k-N+m) \le j \le \min(k, m), \tag{13.10}$$

$$S_A = -\sum_j p_j \log_2 p_j. \tag{13.11}$$

Different j belong to orthogonal excitation sectors, so these are Schmidt probabilities. Their sum is one because grouping counts every allowed string once.

The transfer gate $\hat{G}(\theta)$ of equation (13.2) preserves the number of excitations, and so do the ZZ rotations. Gates of this kind alone keep $|0\ldots0\rangle$ in the zero-excitation sector, so a W or Dicke target also needs an operation that creates excitations, such as $\hat{X}$ or a Hadamard. A W construction from $|0\ldots0\rangle$ with such gates therefore has to start with an excitation-creating gate, for example $\hat{X}$ on qubit 0. A gate set should be checked for such a constraint before an agent's failure is attributed to learning.

14 Compiling states of many qubits

We apply deep Q-learning and policy gradient to the preparation of the entangled targets defined above, on chains of up to six qubits. We record where each method succeeds and where it fails. Since the agents of both families share the observed state, the gate set and the reward, their results can be compared with each other and with exact references.

14.1 Gates, circuits and fidelity

The number of states grows so fast with N that tables of values and preferences have to be replaced by networks.

N qubits have the 2^N basis states $|s_0 s_1 \ldots s_{N-1}\rangle$ with $s_q \in \{0, 1\}$, and a general state has 2^N complex amplitudes. A single-qubit gate $\hat{U}$ on qubit q acts as $\mathbb{1} \otimes \cdots \otimes \hat{U} \otimes \cdots \otimes \mathbb{1}$ and

never creates entanglement. The CNOT of Section 13 on neighbouring qubits $q \to q'$ therefore supplies the entangling operation. The continuous-angle agents also use the controlled-$\hat{Z}$ gate $\widehat{\mathrm{CZ}}$, which multiplies $|11\rangle$ by -1. A circuit is a sequence of gates $\hat{U}_1, \hat{U}_2, \ldots, \hat{U}_n$ and prepares $|\psi_n\rangle = \hat{U}_n \cdots \hat{U}_2 \hat{U}_1 |\psi_0\rangle$, with the first gate rightmost in the product. Figure 34 draws the two-gate circuit $\widehat{\mathrm{CNOT}}_{0\to1}(\hat{H} \otimes \mathbb{1})|00\rangle = |\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ in this notation.

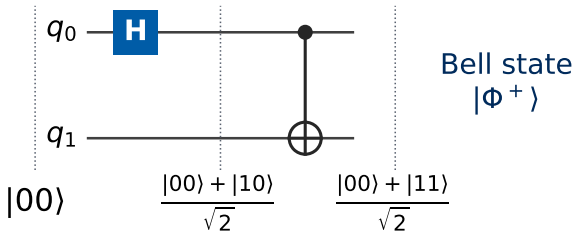


Figure 34: The circuit that prepares the Bell state. Time runs from left to right and each horizontal wire is a qubit. The box is the Hadamard gate, and the dot connected to the circled cross is the CNOT with control q_0 and target q_1. The state of the two qubits is written below each stage.

The fidelity measures the overlap of the prepared state with the target,

$$F = |\langle \psi_{\mathrm{target}} | \psi \rangle|^2 \in [0, 1]. \tag{14.1}$$

It equals one when the states agree up to a global phase. Gates are imperfect, so a circuit should be short. If each gate fails independently with probability p_{g}, a circuit of n gates runs without error with probability $(1 - p_{\mathrm{g}})^n$. For $p_{\mathrm{g}} = 0.01$ the probability equals 0.904 for ten gates and 0.605 for fifty. If a CNOT fails more often than a single-qubit gate, the number of CNOT gates matters in addition to the length. The number of candidate circuits grows as $|\mathcal{A}|^n$ with the size $|\mathcal{A}|$ of the gate set and the length n. An exhaustive search becomes impossible already for a few qubits and moderate lengths.

14.2 Target states

The targets are four families of states on an open chain of N qubits. The first two are the Greenberger–Horne–Zeilinger (GHZ) state [36] and the W state of Section 13, two kinds of multi-qubit entanglement. For three qubits neither can be converted into the other by operations on the single qubits, even with a finite probability of failure [37].

The GHZ state is prepared by $\hat{H}$ on qubit 0 followed by the chain $\widehat{\mathrm{CNOT}}_{0\to1}, \ldots, \widehat{\mathrm{CNOT}}_{N-2\to N-1}$. This circuit of N gates with $N - 1$ CNOT gates extends the Bell circuit. No circuit of our gates is shorter, and none uses fewer CNOT gates. A CNOT maps basis states to basis states and leaves $|0 \ldots 0\rangle$ unchanged, so at least one single-qubit gate is needed. To count the CNOT gates, we draw the qubits as the points of a graph and every CNOT of a circuit as a line between its two qubits. If the graph falls into two groups of qubits with no line between them, no gate ever acts on both groups. The prepared state is then a product of a state of the first group and a state of the second. For every division of the qubits into two groups A and B the GHZ state reads $(|0 \ldots 0\rangle_A |0 \ldots 0\rangle_B + |1 \ldots 1\rangle_A |1 \ldots 1\rangle_B)/\sqrt{2}$. It is entangled in the same way as the Bell state and is no such product. The graph of a GHZ circuit is therefore connected, and a connected graph on N points has at least $N - 1$ lines. The GHZ state needs at least $N - 1$ CNOT gates and at least N gates in total, the numbers of the chain circuit.

For six qubits the reference circuit starts from $|000000\rangle$, applies $\hat{H}$ to qubit 0 and then $\widehat{\mathrm{CNOT}}_{0\to1}, \ldots, \widehat{\mathrm{CNOT}}_{4\to5}$ (Figure 35). The Hadamard creates the branches $|000000\rangle$ and $|100000\rangle$ with equal amplitudes, and the CNOT cascade extends the second branch to $|111111\rangle$. The circuit reaches $F = 1$ with one Hadamard and five CNOT gates, the lower bound just derived.

The W state shares one excitation equally among all qubits. Appendix D shows that the gates $\hat{H}$, $\hat{S}$, $\hat{T}$ and CNOT can prepare it with unit fidelity only when N is a power of two. For $N = 3$ and 6 this gate set reaches it only approximately, and an exact preparation would need rotation angles such as $\arccos(1/\sqrt{3})$ for three qubits.

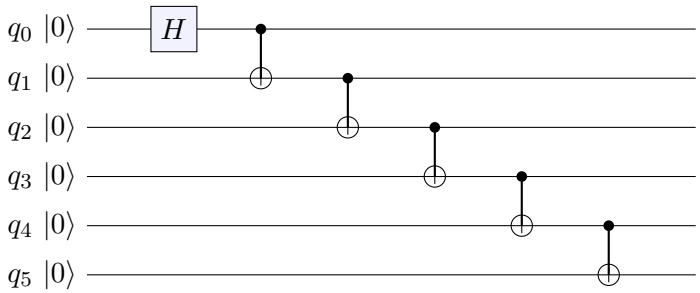


Figure 35: The six-qubit GHZ reference circuit, an analytic construction with one Hadamard and five nearest-neighbour CNOTs from $|000000\rangle$ and ideal fidelity one.

An exact continuous construction of $|\mathrm{W}_6\rangle$ begins with $\hat{X}_0|000000\rangle = |100000\rangle$. On each bond $(j, j+1)$ it applies the transfer gate $\hat{G}(\theta_j)$ of equation (13.2). Choosing $\cos\theta_j = 1/\sqrt{6-j}$ for $j = 0, \ldots, 4$ leaves amplitude $1/\sqrt{6}$ on each successive qubit. The result is the equal positive superposition of the six basis states with one excitation. These continuously chosen gates define a reference construction. The discrete agent with the gates $\hat{H}, \hat{S}, \hat{T}, \widehat{\mathrm{CNOT}}$ approximates the target with this gate set.

The other two families are ground states of the Ising chain

$$\hat{\mathcal{H}} = -J \sum_{i=0}^{N-2} \hat{Z}_i \hat{Z}_{i+1} - h \sum_{i=0}^{N-1} \hat{X}_i - g \sum_{i=0}^{N-1} \hat{Z}_i, \tag{14.2}$$

where $\hat{X}_i$ and $\hat{Z}_i$ are the Pauli operators $\hat{\sigma}_x$ and $\hat{\sigma}_z$ acting on qubit i, J is the coupling, h the transverse field and g the longitudinal field. For $J > 0$ the first term favours aligned neighbours along z. The transverse field favours $|+\rangle$ on every spin, and the longitudinal field favours one direction along z. The ground state is the eigenstate of lowest energy.

For $g = 0$ and $J = 1$ the chain is the transverse-field Ising model (TFIM). It is integrable, which here means that its energies and eigenstates follow from an explicit formula. The Jordan–Wigner transformation rewrites the spins as free fermions, which do not interact with each other, and the many-body problem reduces to independent modes [38]. In the infinite chain the TFIM has a quantum phase transition at $h = 1$ between a ferromagnet, $h < 1$, and a paramagnet, $h > 1$. In a finite chain deep in the ferromagnet the ground state is close to the GHZ state, the symmetric superposition of the two aligned configurations. Deep in the paramagnet it is close to the product state $|+\cdots+\rangle$. The targets are the ground states at $h = 0.5$, 1 and 2.

A longitudinal field $g \neq 0$ destroys the free-fermion solution, and the ground state then has to be computed numerically. The non-integrable target, which we call the longitudinal-field chain, is the ground state at a parameter point far from any integrable chain, taken from Ref. [39]. It has an antiferromagnetic coupling, $J = -1$, and fields of sizes $|h| \approx 0.9045$ and $|g| \approx 0.8090$. Every ground state is computed with the Lanczos method and checked against exact diagonalisation of the full matrix. The Lanczos method is an iterative method that finds the lowest eigenvector using only products of $\hat{\mathcal{H}}$ with a state.

The longitudinal-field target

Reference [39] writes $\sum_i \hat{Z}_i\hat{Z}_{i+1} + c_X \sum_i \hat{X}_i + c_Z \sum_i \hat{Z}_i$ with the transverse coefficient $c_X = (5+\sqrt{5})/8$ and the longitudinal coefficient $c_Z = (1+\sqrt{5})/4$, and reduces the longitudinal field on the two end spins. We keep the field uniform. In the convention (14.2) the chain has $J = -1$, $h = -c_X$, $g = -c_Z$. The signs of h and g do not change the physics. The unitary $\prod_i \hat{X}_i$ turns every $\hat{Z}_i$ into $-\hat{Z}_i$ and leaves $\hat{X}_i$ and $\hat{Z}_i\hat{Z}_{i+1}$ unchanged, so it flips the sign of g, and $\prod_i \hat{Z}_i$ flips the sign of h. The spectrum and the entanglement stay the same, and only the ground state is rotated. The sign of J cannot be removed. Flipping it would require rotating every second spin, which would make the longitudinal field alternate in sign. The sign of the fields does change the target state and with it the fidelity of the target with the initial state $|0\ldots0\rangle$. This section

uses $h = -0.9045$ and $g = -0.8090$. The longitudinal term $-g\sum_i \hat{Z}_i$ then favours $|1\rangle$ on every qubit, and the antiferromagnetic coupling disfavours aligned neighbours. The initial fidelity $F_0 = |\langle 0\ldots0|\psi_{\text{target}}\rangle|^2$ is therefore small, 0.041, 0.005, 0.001 and below 10^{-3} for $N = 2$, 3, 4 and 6. The sign of h leaves F_0 unchanged, since $\prod_i \hat{Z}_i$ leaves $|0\ldots0\rangle$ unchanged. With $g > 0$ the field favours $|0\rangle$ and F_0 is 0.350, 0.122, 0.059 and 0.009, still decreasing with N because of the coupling. We call the two signs the negative-field and the positive-field longitudinal chain.

The comparison uses the half-chain entropy defined in equation (13.4), one property of the target that does not determine compilation difficulty by itself.

14.3 Compilation as a decision process

We formulate compilation as a Markov decision process in which the agent builds the circuit one gate at a time. A separate agent is trained for every target, and the same process serves the value-based and the policy-based agents.

Compilation with discrete gates

State
the current state vector $|\psi_t\rangle$, brought to a standard global phase and written as its 2^N real and 2^N imaginary parts, $s_t = (\operatorname{Re}\psi_t, \operatorname{Im}\psi_t) \in \mathbb{R}^{2^{N+1}}$. This is the full quantum state that a simulation provides.

Action
one gate, $\hat{H}$, $\hat{S}$ or $\hat{T}$ on any qubit or a CNOT between neighbouring qubits in either direction. There are $|\mathcal{A}| = 3N + 2(N-1) = 5N - 2$ actions, from 8 for two qubits to 28 for six. The transition $|\psi_{t+1}\rangle = \hat{U}_{a_t}|\psi_t\rangle$ is deterministic.

Reward
the gain in fidelity minus the cost of the gate, $r_{t+1} = F_{t+1} - F_t - \zeta_{a_t}$, where F_t is the fidelity (14.1) after t gates. A single-qubit gate costs $\zeta_a = \zeta = 0.01$ and a CNOT costs $\zeta_a = \zeta_{\text{CX}}$.

Episode
starts in $|0\ldots0\rangle$ and ends when $F > 0.99$ or after $n_{\max} = 2N + 4$ gates.

Budget
the remaining number of gates is absent from the network input of the recorded DQN studies. Their Bellman targets bootstrap through the gate limit as if the task continued, an approximation to the finite-budget recursion (8.16).

Figure 36 lists the actions for three qubits. Figure 37 follows the state and its fidelity while the circuit of $|\text{GHZ}_3\rangle$ grows gate by gate.

The cost of a CNOT takes one of two values. With the uniform cost, $\zeta_{\text{CX}} = \zeta$, every gate costs the same. With the CNOT cost, $\zeta_{\text{CX}} = 0.03$, a CNOT costs as much as three single-qubit gates. A circuit with fewer CNOT gates can then be the better circuit even when it is longer.

The reward is shaped so that the agent receives a learning signal after every gate. The fidelity gains of an episode of n gates add up, since the intermediate fidelities cancel in pairs,

$$\sum_{t=0}^{n-1} r_{t+1} = \sum_{t=0}^{n-1} (F_{t+1} - F_t) - \sum_{t=0}^{n-1} \zeta_{a_t} = F_n - F_0 - \zeta n_1 - \zeta_{\text{CX}} n_{\text{CX}}, \tag{14.3}$$

where n_1 is the number of single-qubit gates and n_{CX} the number of CNOT gates of the circuit. Without discount the return is the final fidelity minus the constant initial fidelity and minus the cost of the gates. Without gate costs as well, the best circuits of the shaped reward are therefore the best circuits of a reward that pays only the final fidelity. With the discount $\gamma = 0.95$ used in training the

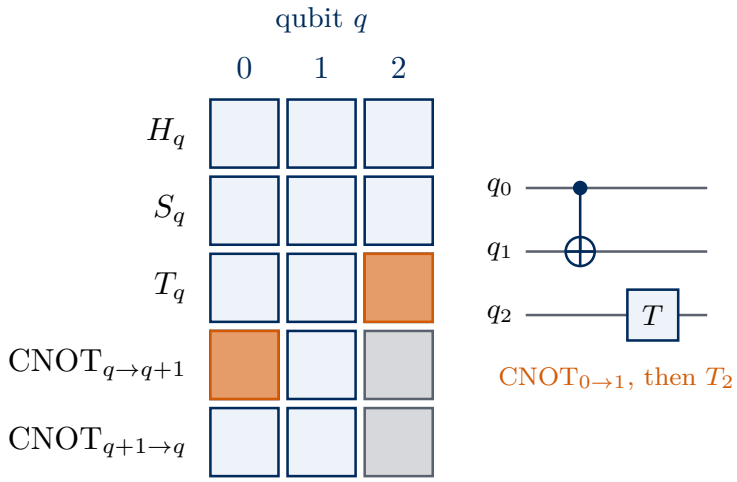


Figure 36: The actions of the compiler for $N = 3$ qubits. Left, the grid of gate types (rows) and qubits q (columns). The gates $\hat{H}_q$, $\hat{S}_q$ and $\hat{T}_q$ act on any qubit, and $\widehat{\mathrm{CNOT}}_{q\to q+1}$ and $\widehat{\mathrm{CNOT}}_{q+1\to q}$ act on neighbouring qubits. The last column of the CNOT rows is therefore empty (grey), which leaves $5N - 2 = 13$ actions. Right, two chosen actions (orange cells) drawn on the wires, $\widehat{\mathrm{CNOT}}_{0\to1}$ followed by $\hat{T}_2$.

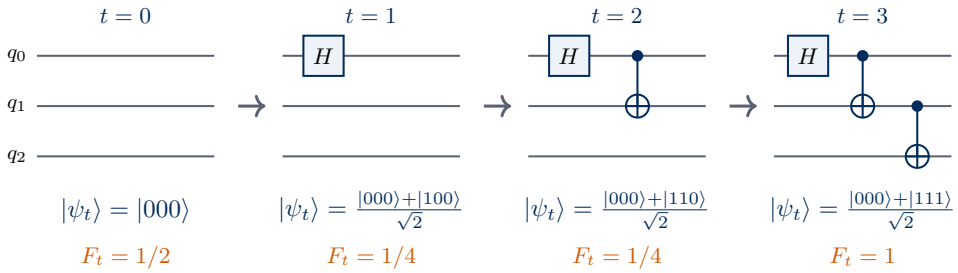


Figure 37: The circuit for $|\mathrm{GHZ}_3\rangle = (|000\rangle + |111\rangle)/\sqrt{2}$ growing gate by gate. Below each stage are the state $|\psi_t\rangle$, which is the state of the decision process, and its fidelity F_t with the target. The fidelity drops from 1/2 to 1/4 after the first gate and reaches 1 only with the last CNOT.

intermediate fidelities also affect the return. Short circuits are favoured because every gate has a cost and the discount makes a fidelity gain worth more when it comes earlier. An episode also ends as soon as $F > 0.99$, so a circuit that reaches the target early stops paying for gates.

For the GHZ state the initial fidelity is already $F_0 = |\langle \mathrm{GHZ}_N|0\dots0\rangle|^2 = 1/2$. After the Hadamard gate on qubit 0 the state is $(|00\dots0\rangle + |10\dots0\rangle)/\sqrt{2}$, whose overlap with the GHZ state is $\frac{1}{2}$, so F drops to 1/4. Each following CNOT of the chain except the last keeps the state of the form $(|0\dots0\rangle + |1\dots10\dots0\rangle)/\sqrt{2}$ and the fidelity at 1/4, and only the last CNOT raises it to 1. For three qubits and the uniform cost the rewards along the optimal circuit are therefore -0.26, -0.01 and $+0.74$, and their sum is $F_n - F_0 - 3\zeta = 0.47$. The optimal circuit begins with a loss and continues through a plateau of zero gains that lasts $N - 2$ CNOT gates. An exploring agent with $5N - 2$ actions has to find this sequence before any positive reward arrives. The phase gates $\hat{S}$ and $\hat{T}$ leave $|0\dots0\rangle$ unchanged. An agent that applies only phase gates therefore keeps $F = F_0$ and loses only the gate costs, less than the loss of the first Hadamard gate.

14.4 Discounted fidelity gains and the task boundary

For a circuit of n gates the discounted fidelity gains satisfy

$$\sum_{t=0}^{n-1} \gamma^t (F_{t+1} - F_t) = \sum_{j=1}^{n} \gamma^{j-1} F_j - \sum_{j=0}^{n-1} \gamma^j F_j \tag{14.4}$$

$$= -F_0 + (1-\gamma) \sum_{j=1}^{n-1} \gamma^{j-1} F_j + \gamma^{n-1} F_n. \tag{14.5}$$

Subtracting $\sum_t \gamma^t \zeta_{a_t}$ gives the complete discounted return. At $\gamma = 1$ the intermediate terms vanish, and the return is the final fidelity minus the initial fidelity and the gate costs. At $\gamma < 1$ the return also rewards intermediate fidelity, so circuits with equal final fidelity and length can have different returns.

Potential-based shaping adds a term to a given base reward,

$$r'_{t+1} = r_{t+1} + \gamma\Phi(s_{t+1}) - \Phi(s_t). \tag{14.6}$$

Summing gives $G'_0 = G_0 - \Phi(s_0) + \gamma^n \Phi(s_n)$. If the potential is zero at all terminal states, the difference is constant for a fixed initial state, and policy rankings are preserved [30]. For bounded potentials in an infinite discounted task the final term vanishes in the limit. A finite-budget task includes its remaining budget in s and imposes the terminal potential there. The plain difference $F_{t+1} - F_t$ is a potential-based increment for the undiscounted case.

The final fidelity, the maximum fidelity over a prefix and the discounted return measure different properties of a circuit. Selecting the best prefix with exact simulator fidelities uses access that an experimental controller lacks, and such a controller needs a stopping rule based on its own observations.

14.5 Exact references

Three references give the exact answer where it is known. For two and three qubits a breadth-first search over distinct states gives the best fidelity $F_*(n)$ reachable with at most n gates. States that differ only by a global phase count as one state. For three qubits and ten gates it visits 68 795 distinct states, while $13^{10} \approx 1.4 \times 10^{11}$ gate sequences exist. Even the best circuit of ten gates reaches only $F_* = 0.829$ for the three-qubit W state. For four qubits the search up to nine gates still fits in memory. It gives $F_*(9) = 0.562$ for the W state, a lower bound on the best fidelity within $n_{\max} = 12$ gates.

The second reference is the largest expected discounted return G_* of any circuit within the gate limit, the finite-horizon objective. Backward induction computes it from the optimal values of the Bellman optimality equation. It runs from the last allowed gate back to the first over all distinct states reachable within $n_{\max}$ gates. With the uniform cost the three-qubit GHZ state has $G_* = 0.398$, the return of the three-gate circuit, which reaches the target and ends the episode long before the gate limit. For this target the finite-horizon optimum and the value targeted by the bootstrapped DQN updates therefore coincide, and G_* can serve as the true value of the initial state. With the CNOT cost the best circuits of the GHZ state stay the chain circuits, which already use the fewest gates and the fewest CNOT gates. Their returns are $G_* = 0.4240$ for two qubits and $G_* = 0.3613$ for three. For the two-qubit W state the CNOT cost changes the best circuit. The shortest circuits have 5 gates with 2 CNOT gates and the discounted return 0.7682. The best circuit has 6 gates with a single CNOT and the return $G_* = 0.7726$, because the saved cost of one CNOT outweighs one additional single-qubit gate. For the three-qubit W state the best circuit is the ten-gate circuit with $F = 0.829$ and three CNOT gates for both costs.

The third reference counts the equivalent shortest circuits. The exhaustive enumeration finds 2, 4 and 8 sequences of N gates that prepare the GHZ state for $N = 2$, 3 and 4, all with $N - 1$ CNOT gates (Figure 38). The graph argument above explains this number, 2^{N-1}. The Hadamard gate may act on any qubit k. The CNOT gates then copy its value outwards along the chain, k of them towards qubit 0 and $N - 1 - k$ towards qubit $N - 1$. Each cascade has a fixed order, and the two cascades may be interleaved in $\binom{N-1}{k}$ ways. The sum over k gives $\sum_k \binom{N-1}{k} = 2^{N-1}$ circuits, and for $N \leq 4$ the enumeration confirms that no other shortest circuit exists. The two-qubit W state has 8 shortest sequences of five gates, and among its six-gate sequences 58 use a single CNOT. A task with many equivalent optima tests whether an agent learns one answer or a distribution over the answers.

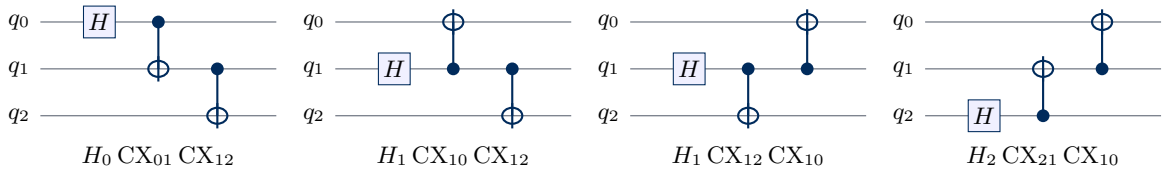


Figure 38: The four shortest circuits of the gates $\hat{H}$, $\hat{S}$, $\hat{T}$ and CNOT that prepare the three-qubit GHZ state from $|000\rangle$. Each horizontal wire is a qubit and time runs from left to right. The box is the Hadamard gate, and a dot joined to a circled cross is a CNOT with its control at the dot. Below each circuit its gates are listed in order, H_k for the Hadamard gate on qubit k and $\mathrm{CX}_{qq'}$ for $\widehat{\mathrm{CNOT}}_{q\to q'}$. The Hadamard gate creates the superposition on one qubit, and the CNOT gates copy its value outwards to the ends of the chain.

14.6 Deep Q-learning on six targets

The action-value function is a network $Q(s, a; \theta)$ with 2^{N+1} inputs and $5N - 2$ outputs (Figure 39). It is trained with deep Q-learning, which fits the network to Bellman targets drawn from a replay memory and computed with a target network. We train it in its plain form (DQN) and with the Double DQN target (DDQN), in which the online network chooses the next gate and the target network evaluates it. DQN and DDQN start from the same network for a given seed, so the only difference between them is the target. The network has two hidden layers of 128 rectified linear units, $\mathrm{ReLU}(x) = \max(0, x)$. It is trained with the discount $\gamma = 0.95$ and an exploration rate ε that falls linearly from 1 to 0.05.

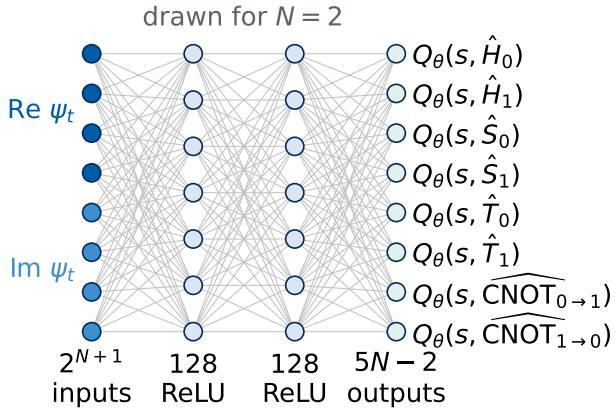


Figure 39: The Q-network of the discrete compilation agents, drawn for $N = 2$ qubits with eight of the 128 units of each hidden layer. The inputs are the real and imaginary parts of the state vector, and each output is the action value $Q(s, a; \theta)$ of one gate.

The first study surveys all six targets with the uniform gate cost and exact rewards. Every greedy circuit, which applies at each step the gate with the largest action value, is cut where its fidelity is largest. A run is a success when this fidelity exceeds 0.99. Each algorithm has 72 runs, six targets on four chain lengths with three seeds, and DDQN reaches the target in more of them than DQN. DDQN ends with $F > 0.99$ in 11 runs and DQN in 4. Of the 42 runs whose exact optimum is known, DDQN reaches it in 18 and DQN in 10. Per target the picture is mixed. With three seeds the mean fidelities of the two algorithms overlap within their spread, and for several targets the mean of DQN is the higher one. Both algorithms learn the targets whose fidelity rises along a short circuit (Figure 40). They struggle where the fidelity offers a long plateau or the gate set reaches the target only with long circuits. All three DQN agents miss the Bell state of two qubits, a two-gate circuit, while all three DDQN agents learn it. The greedy circuits of the DQN agents keep the initial fidelity 1/2 and never take the loss of the first Hadamard gate. For the TFIM ground states the agents often stop at one Hadamard gate per qubit, which prepares the product state $|+\cdots+\rangle$. The fidelities at which DQN ends for these targets are the fidelities of $|+\cdots+\rangle$ with the ground state, for example 0.682 at $h = 1$ and 0.921 at $h = 2$ on six qubits. For $h = 2$ and $N \leq 3$ this circuit is the exact optimum. Some agents reach the GHZ state at every chain length, and it becomes harder as N grows because of its initial

loss and its plateau. The W state and the non-integrable ground state stay well below one already in the exhaustive search for three qubits, with best fidelities 0.829 and 0.769. The agents reach less.

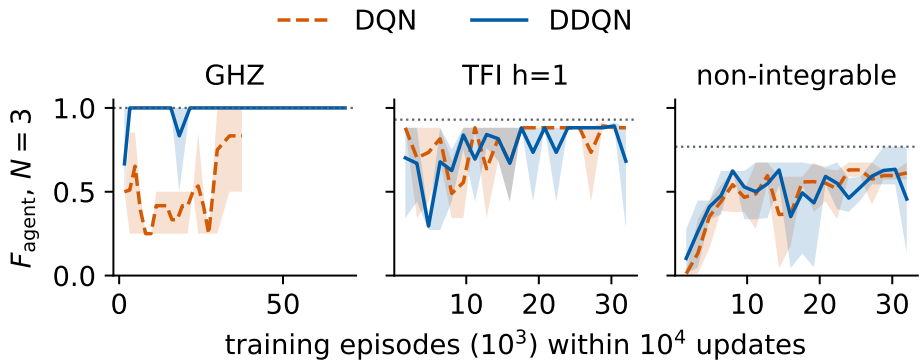


Figure 40: Training of the discrete agents for three targets of three qubits. The fidelity F_{agent} of the greedy circuit against the number of training episodes, the mean over three seeds with the band between the lowest and the highest seed, for DQN and DDQN. The greedy circuit applies $n_{\max}$ gates. The fidelity is known after every gate. The quantity F_{agent} is the largest of the fidelities after the first, second, up to the last gate, the fidelity at which the agent would stop after at least one gate. The fidelity F_0 of the empty circuit is excluded, so an agent can end below it. Training runs for a fixed number of updates, and episodes that reach the target end early, so more episodes fit into the training of the GHZ state. The dotted line is the exact best fidelity $F_*(n_{\max})$ of the exhaustive search.

The learned circuits can be read off Figure 41. For the four-qubit GHZ state a DDQN agent starts with $\hat{H}$ on qubit 3 and runs the CNOT cascade from the other end of the chain. This circuit is one of the eight shortest circuits and differs from the textbook cascade from qubit 0. For the non-integrable ground state of three qubits the agent uses only single-qubit gates, so it prepares a product state below the exact optimum. The entanglement entropy predicts difficulty poorly in these runs, with a correlation coefficient of -0.25 with the reached fidelity over all targets. The GHZ state carries one bit at every N and is reached, and the non-integrable ground state carries less and is reached poorly.

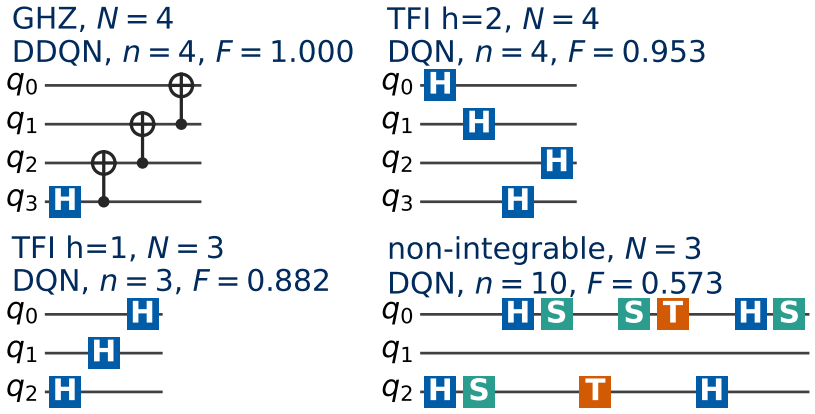


Figure 41: Greedy circuits of the discrete agents, cut at their best fidelity, for four targets, with the algorithm, the number of gates n and the fidelity F.

The GHZ plateau suggests that the form of the reward matters. We test this with DDQN agents trained with three seeds on the three-qubit GHZ state. The four rewards are the fidelity gain or the final fidelity, each with or without the gate cost. With both rewards built on the fidelity gain every agent finds the three-gate circuit, and with the final-fidelity reward no agent reaches the target. That reward arrives only when the episode ends, at $F > 0.99$ or after $n_{\max} = 10$ gates. An exploring agent rarely meets the GHZ state on the way, and the reward carries little information about which early gate led to a good end. The final-fidelity reward at the gate limit also depends on the step counter, which the network does not observe, and the limit transition is bootstrapped. The failure therefore mixes sparse credit assignment with a non-Markov reward.

14.7 Measured rewards and overestimation

The maximisation bias of Section 8.9, the upward bias of a maximum taken over noisy estimates, should matter most when the reward itself is measured with noise. The study covers one target, the three-qubit GHZ state, with five seeds per setting and the uniform gate cost.

On a device the fidelity has to be measured, and we model this with n_s measurement shots per fidelity estimate. One shot applies the inverse of a circuit that prepares the target and measures all qubits, and it returns $0 \ldots 0$ with probability F. Such a circuit is known for the GHZ state, and the procedure serves here as a model of measurement noise with the right binomial statistics. Without such a circuit an experiment would estimate the fidelity in other ways, for example from measurements in several bases, with noise of the same origin. The fraction of outcomes $0 \ldots 0$, $\tilde{F} = k/n_s$ with binomially distributed k, is an unbiased estimate with the standard deviation $\sigma_F = \sqrt{F(1-F)/n_s}$. At $F = 1/2$ and ten shots it is about 0.16. The reward is computed from such estimates, with fresh shots each time, while the end of the episode is still decided by the exact fidelity. The noise has zero mean, so the expected reward and the best circuit are unchanged, and only the learned values become noisy.

The size of the effect follows from the analysis of the maximisation bias in Part I. A reward is the difference of two independent estimates, with the standard deviation $\sqrt{2}\,\sigma_F \approx 0.22$ at ten shots, and the learned values carry noise of this order. By equation (8.25) the maximum of two equal values with noise σ_e exceeds the true value by $0.56\,\sigma_\mathrm{e}$. A simulation gives $1.42\,\sigma_\mathrm{e}$ for eight actions, the gate set of two qubits, while the double estimator stays at zero within the sampling error. The three-qubit agent maximises over 13 gates at every step, and the bias enters every target and is passed back through the Bellman equation.

An overestimation can be measured only against the true value of the initial state, the return $G_* = 0.398$ of the exact references. A predicted value $\max_a Q(s_0, a; \theta)$ above G_* is an overestimation. Table 4 and Figure 42 give the result. With the exact reward the DQN prediction lies above G_* for most of the training (Figure 42, left). It peaks in the middle of the run and comes back close to G_* only near its end. After training both algorithms predict close to G_* and learn the circuit. The overshoot occurs without any measurement noise. The approximation error of the network and the bootstrapped targets make the network's own estimates noisy. The maximum over these estimates is biased upwards in the same way as the maximum over noisy rewards. This overshoot is one candidate cause of the DQN failure on the Bell state in the survey. With 10 shots the reward noise adds to it, and the overshoot persists and grows. The DQN prediction ends at more than twice G_* and no DQN agent learns the circuit. The DDQN prediction stays at G_* and three of five DDQN agents learn it. An agent that ranks gates by inflated values prefers the gates whose values are overestimated most. With 3 shots the noise of one estimate exceeds most fidelity steps, and neither algorithm learns the circuit within the training budget, although DDQN still predicts close to G_*. In this study the double estimator reduced the overestimation to below the resolution of the measurement, and it was the setting in which the circuit was learned from measured rewards.

reward	seeds reaching the target		$\max_a Q(s_0, a; \theta)$, mean	
	DQN	DDQN	DQN	DDQN
exact	4	5	0.38	0.40
10 shots	0	3	0.93	0.40
3 shots	0	0	1.47	0.36

Table 4: Training on the three-qubit GHZ state with exact and measured rewards, 5 seeds per setting. The predicted value of the initial state after training is compared with the exact optimum $G_* = 0.398$.

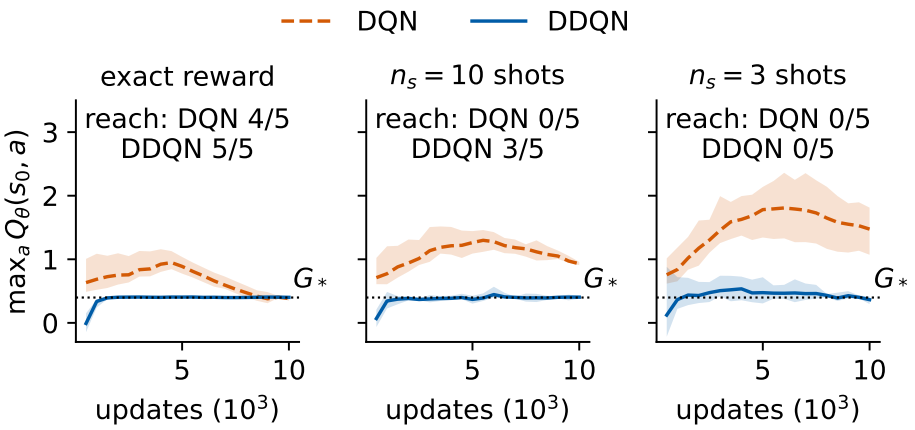


Figure 42: The predicted value $\max_a Q(s_0, a; \theta)$ of the initial state during training on the three-qubit GHZ state, for the exact reward and for rewards estimated from $n_s = 10$ and 3 shots. Lines show the mean over five seeds with the band between the lowest and the highest seed. The dotted line is the exact optimum G_*, and each panel gives the number of seeds whose final greedy circuit reaches $F > 0.99$.

14.8 A policy network for the gates

A policy-gradient agent learns the same task without action values. For the state s a network returns a probability $\pi_\theta(a|s)$ for every gate a, and the agent samples the next gate from these probabilities. Training makes circuits with a large return more probable. The rest of the task is unchanged. The input is the state vector s_t and the actions are the same $5N - 2$ gates. The reward, the discount $\gamma = 0.95$, the success threshold and the time limit $n_{\max}$ are those of the DQN agents. The REINFORCE agents always use the CNOT cost. The policy network has the shape of the Q-network, two hidden layers of 128 ReLU units and one output $z_\theta(s, a)$ per gate. The softmax (10.6) turns the outputs into probabilities, a Boltzmann distribution over the gates with the energies $-z_\theta(s, a)$. The weights of the output layer start at zero. The initial policy therefore chooses every gate with probability $1/(5N - 2)$, as the DQN agent does at the start of its training, when $\varepsilon = 1$.

The agent maximises the expected discounted return from the initial state, $\mathcal{J}(\theta) = \mathbb{E}[G_0 \,|\, \pi_\theta]$. REINFORCE episodes end at the gate limit, so $\mathcal{J}$ is the finite-horizon objective and its best value is the return G_* of the exact references. The agent uses REINFORCE with the reward-to-go and a baseline. It samples a batch of $B = 32$ episodes with the current policy and computes for every episode i and step t the reward-to-go $G_t^{(i)} = \sum_{k \geq 0} \gamma^k r_{t+k+1}^{(i)}$. It then takes one step of the Adam optimiser with the learning rate 10^{-3} along the estimate

$$\tilde{g} = \frac{1}{B} \sum_{i=1}^{B} \sum_{t} \gamma^t (G_t^{(i)} - b_t^{(i)}) \nabla_\theta \ln \pi_\theta(a_t^{(i)} \,|\, s_t^{(i)}), \tag{14.7}$$

where the sum over t runs over the steps of episode i and $b_t^{(i)}$ is the baseline. Adam is a variant of gradient descent that adapts the step size of each parameter to the size of its recent gradients. A gate followed by a return above the baseline becomes more probable and a gate followed by a return below it less probable. The method needs no value network, no replay memory and no target network. Each batch serves one update only, because equation (14.7) is an average over the episodes of the current policy.

14.9 The baseline and the entropy bonus

Every episode starts in $|0 \ldots 0\rangle$ and the gates act deterministically, so the episodes of one batch can be compared at equal step t. The baseline of episode i at step t is the mean reward-to-go of the other episodes of the batch that are still running at that step,

$$b_t^{(i)} = \frac{1}{|\mathcal{R}_t| - 1} \sum_{j \in \mathcal{R}_t,\, j \neq i} G_t^{(j)}, \tag{14.8}$$

where $\mathcal{R}_t$ is the set of episodes still running at step t. Leaving episode i out makes $b_t^{(i)}$ independent of the gates of episode i. The baseline then adds no bias, by the argument that holds for any baseline that does not depend on the action. The weight $G_t - b_t$ is positive for a gate that did better than the other episodes at the same step. A learned value network would adapt the baseline to the state, at the price of a second network with its own training. For a fixed initial state and a batch of 32 episodes the batch mean needs no extra parameters.

The policy explores only by sampling, and its exploration fades as the softmax sharpens. On this task it sharpens too early. Along the GHZ circuit the reward starts with a loss, while the phase gates keep $F = F_0$ and cost only ζ each. A Hadamard gate on $|0\ldots0\rangle$ is followed at once by the loss of a quarter in fidelity, against ζ for a phase gate. Without a further device the weights $G_t - b_t$ of the sampled episodes favour the idle phase gates, and the policy concentrates on them. Once the softmax is sharp, the gate it prefers is sampled almost always and its score $\nabla_\theta z_\theta(s,a) - \sum_{a'} \pi_\theta(a'|s) \nabla_\theta z_\theta(s,a')$ is close to zero. The other gates are rarely sampled, the rare successful episodes stop appearing, and the policy stays in place. We therefore add the entropy of the policy to the objective [33],

$$\mathcal{J}_\beta(\theta) = \mathcal{J}(\theta) + \beta\, \mathbb{E}\big[\mathcal{E}(s_t)\big], \qquad \mathcal{E}(s) \coloneqq -\sum_a \pi_\theta(a \,|\, s) \ln \pi_\theta(a \,|\, s), \tag{14.9}$$

where the average runs over the states visited by the policy and $\beta \geq 0$ is the entropy coefficient. The Gibbs entropy $\mathcal{E}(s)$ of the gate distribution in the state s is largest for the uniform policy. The gradient of the added term pulls the probabilities back towards equal values and keeps alternative gates in the samples. For a single decision with the expected rewards $\bar{r}(a)$ the policy maximises the expected reward plus β times the entropy,

$$\sum_a \pi(a)\, \bar{r}(a) - \beta \sum_a \pi(a) \ln \pi(a), \qquad \sum_a \pi(a) = 1, \tag{14.10}$$

and the stationarity condition with a Lagrange multiplier λ_L for the constraint reads $\bar{r}(a) - \beta(\ln \pi(a) + 1) = \lambda_\mathrm{L}$, so

$$\pi(a) = \frac{e^{\bar{r}(a)/\beta}}{\sum_{a'} e^{\bar{r}(a')/\beta}}, \tag{14.11}$$

a Boltzmann distribution with the energies $-\bar{r}(a)$ at the temperature β. Here β therefore plays the role of a temperature, unlike the inverse temperature β of the fluctuation identity in Section 10. The coefficient falls linearly from $\beta_0 = 1.0$ to zero over the first 60% of the training, the shape of the ε schedule of DQN. The final policy then maximises the return alone. As an analogy, this schedule resembles simulated annealing, which lowers a temperature slowly to find a low-energy state.

The two families use equal numbers of gate applications per agent, about 3.2×10^5. Each records its greedy circuit, in which REINFORCE applies the most probable gate, after every block of about 16 000 gate applications, 20 times per training. The settings of the REINFORCE agents were tuned in a pilot study on the three-qubit GHZ state. DQN and DDQN kept the settings of the survey, so REINFORCE had the larger tuning budget.

We test the baseline and the entropy bonus on the three-qubit GHZ state with 8 seeds and three variants (Figure 43). The first variant has the baseline and the entropy bonus, the second has no baseline, and the third has the baseline without the entropy bonus. During training we record for every update the fraction P_success of the 32 sampled episodes that reach $F > 0.99$. With the entropy bonus the greedy circuit reaches the target in 8 of 8 seeds with the baseline and in 8 of 8 without it. Without the bonus it reaches the target in 0 seeds, and P_success stays at zero, the collapse onto the phase gates described above.

We measure the variance of the gradient estimate at the initial policy θ_0 by drawing 256 independent batches and computing $\tilde{g}$ from each with and without the baseline. We sum the variances of its components, $\mathrm{Var}\, \tilde{g} \coloneqq \sum_k \mathrm{Var}\, \tilde{g}_k$, where k runs over all parameters of the network. The baseline lowers this variance by a factor 2.6, while its effect on the learning curves is small. The agents with the baseline reach $P_\mathrm{success} = 0.5$ after a median of 4 784 training episodes and those without it after 5 168, and the bands of the two variants overlap.

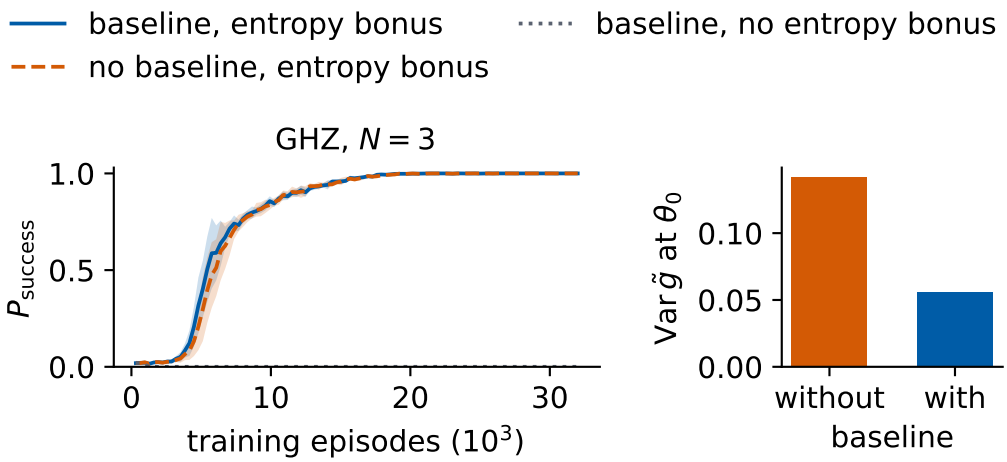


Figure 43: REINFORCE on the three-qubit GHZ state with the CNOT cost. Left, the success fraction P_{success} against the number of training episodes in thousands. It is the fraction of the 32 episodes sampled in one update whose fidelity exceeds 0.99. Lines show averages over ten consecutive updates and over 8 seeds, with a band of one standard deviation between the seeds. The solid line has the baseline (14.8) and the entropy bonus, and the dashed line has no baseline. The dotted line, with the baseline and without the entropy bonus, stays at zero. Right, the total variance $\operatorname{Var} \tilde{g}$ of the gradient estimate (14.7) at the initial policy θ_0, from 256 independent batches, without and with the baseline.

14.10 The two families compared

We train DQN, DDQN and REINFORCE agents on the GHZ and W states for $N = 2$, 3, 4 and 6, with 5 seeds per method. All agents have the CNOT cost, the same gate set and input, the same episodes and the same number of gate applications. A seed fixes the initial network, and the three agents of one seed share the hidden layers of their initial network. For every agent we take the greedy circuit after the last update. We cut it at its first gate with $F > 0.99$, or at its best fidelity if it never gets there. Table 5 and Figure 44 give the result.

target	method	$N = 2$	$N = 3$	$N = 4$	$N = 6$
GHZ_N	DQN	0 (0.500)	1 (0.550)	3 (0.800)	0 (0.500)
	DDQN	5 (1.000)	5 (1.000)	5 (1.000)	0 (0.500)
	REINFORCE	5 (1.000)	5 (1.000)	0 (0.500)	0 (0.500)
	exact	1	1	1	1
W_N	DQN	0 (0.697)	0 (0.514)	0 (0.192)	0 (0.208)
	DDQN	3 (0.871)	0 (0.330)	0 (0.354)	0 (0.237)
	REINFORCE	5 (1.000)	0 (0.390)	0 (0.281)	0 (0.188)
	exact	1	0.829	≥ 0.562	unknown

Table 5: DQN, DDQN and REINFORCE on the GHZ and W states with the CNOT cost. Each entry gives the number of seeds out of 5 whose final greedy circuit reaches $F > 0.99$. In parentheses is the mean over the seeds of the fidelity F_{agent} of that circuit, cut at its first gate with $F > 0.99$ or at its best fidelity. The rows labelled exact give the best fidelity of any circuit within $n_{\max}$ gates. It is known from the GHZ circuit and, for the W state, from the exhaustive search, which for $N = 4$ covers only circuits of up to nine gates and gives a lower bound.

The three methods agree on the shortest GHZ circuits wherever they reach them. Every successful GHZ circuit of every method has N gates and $N-1$ CNOT gates, the optimum of the graph argument. The methods differ in how often they reach it. DDQN learns the GHZ state in all five seeds for $N = 2$, 3 and 4, and plain DQN in none, one and three seeds. REINFORCE learns it in all seeds for $N = 2$ and 3 and in none for $N = 4$. For the two-qubit W state REINFORCE succeeds in all five seeds, DDQN in three and plain DQN in none, and the two successful methods find different circuits. All REINFORCE circuits are shortest circuits, 5 gates with 2 CNOT gates and the discounted return 0.7682. The successful DDQN circuits use a single CNOT. Two of them have 6 gates, the circuit

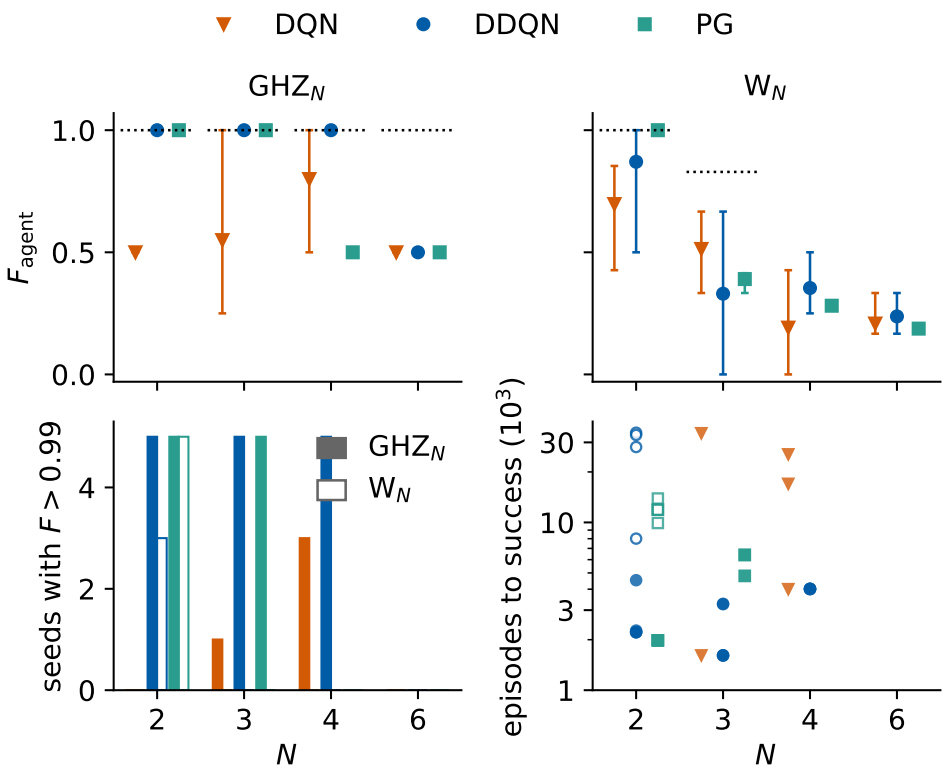


Figure 44: DQN (orange triangles), DDQN (blue circles) and REINFORCE (PG, policy gradient, green squares) on the GHZ and W states with the CNOT cost against the number of qubits N, with 5 seeds per method. Top, the fidelity F_{agent} of the final greedy circuit, cut as in Table 5, for GHZ$_N$ (left) and W$_N$ (right). Markers show the mean over the seeds, and error bars the range between the lowest and the highest seed. The dotted lines mark the exact best fidelity where it is known. Bottom left, the number of seeds whose greedy circuit reaches $F > 0.99$, filled bars for the GHZ state and open bars for the W state. Bottom right, the number of training episodes until the greedy circuit first reaches $F > 0.99$, in thousands and on a logarithmic axis. Each successful seed has one marker, filled for the GHZ state and open for the W state.

with the best return $G_* = 0.7726$, and one has seven gates. On this target DDQN found the circuit preferred by the CNOT cost in two seeds. REINFORCE found a shortest circuit, slightly worse under this cost, in every seed.

A trained DQN agent prepares one circuit, its greedy circuit. A REINFORCE policy assigns a probability to every circuit, and sampling it shows which circuits it has learned. For every successful REINFORCE seed we sample 1000 episodes from the final policy. A single policy puts most of its weight on one shortest circuit and keeps a small weight on other circuits, among them other shortest circuits and a few longer ones. One three-qubit seed splits its weight almost equally between two shortest GHZ circuits. They differ only in the order of the two CNOT gates and are the second and third circuit of Figure 38. Taken over the five seeds, the policies sample all 2 shortest GHZ circuits for $N = 2$, all 4 for $N = 3$ and all 8 shortest circuits of the two-qubit W state. The DQN and DDQN agents settle on one deterministic circuit per seed. Their six successful agents for the three-qubit GHZ state produce three distinct circuits together, and for the two-qubit GHZ state all of them produce the same circuit. The final policies stay slightly random, because the entropy bonus is switched off only after 60% of the training, and the remaining updates sharpen the policy without making it deterministic. Every successful seed nevertheless samples a successful circuit in at least 997 of 1000 episodes, and a shortest one in at least 984.

The speed of the two families can be compared on the targets that both learn, to the resolution of the records, one every 16 000 gate applications. For the two-qubit GHZ state DDQN needs a median of 2 224 training episodes and REINFORCE 1 984. For three qubits DDQN reaches the target at its first record, after 16 000 gate applications or a median of 1 615 episodes. REINFORCE needs a median of 6 400 episodes or about 59 000 gate applications. A DQN agent stores every transition in the replay memory and reuses it in many updates. The Bellman target passes the large reward of the

last CNOT back to the earlier gates. A REINFORCE agent uses every episode once and learns about the Hadamard gate only from the returns of the episodes that sampled it. The random policy of the early phase is the same in both families, the uniform policy of $\varepsilon = 1$, and the methods differ in what they do with these episodes.

14.11 Limits with growing chains

Both families stop as N grows. In this CNOT-cost comparison no agent of any method reaches the six-qubit GHZ state within the training budget. All of them end at $F = 0.5$, the fidelity of $|0\ldots0\rangle$, although the shortest circuit is known and has only six gates. In the uniform-cost survey one of three DQN and one of three DDQN agents did reach it. The six-qubit GHZ state is thus hard for these agents and settings without being out of reach. For the W state no method reaches $F > 0.99$ for $N \geq 3$. For $N = 3$ the exhaustive search shows that no circuit within $n_{\max}$ gates can, and the mean fidelities of all three methods stay well below the exact optimum 0.829. For $N = 4$ they stay below the lower bound 0.562 that a circuit of nine gates already reaches. REINFORCE also misses the four-qubit GHZ state, which DDQN learns in every seed. REINFORCE behaves alike in all five seeds of a target, successful in every seed or in none.

The scaling with N is the hard part of the task, since an agent has to meet the target by chance before the reward can guide it there. Random circuits reach the four-qubit GHZ state far more rarely than the three-qubit one.

These runs set the scope of the comparison, two target families, chain lengths up to six qubits, five seeds per method and about 3.2×10^5 gate applications per agent.

The comparison leaves out one property of the policy gradient, whose estimate needs only the probability of the chosen action and its gradient and never takes a maximum over actions. When the gates are rotations with a continuous angle, a policy-gradient agent samples the angle from a probability density. Deep Q-learning would need the maximum of $Q(s, a; \theta)$ over a continuum in every target. A continuous gate set also contains the angles that the W state needs, which the discrete gate set reaches only approximately.

14.12 Compilation with continuous angles

Most gates of a real device have a continuous parameter, the rotation angle. A deep Q-network has one output per discrete action and a target with a maximum over actions. With an action $\mathbf{a} \in \mathbb{R}^N$ both become optimisation problems over a continuum at every step. Discretising each angle into K values turns one step into K^N actions, and eight values per angle on eight qubits give 16 777 216 actions per step. We therefore learn the policy itself with the policy gradient. The circuit is built along a template with D entangling layers,

$$
\begin{aligned}
|\psi(\boldsymbol{\theta})\rangle &= \hat{R}(\boldsymbol{\theta}_D)\,\hat{U}_{\text{ent}}\,\hat{R}(\boldsymbol{\theta}_{D-1})\,\hat{U}_{\text{ent}}\cdots\hat{U}_{\text{ent}}\,\hat{R}(\boldsymbol{\theta}_0)|0\ldots0\rangle, \\
\hat{R}(\boldsymbol{\theta}_t) &= \prod_{q=0}^{N-1}\hat{R}_y(\theta_{t,q}), \qquad \hat{U}_{\text{ent}} = \widehat{\text{CNOT}}_{N-2\to N-1}\cdots\widehat{\text{CNOT}}_{0\to1},
\end{aligned}
\tag{14.12}
$$

with $n_{\text{ang}} = N(D+1)$ angles $\theta_{t,q}$ and $D(N-1)$ CNOT gates (Figure 45). The angles are the parameters of the circuit, which gradient ascent optimises directly and a policy-gradient agent can choose as its actions $\mathbf{a}_t = (\theta_{t,0}, \ldots, \theta_{t,N-1})$. With $D = 1$, $\theta_{0,0} = \pi/2$ and all other angles zero, the template is the GHZ circuit, since $\hat{R}_y(\pi/2)|0\rangle = |+\rangle$. The gates $\hat{R}_y$ and CNOT are real matrices and produce real states only, and all targets of this section have real amplitudes up to a global phase. For the Ising chains the off-diagonal elements of $\hat{\mathcal{H}}$ in the basis of $\hat{Z}$ eigenstates come from the $\hat{X}$ terms only. The Hamiltonian is therefore a real symmetric matrix, and its non-degenerate ground state can be chosen real.

Before training an agent, we search for a sufficient template depth. The fidelity of the template is a differentiable function of the angles. Gradient ascent of $F(\boldsymbol{\theta})$ through the simulator, from 4 random

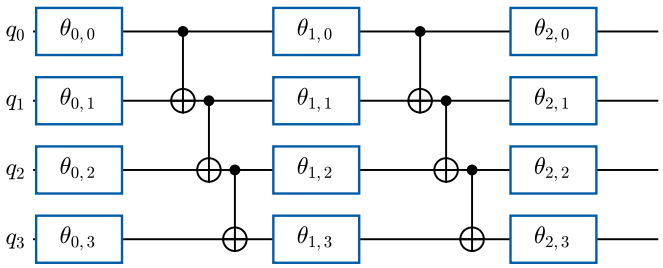


Figure 45: The circuit template (14.12) for $N = 4$ qubits and $D = 2$ entangling layers. Every box is a rotation $\hat{R}_y(\theta_{t,q})$, each entangling layer is the ladder of CNOT gates from q_0 to q_3, and time runs from left to right.

starts, gives the smallest tested number of layers D^* at which that search reaches $F \geq 0.99$ (Figure 46). The same gradient is available on a device without a simulator, through the parameter-shift rule [40,41]. For one qubit and the target $|1\rangle$, $F(\theta) = \sin^2(\theta/2) = \frac{1}{2}(1 - \cos\theta)$, and the difference of two fidelities shifted by $\pm\pi/2$ gives

$$\frac{F(\theta + \frac{\pi}{2}) - F(\theta - \frac{\pi}{2})}{2} = \frac{\cos(\theta - \frac{\pi}{2}) - \cos(\theta + \frac{\pi}{2})}{4} = \frac{1}{2}\sin\theta = F'(\theta), \tag{14.13}$$

the derivative itself, without any finite-difference error. The same holds for every gate $e^{-i\theta\hat{\mathcal{G}}/2}$ whose generator satisfies $\hat{\mathcal{G}}^2 = \mathbb{1}$. The gate is $\cos\frac{\theta}{2}\mathbb{1} - i\sin\frac{\theta}{2}\hat{\mathcal{G}}$, and the fidelity is quadratic in $\cos\frac{\theta}{2}$ and $\sin\frac{\theta}{2}$. In each angle it therefore has the form $a_0 + a_1\cos\theta + a_2\sin\theta$, for which the shift rule is exact. Then

$$\frac{\partial F}{\partial \theta_k} = \frac{F(\boldsymbol{\theta} + \frac{\pi}{2}\mathbf{e}_k) - F(\boldsymbol{\theta} - \frac{\pi}{2}\mathbf{e}_k)}{2}, \tag{14.14}$$

with $\mathbf{e}_k$ the unit vector of angle k, so a gradient costs $2n_{\text{ang}}$ fidelity evaluations. The explicit GHZ construction succeeds with one layer at every N. For the W state, with the same entropy of one bit, the search finds the deepest successful template. It needs $D^* = 2$ on four qubits, a tie with the TFIM ground state at $h = 0.5$, and 4 on six. On eight qubits the scan stopped at its limit $D = N = 8$ without a recorded confirmation that $F \geq 0.99$ was reached. The depth therefore depends on the structure of the state relative to the gate set in addition to the entanglement entropy.

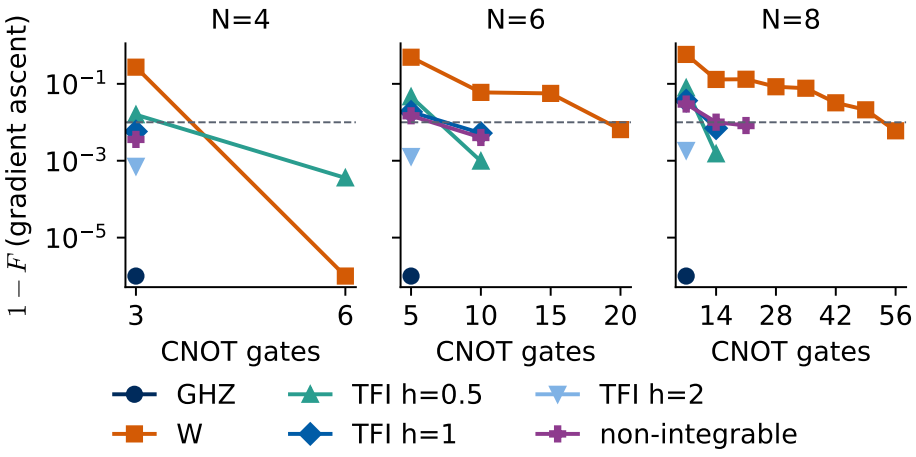


Figure 46: The infidelity $1 - F$ of the best of 4 gradient-ascent starts against the number of CNOT gates $D(N-1)$ of the template, on four, six and eight qubits. The depth runs over $D = 1, 2, \ldots$ until $F \geq 0.99$ (dashed line) or $D = N$.

Compilation with continuous angles

Episode

$D^* + 1$ steps $t = 0, \ldots, D^*$, one layer of the circuit per step. The depth D^* is found by

gradient ascent, so a circuit with $F \geq 0.99$ is known to exist.

Action
the N rotation angles $\mathbf{a}_t$ of layer t, real numbers, and for every bond $(q, q+1)$ a gate type $e_{t,q} \in \{\text{none}, \text{CNOT}, \text{CZ}\}$, a discrete choice. The last step has rotations only. With a CNOT on every bond the circuit is the template.

Reward
the fidelity F after the last layer, and zero before. The return of an episode is $G = F$.

Observation
the 2^N real amplitudes of the current state, supplied by the simulator, and the index t of the step. The index is written as a vector of $D^* + 1$ entries with a one at position t.

The reward is the final fidelity alone, which the discrete agent could not learn from. Here an episode has only $D^* + 1$ steps, each step sets a whole layer, and the template is known to be deep enough. The policy gradient weighs every action with the return of its whole episode in any case.

The policy is a neural network with three outputs, or heads. One head gives the means of Gaussians for the angles, one their widths, and one the probabilities of the gate types, which the agent samples to build one layer. The circuit that the agent proposes as its answer, the greedy circuit, uses the means as angles and the most probable gate type on every bond. The log-likelihood of one step is the sum of the Gaussian log-likelihoods of the angles and the softmax log-likelihoods of the gate types. The policy gradient (10.5) holds for this sum unchanged.

Policy network and training

Network
2 hidden layers of 64 units with the hyperbolic tangent (Figure 47).

Heads
means μ_q of the N angles and log-widths $\nu_q = \ln \sigma_q$ clipped to $[-5, 0]$, so that every width lies between $e^{-5} \approx 0.007$ and 1 rad. For every bond there are three logits z_e of a softmax over (none, CNOT, CZ), with the score $\partial \ln p_e / \partial z_j = \delta_{ej} - p_j$.

Start
widths near 0.5 rad and gate probabilities near $(0.05, 0.90, 0.05)$, so the search starts near the template and can learn to change gates.

Training
REINFORCE with reward-to-go and baseline, with $B = 32$ episodes per update and 1500 updates, 48 000 episodes per agent. The weight of episode i is $w_i = (G_i - \bar{G})/\text{std}(G)$, the return minus the batch-mean baseline divided by the batch standard deviation. This normalisation keeps the step useful when all fidelities of a batch approach one. The steps are taken with Adam, with the norm of each gradient clipped to 1.

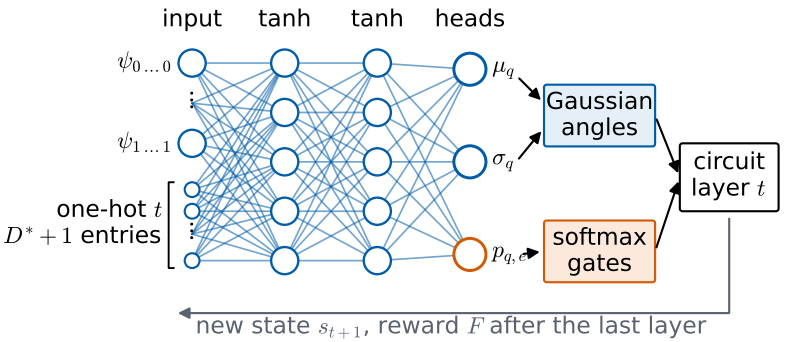


Figure 47: The policy network of the continuous compilation agent. The input is the state, its amplitudes $\psi_{s_0\ldots s_{N-1}}$, and the step index t, a one-hot vector of D^*+1 entries drawn as a column of small nodes. Two hidden layers of tanh units feed three heads, the means μ_q and widths σ_q of the Gaussian angles and the softmax probabilities $p_{q,e}$ of the gate types. The sampled angles and gates form one layer of the circuit, and the new state is the next input.

14.13 The gradient for angles and gate types

Conditioned on the current state, the continuous-compilation policy samples independent Gaussian angles and a categorical gate on each bond. With $p_{q,e}(s_t)$ the probability of gate type e on bond q, the joint density of the policy is

$$\pi_\theta(\mathbf{a}_t, \mathbf{e}_t \mid s_t) = \prod_q \mathcal{N}(a_{t,q}; \mu_q(s_t), e^{2\nu_q(s_t)}) \prod_{q \text{ a bond}} p_{q,e_{t,q}}(s_t), \tag{14.15}$$

$$\ln \pi_\theta = -\sum_q \left[\nu_q + \frac{(a_{t,q}-\mu_q)^2}{2e^{2\nu_q}} + \frac{1}{2}\ln(2\pi)\right] + \sum_{q \text{ a bond}} \ln p_{q,e_{t,q}}. \tag{14.16}$$

The bond term is omitted in the final layer, which has rotations only. Holding the sampled angle fixed gives

$$\partial_{\mu_q} \ln \pi_\theta = \frac{a_{t,q}-\mu_q}{\sigma_q^2}, \tag{14.17}$$

$$\partial_{\nu_q} \ln \pi_\theta = \frac{(a_{t,q}-\mu_q)^2}{\sigma_q^2} - 1, \tag{14.18}$$

$$\partial_{z_{q,j}} \ln p_{q,e_{t,q}} = \mathbf{1}\{j = e_{t,q}\} - p_{q,j}. \tag{14.19}$$

The chain rule of the network combines these output derivatives with the derivatives with respect to its parameters. For a fixed-depth terminal-fidelity objective with $\gamma = 1$, the same final fidelity weights the scores of every layer.

The Gaussian means and the most probable gates define one deterministic circuit, whose fidelity can differ from the mean fidelity under the stochastic policy, so the evaluation reports both. The depth search finds a successful circuit at a tested depth, and a failure at a smaller depth can come from the optimisation.

14.14 Learned continuous-angle circuits

The policy gradient compiles the targets without differentiating through a model of the circuit. Within the training budget it reaches $F \geq 0.99$ for all of them except the W state on six qubits and the non-integrable ground state on four qubits. It needs between 1.7 and 4.1 times more fidelity evaluations than gradient ascent with the parameter-shift rule. This range covers the targets on two, four and six qubits that both methods reach. The agents reach high fidelities for every target family on two, four and six qubits, close to the fidelity that gradient ascent reaches with the same template (Figure 48). The one clear shortfall is the non-integrable ground state on four qubits, where the

median is 0.9607 against 0.9964 for gradient ascent. Two of the three agents there settled on circuits with fewer or different two-qubit gates. The W state on six qubits, with 4 layers and 30 angles, is still improving at the end of the training budget. Consider the agent for the critical TFIM ground state on four qubits. During training the widths of its Gaussians contract from 0.5 rad to the lower bound of about 0.007 rad. The policy becomes nearly deterministic in every angle.

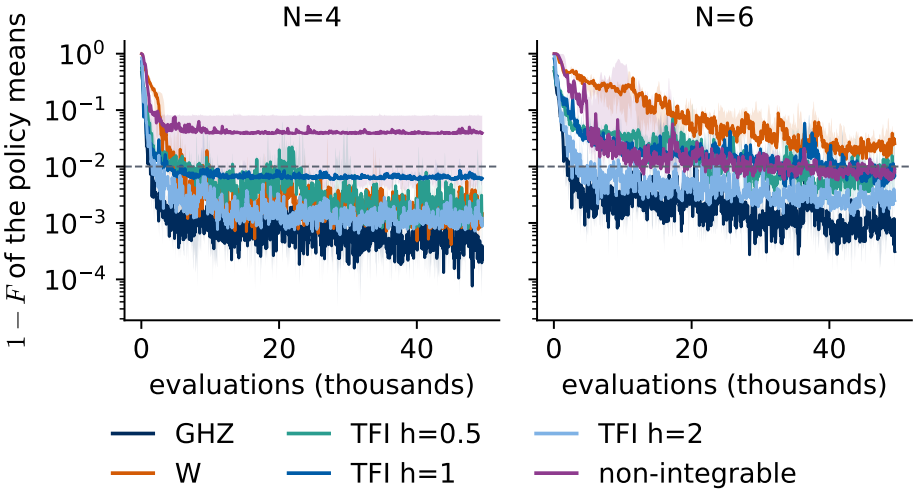


Figure 48: Training of the policy-gradient agents on four and six qubits. The infidelity $1 - F$ of the greedy circuit against the number of fidelity evaluations, $B + 1$ per update for the B sampled episodes and one evaluation of the greedy circuit. Lines show the median over three seeds with the band between the best and the worst seed. The dashed line marks $F = 0.99$.

The cost of a compilation is the number of fidelity evaluations until the greedy circuit first reaches $F \geq 0.99$, since on a device every evaluation is an experiment. The cost grows with the depth of the template more than with the entanglement entropy, and the depth reflects the structure of the target relative to the gate set. Gradient ascent with the parameter-shift rule needs fewer fidelity evaluations for every target that both methods reach, on six qubits between about 1.9 and 3.0 times fewer. Gradient ascent was given the template, and the policy-gradient agents the depth D^* found by the gradient-ascent scan, whose cost the ratio excludes. The gradient-ascent count is taken per successful start and needs the parameter-shift gradient. The policy gradient uses only the returned fidelities and chooses discrete gates and continuous angles in one framework.

Training with measured fidelities works at a cost in precision. With the reward estimated from 1000 shots, the median final fidelity is 0.9686 for the critical TFIM ground state on six qubits, against 0.9930 with the exact reward. For the W state on four qubits it is 0.9766, against 0.9985. Near $F = 1$ the differences between the sampled circuits of a batch become comparable with the measurement error of about 0.003. Rounding every learned angle to the nearest multiple of $\pi/4$ tests the sensitivity of the learned circuit to an angle grid. The GHZ state keeps $F = 1.0000$ on six qubits. The W state drops to 0.9161 on four qubits and 0.7540 on six, and the critical TFIM ground state on six qubits to 0.8765. The discrete DQN and DDQN agents of the uniform-cost survey reached lower fidelities on the same states, for example 0.682 for the critical six-qubit TFIM ground state. In the two setups the discrete agents choose freely among their gates within at most 16 gates. The policy-gradient agents fill a template of up to 50 gates with continuous angles.

14.15 Discussion

Our compilation agents work with exact state vectors of at most six qubits for training. Deep reinforcement learning has been applied to quantum compiling on larger problems, for example for topological quantum computation [42] and for approximating single-qubit unitaries with discrete gate sets [43]. Our small examples do not repeat any result of these works. At our size the references are strong. The exhaustive search gives the exact optimum for two and three qubits, and the GHZ

circuit and its 2^{N-1} equivalent forms are known for every N. Gradient ascent optimises the angles of a template with fewer fidelity evaluations than the policy gradient.

A reward that pays every gain in fidelity guided the discrete agent where a reward for the final fidelity alone did not. With rewards measured from ten shots DDQN, which reduces the overestimation, learned the three-qubit GHZ circuit in three of five seeds and DQN in none. In the CNOT-cost comparison neither family reached the six-qubit GHZ state or the W state beyond two qubits. For compilation, reinforcement learning is worth its cost where these references fail, when the space of circuits is too large to search or no model of the device exists.

15 State preparation by switching Hamiltonians

Many experiments cannot apply arbitrary gates but can switch fields and couplings on and off, and the quantum state evolves under the resulting Hamiltonian. Preparing a target state then means choosing a time dependence of these controls, a problem of quantum control. We follow Ref. [44], in which ground states of interacting spin chains are prepared with reinforcement learning. We reimplement this study at small size with exact state vectors and state the differences from it where they arise.

15.1 Ground states of an Ising chain

Reference [44] prepares ground states of the Ising chain (14.2) on an open chain. It writes the Hamiltonian as $\hat{\mathcal{H}} = J\sum \hat{Z}_j\hat{Z}_{j+1} - g_x\sum \hat{X}_j - g_z\sum \hat{Z}_j$, so that its J is our $-J$, its g_x our h and its g_z our g. The main target is the ground state of the transverse-field Ising model at $J = 1$ and $h = 1$, the point of the quantum phase transition of the infinite chain. It is also the target of the four-spin study in that work. The second target is the ground state of the positive-field longitudinal chain, the non-integrable chain with uniform fields $J = -1$, $h = 0.9045$ and $g = 0.8090$ at the parameter point of Section 14. Compared with the negative-field chain of Section 14 both fields have the opposite sign. The unitary $\prod_i \hat{Y}_i$, a rotation of every spin by π about y, changes the sign of every $\hat{X}_i$ and every $\hat{Z}_i$ and leaves $\hat{Z}_i\hat{Z}_{i+1}$ unchanged. The two chains therefore have the same spectrum, while their ground states are different states. Every control problem starts in the state with all spins up, $|\uparrow\ldots\uparrow\rangle = |0\ldots0\rangle$.

The reference ground states come from exact diagonalisation of the $2^N \times 2^N$ Hamiltonian matrix, and a Lanczos run gives the same energies to six decimals. Table 6 lists the targets for four and six spins. The overlap of the all-up state with the target falls as the chain grows and is much smaller for the longitudinal chain, whose antiferromagnetic coupling disfavours aligned neighbours. The half-chain entanglement entropy (13.4) of all four targets stays below half a bit. In a chain of six spins the transition of the transverse-field chain is a smooth crossover (Figure 49). Deep in the ferromagnet, at $h = 0.5$, the ground state is close to the superposition of all-up and all-down with almost one bit of entanglement. In the paramagnet, at $h = 2$, it is close to the product state $|+\cdots+\rangle$.

N	target	E_0	gap	F with $\|0\ldots0\rangle$	$S_{N/2}$ (bits)
4	TFIM, $h = 1$	-4.7588	0.695	0.230	0.411
4	longitudinal, $h, g > 0$	-4.8424	0.732	0.059	0.399
6	TFIM, $h = 1$	-7.2962	0.482	0.160	0.473
6	longitudinal, $h, g > 0$	-7.3788	0.527	0.009	0.394

Table 6: The ground-state targets. The ground-state energy E_0 and the gap to the first excited state come from exact diagonalisation. The last columns give the fidelity of the all-up initial state with the target and the half-chain entanglement entropy.

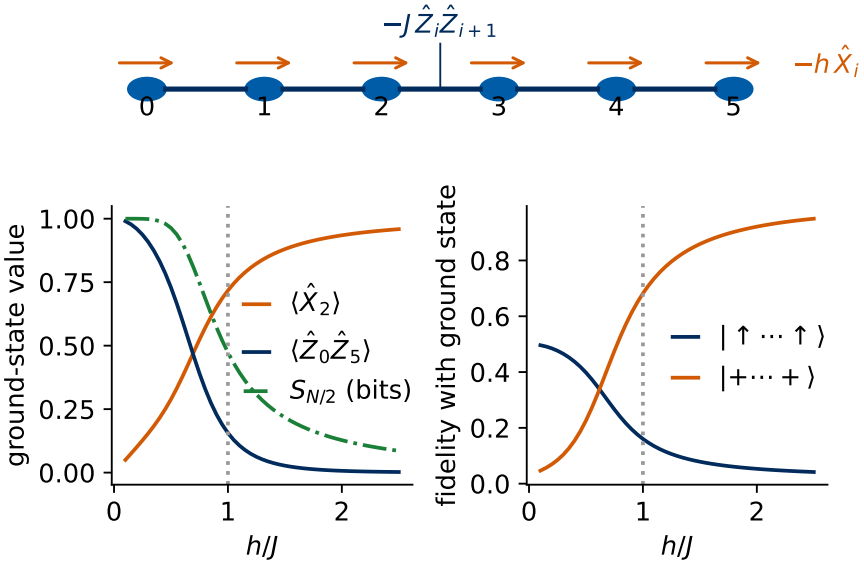


Figure 49: The ground state of the transverse-field Ising chain of six spins against the field h/J, from exact diagonalisation. Left, the transverse magnetisation $\langle\hat{X}_2\rangle$ of a middle spin, the end-to-end correlation $\langle\hat{Z}_0\hat{Z}_5\rangle$ and the half-chain entanglement entropy $S_{N/2}$. Right, the fidelity of two product states with the ground state. The dotted line marks $h = J$.

15.2 Evolutions under Hamiltonian terms

Like an analog quantum simulator, the agent of Ref. [44] switches on one term of a Hamiltonian for a short time and lets the whole chain evolve. The six available terms are a uniform field along x, y or z and a uniform nearest-neighbour coupling of the same three components,

$$\hat{O} \in \Big\{ \sum_i \hat{X}_i,\ \sum_i \hat{Y}_i,\ \sum_i \hat{Z}_i,\ \sum_i \hat{X}_i\hat{X}_{i+1},\ \sum_i \hat{Y}_i\hat{Y}_{i+1},\ \sum_i \hat{Z}_i\hat{Z}_{i+1} \Big\}, \tag{15.1}$$

and each can be applied in two ways,

$$\hat{U}^{+}_{\hat{O}} = e^{+it_+\hat{O}}, \quad t_+ = \pi/8, \qquad\qquad \hat{U}^{-}_{\hat{O}} = e^{-it_-\hat{O}}, \quad t_- = \pi/13, \tag{15.2}$$

with the steps of the four-spin study of Ref. [44]. The $6 \times 2 = 12$ coarse actions form the action set, and a protocol is a sequence of them (Figure 50). The evolution (15.2) is the time evolution under the Hamiltonian $\mp\hat{O}$ for the time $t_\pm$. All terms of one sum commute with each other, so each action factorises without approximation into N single-spin rotations or $N-1$ two-spin rotations. For example, $\hat{U}^{-}_{\hat{Y}} = \prod_i e^{-it_-\hat{Y}_i}$ turns every spin by the angle $2t_-$ about the y axis, from z towards $+x$. The two steps differ, so an action is never undone by the action of opposite sign, a choice that the source study reports to stabilise the training. For a second agent we add fine versions of all twelve actions with half the steps, $\pi/16$ and $\pi/26$, which gives 24 actions.

Short protocols can be checked exhaustively. There are 12^n coarse sequences of length n, about 3.6×10^7 for $n = 7$. Table 7 gives the best fidelity they reach from the all-up state of six spins for four ground states. Only the paramagnetic ground state at $h = 2$, close to a product state, comes close to one, and three actions already give it $F > 0.9$. Seven global actions do not build the ferromagnetic ground state at $h = 0.5$, which has almost one bit of entanglement. For the two targets of the control problem the best seven-action sequences reach 0.933 and 0.919. The number of sequences grows as 12^n, so longer protocols are out of reach of this search.

target ($N = 6$)	$S_{N/2}$ (bits)	shortest sequence with $F \geq 0.9$	best F of seven actions
TFIM, $h = 0.5$	0.964	>7	0.646
TFIM, $h = 1$	0.473	5	0.933
TFIM, $h = 2$	0.128	3	0.991
longitudinal, $h, g > 0$	0.394	6	0.919

Table 7: Exhaustive search over all sequences of up to seven coarse actions from the all-up state of six spins. The columns give the half-chain entropy of the target, the length of the shortest sequence that reaches $F \geq 0.9$, and the best fidelity of seven actions.

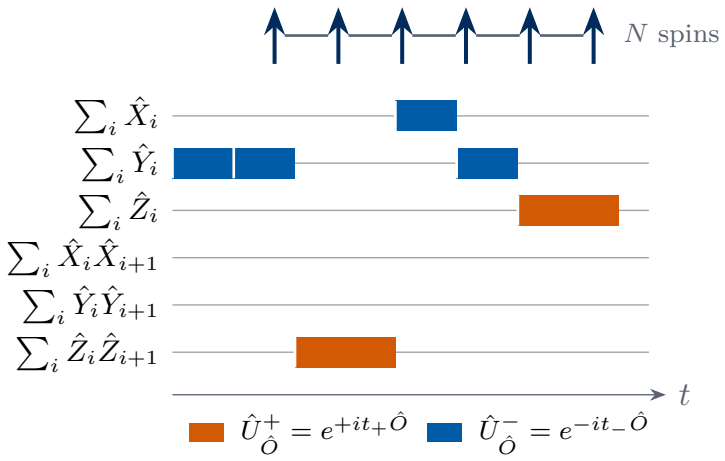


Figure 50: An analog protocol drawn as an illustration. Each row is one of the six Hamiltonian terms $\hat{O}$ of equation (15.1), which act on all N spins at once, and time runs to the right. Each bar is one action, an evolution under one term, orange for $\hat{U}^+_{\hat{O}}$ with the duration t_+ and blue for $\hat{U}^-_{\hat{O}}$ with t_-. The bar length is proportional to the duration.

15.3 The ground-state decision process

In Ref. [44] the agent chooses every action from the current quantum state, which allows it to react to errors. The reward after each action is the log-fidelity per spin,

$$r_{t+1} = \frac{1}{N} \ln F_{t+1}, \qquad F_{t+1} = |\langle \psi_{\text{target}} | \psi_{t+1} \rangle|^2, \tag{15.3}$$

with the fidelity (14.1) of the new state with the target. Figure 51 shows the control tasks of that work on the phase diagram of the Ising chain.

Ground-state preparation

State
the quantum state $|\psi_t\rangle$ of the chain. Reference [44] stores the state of long chains in a compressed tensor-network representation. For $N \leq 6$ we use the exact state vector and give the network its 2^N amplitudes as 2^{N+1} real numbers. The global phase is fixed so that the overlap with the target is real and positive, which gives two vectors of the same physical state the same input.

Action
one of the 12 coarse evolutions $\hat{U}^{\pm}_{\hat{O}}$.

Reward
the log-fidelity per spin (15.3) after each action. The fidelity of a many-body state typically falls exponentially with N, so $\ln F$ grows in proportion to N and the division keeps the reward of order one. Every reward is negative and approaches zero at the target, so short protocols collect the least negative return.

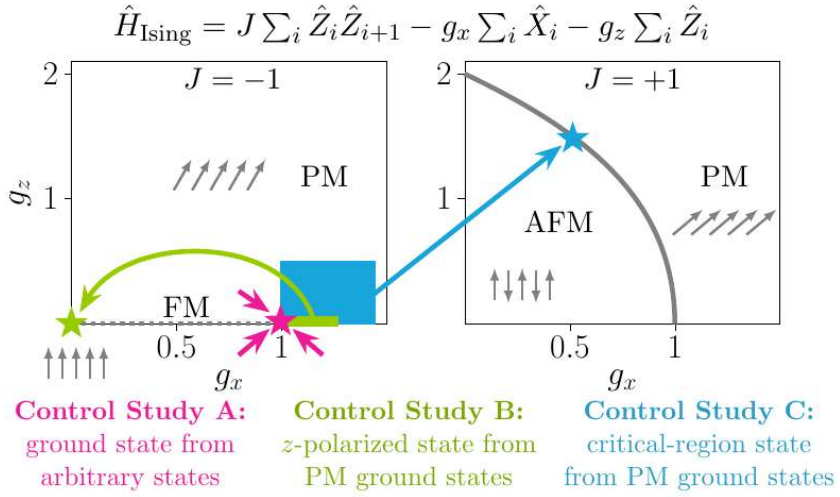


Figure 51: The control tasks of Ref. [44] on the phase diagram of the Ising chain $\hat{H}_{\text{Ising}} = J\sum_i \hat{Z}_i\hat{Z}_{i+1} - g_x\sum_i \hat{X}_i - g_z\sum_i \hat{Z}_i$. The axes are the transverse field g_x and the longitudinal field g_z, for a ferromagnetic coupling $J = -1$ (left) and an antiferromagnetic coupling $J = +1$ (right). FM, AFM and PM mark the ferromagnetic, antiferromagnetic and paramagnetic phases, and the arrows sketch the spin configurations. Study A prepares the ground state at the star from arbitrary states, study B a z-polarised state from paramagnetic ground states, and study C a state in the critical region. Their J, g_x and g_z are our $-J$, h and g.

Episode
ends when F reaches a threshold F^* or after 50 actions.

A single agent of this kind does not reach arbitrarily high fidelities, for three reasons given in Ref. [44]. The steps of the actions are fixed and the episode length is fixed. Close to the target all log-fidelities approach zero, so the differences between the rewards of good and very good states become too small for the network to resolve. The exhaustive search shows the first reason at our size. For both six-spin targets no coarse sequence of up to seven actions reaches the fidelity 0.953 that the final controller has to reach. A threshold that the exploring agent almost never meets gives it no example of success to learn from. Two agents are therefore trained [44]. The first agent works with the coarse steps and stops at the threshold F_1^*. The second agent starts from the states that the first one prepares, works with finer steps and stops at the threshold F_2^*, with

$$F_1^* = 0.96^N, \qquad F_2^* = 0.992^N. \tag{15.4}$$

For $N = 4$ the thresholds (15.4) are $F_1^* = 0.849$ and $F_2^* = 0.968$, and for $N = 6$ they are $F_1^* = 0.783$ and $F_2^* = 0.953$. In the four-spin study of Ref. [44] the combined agents reach F_2^* within 50 actions from about 93 percent of random initial states. With 100 actions they reach it from 99.5 percent.

As in Ref. [44], both agents are trained by deep Q-learning with experience replay, a target network and the Double DQN loss (9.2). In this loss the online network chooses the next action and the target network evaluates it. The network is a multilayer perceptron, a stack of fully connected layers, with two hidden layers (Figure 52). Training episodes start from the all-up state with every spin tilted away from the z axis by a random angle $|\mathcal{N}(0, \sigma_{\text{tilt}}^2)|$ in a random azimuthal direction. The tilt scale is $\sigma_{\text{tilt}} = 0.15$. The agent thus meets a variety of states. Our second agent differs from that of Ref. [44] in two ways that address the stall directly. It keeps the twelve coarse actions next to the twelve fine ones, so it can still make large corrections, and its reward is

$$r_{t+1} = \frac{\ln F_{t+1}}{|\ln F_1^*|}, \tag{15.5}$$

which is -1 at the threshold of the first agent and 0 at the target. Its action values are then of order one, and the network can resolve the small differences between states near the target. This rescaling multiplies all rewards by one constant and leaves the optimal policy unchanged.

After training the two agents act together as one controller. Before every action it computes the fidelity with the target and stops once $F \geq F_2^*$. It lets the first agent act while $F < F_1^*$ and the second agent otherwise. An error that throws the state back below F_1^* therefore hands the control back to the first agent. At test time the controller has a budget of 100 actions, twice the training episode, as in the source study.

The main remaining differences from the source study are the size and the state representation. We simulate four and six spins against 4, 16 and 32, and we use the exact state vector in place of its tensor-network representation. Our training and test starts are tilted all-up states, while the source study also uses random product states and random reflection-symmetric states. Appendix C lists the training settings.

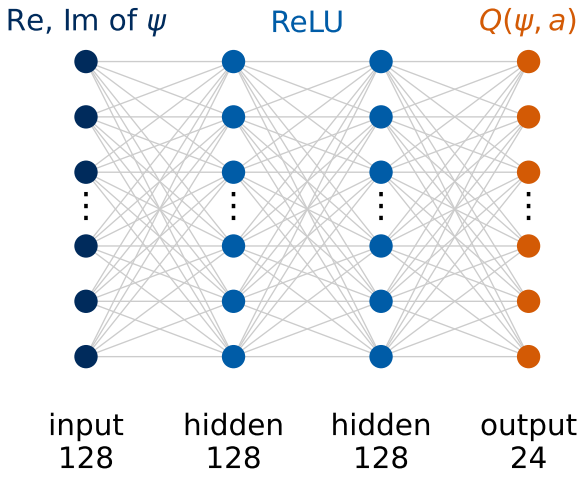


Figure 52: The Q-network of the second agent for six spins. The 128 inputs are the real and imaginary parts of the 64 amplitudes, followed by two hidden layers of 128 rectified linear units and one output $Q(\psi, a)$ for each of the 24 actions.

15.4 Learned protocols

The first agents learn quickly (Figure 53, left). After 4000 training iterations every run succeeds in at least 99 percent of its latest training episodes. From 512 new tilted starts every first agent reaches F_1^* in at least 509 runs, with five to seven actions on average. The second task is much harder, and the success rate of the second agents stays at zero for several thousand episodes and saturates below one. At the end of training it is 0.73 to 0.79 for the four-spin TFIM runs and 0.41 to 0.61 for the four-spin longitudinal-chain runs. For six spins it is 0.24 to 0.47 and 0.09 to 0.25. These rates are measured with exploration switched on, a budget of 50 actions and second-agent episodes that start from states prepared by the first agent. The tests of the combined greedy controller use other start distributions, so the two kinds of rate cannot be compared directly.

From the exact all-up state all six controllers for four spins reach $F_2^* = 0.968$. They need $12, 13, 10$ actions with final fidelities $0.970, 0.972, 0.969$ for the TFIM target and $15, 21, 16$ actions with $0.973, 0.970, 0.971$ for the longitudinal-chain target, one number per seed. For six spins all three TFIM controllers reach $F_2^* = 0.953$, with $19, 19, 13$ actions and $F = 0.954, 0.957, 0.957$. For the six-spin longitudinal-chain target one controller of three reaches the threshold, after 47 actions. The other two end after 100 actions at $F = 0.882$ and 0.908, oscillating below it. Every protocol begins with five actions of the first agent, three of them $\hat{U}_{\hat{Y}}^{-}$. Together these three turn every spin by $3 \cdot 2\pi/13 \approx 83°$ from z towards $+x$, the direction of the transverse field. The second agent then raises the fidelity in small steps of both signs, and the fidelity no longer increases monotonically (Figure 54). The energies of the prepared states lie above the exact ground-state energies, for the shortest six-spin TFIM protocol -7.1642 against $E_0 = -7.2962$.

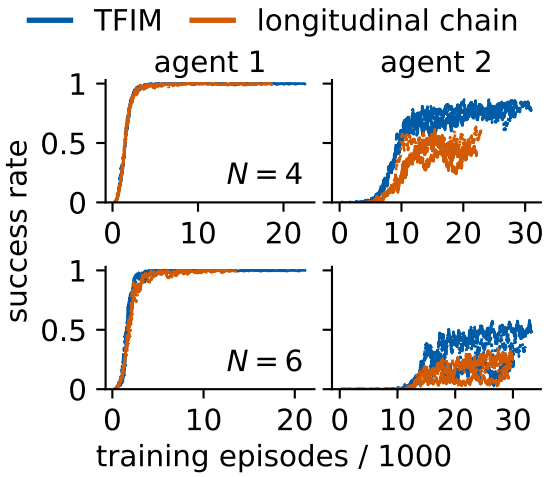


Figure 53: Training of the ground-state agents. The success rate, the fraction of training episodes that reach the threshold of the agent, is shown as a running mean over 300 episodes against the number of finished training episodes. Panels show the first agents (left) and the second agents (right), for four spins (top) and six spins (bottom). Colour marks the target and line style the seed. The rates are measured with exploration switched on. Runs whose episodes end early finish more episodes in the same number of iterations.

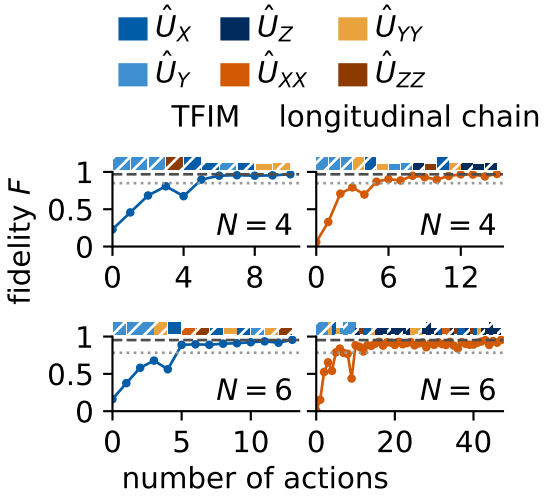


Figure 54: The fidelity F with the target after each action of the controller, from the all-up state, for each chain length and target, shown for the seed with the shortest successful protocol. The boxes above each panel are the actions, coloured by the Hamiltonian term $\hat{O}$. They are tall for a coarse step and short for a fine step, hatched for $\hat{U}^-_{\hat{O}}$ and plain for $\hat{U}^+_{\hat{O}}$. The dotted line is F_1^* and the dashed line F_2^*.

We test the controllers from 300 tilted starts drawn like the training starts. Within 100 actions they reach F_2^* in 80 to 83 percent of the runs for the four-spin TFIM target and 31 to 70 percent for the four-spin longitudinal-chain target. For six spins the rates are 26 to 57 percent and 3 to 24 percent, the ranges over the three seeds. Within 50 actions the four-spin TFIM controllers succeed in 77 to 80 percent of the runs, on a narrower family of starts. Reference [44] reports 93 percent for random initial states with its tensor-network Q-network.

A fixed sequence of actions is an open-loop protocol, applied in the same way whatever happens to the system. The trained controller chooses every action from the current state, a closed loop (Figure 55). Started from all-up states tilted with the scale $\sigma_{\text{tilt}} = 0.2$, the four-spin TFIM controllers reach F_2^* in 53 of 100 runs, while replaying their own fixed protocol succeeds in 2. The result resembles the self-correction reported for longer chains in Ref. [44]. The two loops differ in access. The closed-loop controller reads the exact fidelity before every action, stops at the first crossing of F_2^* and may use up to 100 actions. The replayed protocol has the fixed length of one successful run and

typically ends just above the threshold. Success at this threshold therefore exaggerates the difference in final fidelity.

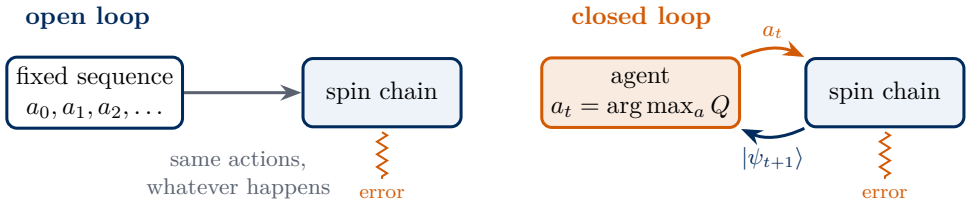


Figure 55: Open and closed loop. Left, a fixed sequence of actions is applied to the spin chain whatever happens, so an error (zigzag) is never corrected. Right, the agent chooses each action $a_t = \arg\max_a Q$ from the current state $|\psi_{t+1}\rangle$ that the chain returns and can react to an error.

15.5 Limits

The six-spin longitudinal-chain target marks the limit at this size. One controller of three reaches it from the all-up state, and at most a quarter of the runs reach it from tilted starts. Reference [44] also controls chains of 16 and 32 spins with a tensor-network Q-network and transfers trained agents to 8 and 64 spins, a scale beyond exact state vectors. Without reinforcement learning, short protocols can be found by exhaustive search and, more generally, by gradient-based optimal control of the protocol, which needs a model of the dynamics. Reinforcement learning adds the closed loop, a controller that reacts to deviations. Here the controller sees the exact simulated state and fidelity, whereas a device controller would have to work from measurement records.

16 A one-photon state in a leaky cavity

16.1 One mode of light

We apply the policy gradient to a feedback task in which the agent sees only a noisy measurement record and never the quantum state. The agent learns to hold one photon in a leaky cavity with a probability that no drive without feedback can reach. The cavity loses photons through its mirrors, and the only control is a classical beam that drives it, while the only information is the noisy reading of a photon-number meter. The example and the method come from Ref. [47], and we reimplement them in a simplified form.

The target is the one-photon Fock state $|1\rangle$, the state of the cavity mode with exactly one photon, with $\hat{\rho}_{\text{target}} = |1\rangle\langle 1|$. Its fidelity is the one-photon probability $P_1(t) = \langle 1|\hat{\rho}_{\text{c}}(t)|1\rangle$, where $\hat{\rho}_{\text{c}}(t)$ is the state of the cavity conditioned on the meter readings up to time t. The observation is a recent noisy photon-number record, and the action is the next drive amplitude ε_t. The symbol ε denotes the drive strength throughout this section and is unrelated to the exploration rate of Part I. We use the reward $r_{t+1} = P_1(t+1)$ and maximise $\mathbb{E}_{\pi_\theta}[T^{-1}\sum_{t=0}^{T-1} r_{t+1}]$ over policy parameters θ, an objective that rewards maintaining one photon throughout the episode.

Two mirrors facing each other trap light of the frequencies whose standing waves fit between them. We keep one of these standing waves, one mode of frequency ω_c. Its electric field oscillates like the position of a mass on a spring, so in quantum mechanics the mode is a harmonic oscillator,

$$\hat{\mathcal{H}}_{\text{c}} = \hbar\omega_c(\hat{a}^\dagger\hat{a} + \tfrac{1}{2}), \qquad E_n = \hbar\omega_c(n + \tfrac{1}{2}), \quad n = 0, 1, 2, \ldots \tag{16.1}$$

The energy levels E_n are equally spaced, and the eigenstate $|n\rangle$ holds n quanta of energy $\hbar\omega_c$, n photons. These are the Fock states, and $|0\rangle$ is the empty cavity. The annihilation operator removes a photon and the creation operator adds one,

$$\hat{a}|n\rangle = \sqrt{n}\,|n-1\rangle, \qquad \hat{a}^\dagger|n\rangle = \sqrt{n+1}\,|n+1\rangle, \qquad \hat{n} = \hat{a}^\dagger\hat{a}, \quad \hat{n}|n\rangle = n|n\rangle. \tag{16.2}$$

16.2 A classical drive makes a coherent state

A laser or microwave beam at the mode frequency drives the mode. In the frame rotating with ω_c only the drive is left,

$$\hat{\mathcal{H}}_\mathrm{d} = i\varepsilon(\hat{a}^\dagger - \hat{a}), \tag{16.3}$$

with $\hbar = 1$ and a real drive strength ε. Acting on the empty cavity for a time t, the drive produces the coherent state $|\alpha\rangle$ with $\alpha = \varepsilon t$. A classical oscillating current or a laser produces a field in this state. In the Fock basis the coherent state reads

$$|\alpha\rangle = e^{-|\alpha|^2/2} \sum_{n\geq 0} \frac{\alpha^n}{\sqrt{n!}} |n\rangle, \qquad P(n) = |\langle n|\alpha\rangle|^2 = e^{-\bar{n}} \frac{\bar{n}^{\,n}}{n!}, \qquad \bar{n} = |\alpha|^2, \tag{16.4}$$

a Poisson distribution of the photon number with mean and variance $\bar{n}$ (Figure 56). The probability of one photon is $P(1) = \bar{n}e^{-\bar{n}}$. Its derivative $(1-\bar{n})e^{-\bar{n}}$ vanishes at $\bar{n} = 1$, so

$$\max_{\bar{n}} P(1) = e^{-1} \approx 0.368. \tag{16.5}$$

A drive with fluctuating strength produces a mixture of coherent states. The value of $P(1)$ for a mixture is the average over its members, so it never exceeds e^{-1} either. The Fock state $|1\rangle$ has $P(1) = 1$ and no spread of the photon number. Mandel's parameter $Q_\mathrm{M} = \mathrm{Var}\, n/\bar{n} - 1$ compares the spread with the Poisson value [48]. It is 0 for every coherent state and at least 0 for every mixture of coherent states, while $|1\rangle$ has $Q_\mathrm{M} = -1$. Light with $Q_\mathrm{M} < 0$ is called nonclassical, and a linear drive alone cannot produce it. Going beyond e^{-1} requires an element that acts differently on different photon numbers, here a photon-number meter together with feedback.

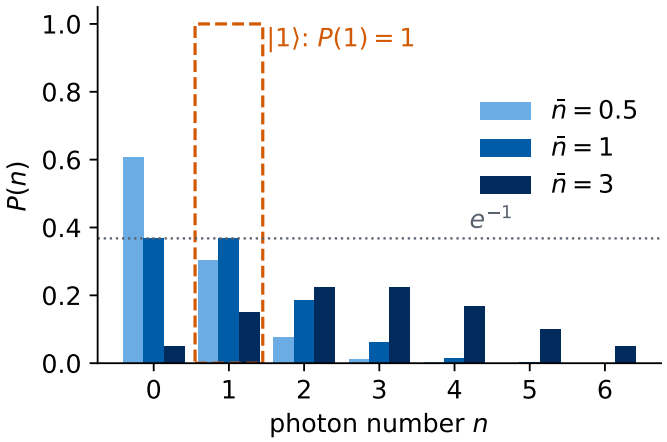


Figure 56: The photon-number distribution $P(n)$ of coherent states for three mean photon numbers $\bar{n}$. The probability of one photon is largest, e^{-1}, at $\bar{n} = 1$.

16.3 Loss and the meter

Each photon leaves the cavity with probability $\kappa\, dt$ in a short time dt, so a cavity with n photons loses one with probability $\kappa n\, dt$. The rate κ sets the unit of time, and all times are given in units of $1/\kappa$. We simulate the loss as a random process, one run of the cavity at a time, called a quantum trajectory [49]. In each short simulation step Δt_s a photon leaves with probability $\kappa \Delta t_\mathrm{s} \langle\hat{n}\rangle$, and the state jumps to $\hat{a}|\psi\rangle$, normalised. Otherwise the state changes to $\sqrt{1 - \kappa\Delta t_\mathrm{s}\hat{n}}\,|\psi\rangle$, normalised. This step reduces the weight of states with many photons, since a stretch of time without a leaked photon is evidence that the cavity holds few.

The average over many trajectories is described by the density matrix $\hat{\rho} = \overline{|\psi\rangle\langle\psi|}$, the average of the projectors of the trajectory states. Its diagonal elements ρ_{nn} are the probabilities of n photons, and its off-diagonal elements ρ_{nm} record the coherence between different photon numbers. To first

order in Δt_{s} the two rules, averaged in this way, give the Lindblad master equation. It is the standard equation of motion of a quantum system that loses energy to its surroundings,

$$\frac{d\hat{\rho}}{dt} = -i[\hat{\mathcal{H}}_{\mathrm{d}}, \hat{\rho}] + \kappa\Big(\hat{a}\hat{\rho}\hat{a}^{\dagger} - \tfrac{1}{2}\hat{a}^{\dagger}\hat{a}\hat{\rho} - \tfrac{1}{2}\hat{\rho}\hat{a}^{\dagger}\hat{a}\Big). \tag{16.6}$$

The first term is the evolution under the drive, and the second describes the loss of photons. With drive and loss a coherent state stays coherent. A jump leaves it unchanged, $\hat{a}|\alpha\rangle = \alpha|\alpha\rangle$, the evolution without a jump only shrinks its amplitude, and the drive adds to it. The amplitude therefore obeys $\dot{\alpha} = \varepsilon - \kappa\alpha/2$, derived in Appendix E, and settles at $\alpha = 2\varepsilon/\kappa$. A constant drive $\varepsilon = \kappa/2$ holds the cavity at $\bar{n} = 1$ and approaches $P(1) = e^{-1}$. Counting the leaked photons does not help, since the state stays coherent whatever the clicks, and $P(1)$ stays at most e^{-1}.

As in Ref. [47], the controller is given a weak measurement of the photon number inside the cavity. The measurement is of the quantum nondemolition kind, which gains information about the photon number without changing it. A photodetector, by contrast, absorbs the photons it counts. In circuit quantum electrodynamics such a meter is built from a qubit or a second mode whose frequency depends on the photon number of the cavity. Every $\Delta t_{\mathrm{c}} = 0.05/\kappa$ the meter shows the number

$$x_t = n + \sigma_{\mathrm{m}}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0,1), \tag{16.7}$$

where n is drawn with probability $|\psi_n|^2$ and $\sigma_{\mathrm{m}} = 1$ is the noise of a single reading. After the reading we update the state by Bayes' rule, multiplying each amplitude by the square root of the likelihood of the reading,

$$\psi_n \;\to\; \frac{1}{\mathcal{N}_t}\,\psi_n \exp\Big[-\frac{(x_t - n)^2}{4\sigma_{\mathrm{m}}^2}\Big], \tag{16.8}$$

with a normalisation $\mathcal{N}_t$. This update depends on the state and on the random reading, and it is the nonlinear element of the model. A single reading is unreliable, since the rule "one photon if $x_t > 1/2$" confuses an empty cavity with one photon with probability 0.31, and after averaging four readings with probability 0.16. Repeated readings alone turn a coherent state with $\bar{n} = 1$ into a Fock state by chance. In our simulation, with loss and drive switched off, a fraction 0.369 of such trajectories ends in $|1\rangle$, the value e^{-1} that quantum mechanics predicts. Feedback has to make this chance large and to refill the cavity after each loss.

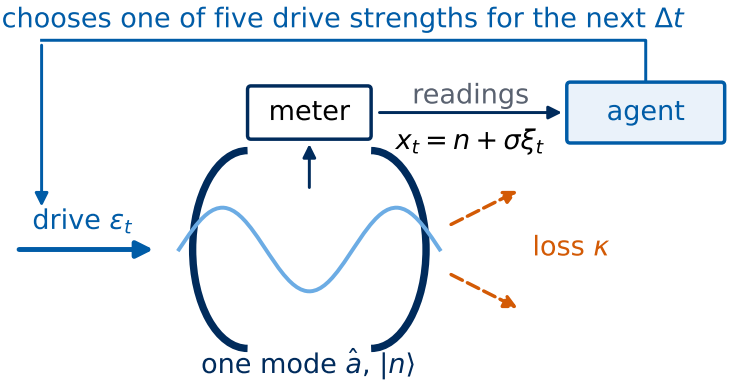


Figure 57: The control loop of the cavity. One mode $\hat{a}$ with Fock states $|n\rangle$ loses photons at the rate κ. A meter reads $x_t = n + \sigma_{\mathrm{m}}\xi_t$ every Δt_{c}, and the agent receives only these readings and chooses one of five drive strengths ε_t for the next interval.

16.4 The feedback result of the source study

Figure 58 shows the feedback setting and the learning result of the numerical example of Ref. [47], which is itself a simulation. A recent segment of the weak photon-number measurement record is passed to a network, which returns probabilities for the available displacement drives. The training

plot shows the drive and the one-photon probability over successive epochs, with a population of about 50% in the source example. The task is the preparation and stabilisation of a photon-number state under loss, with a measured observation in place of the exact state vector used in the compilation examples. Our own model, defined above, simplifies the measurement to discrete Gaussian readings.

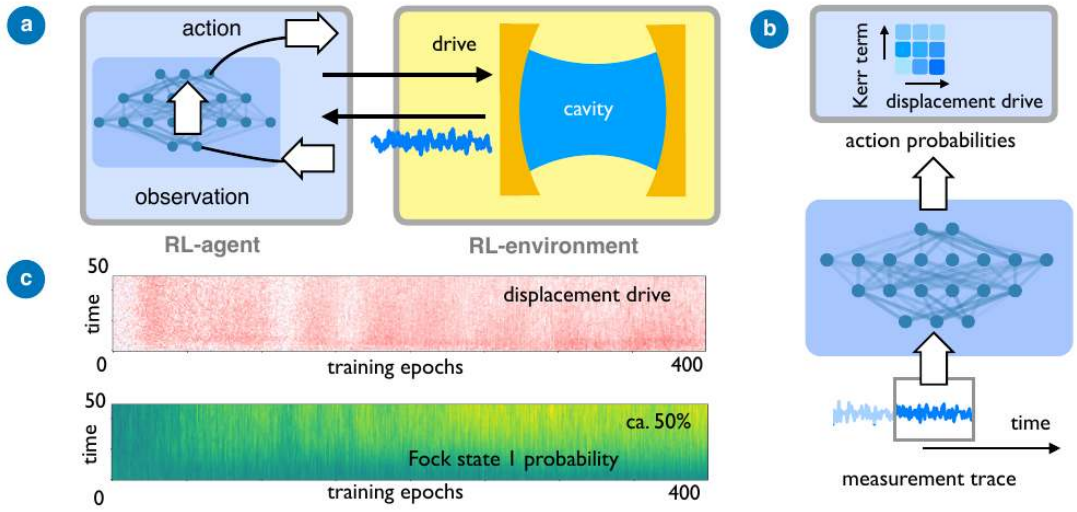


Figure 58: Source figure of the cavity feedback study. Panel (a) gives the control loop, panel (b) the policy network and its recent observation window, and panel (c) the drive and one-photon population during training. Our numerical study uses a simplified discrete measurement model and gives a separate set of results.

16.5 The best constant drive

Before any learning, we compute what a drive without feedback achieves in the same model. An episode lasts $T\Delta t_c = 5/\kappa$, or $T = 100$ decisions, and starts in the empty cavity. It is scored by the probability of one photon averaged over the episode,

$$\bar{P}_1 = \frac{1}{T}\sum_{t=1}^{T}\langle 1|\hat{\rho}_t|1\rangle. \tag{16.9}$$

For a constant drive the readings are unused, and the average over trajectories is an exact linear map of the density matrix. Averaged over its outcomes, one meter reading multiplies the element ρ_{nm} by $e^{-(n-m)^2/8\sigma_m^2}$, a Gaussian integral done in Appendix E. The meter thus destroys the coherence between photon numbers that the drive builds up. With the meter on, the best constant drive is $\varepsilon = 1.35\,\kappa$ and reaches $\bar{P}_1 = 0.258$. With the meter off, the best is $\varepsilon = 0.7\,\kappa$ with $\bar{P}_1 = 0.292$, below e^{-1} because the cavity starts empty (Figure 59). The value 0.258 is the reference for the agent, the best that a drive without feedback achieves in the model in which the agent acts. The averaged factor $e^{-(n-m)^2/8\sigma_m^2}$ is the effect of a random phase rotation $e^{i\phi\hat{n}}$ with a Gaussian angle ϕ of variance $1/4\sigma_m^2$. The average of $e^{i\phi(n-m)}$ over such ϕ is $e^{-(n-m)^2/8\sigma_m^2}$. A phase rotation turns a coherent state $|\alpha\rangle$ into the coherent state $|\alpha e^{i\phi}\rangle$. Drive, loss and averaged meter together therefore keep the cavity in a mixture of coherent states, whose $P(1)$ is at most e^{-1}. No drive without feedback can exceed e^{-1} with the meter on either.

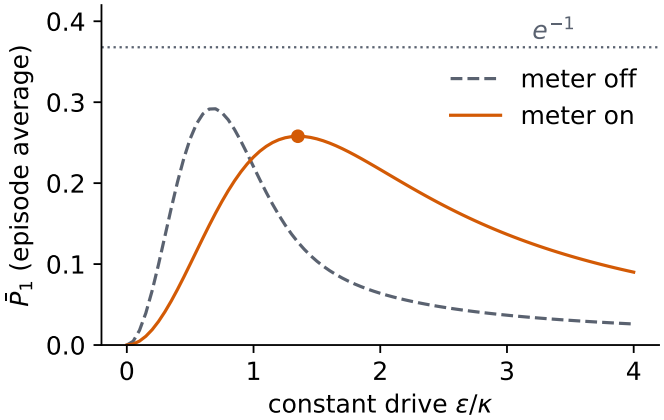


Figure 59: The episode average $\bar{P}_1$ of the probability of one photon for a constant drive ε, with the meter switched on and off, computed from the exact density-matrix map.

16.6 The control task

Fock-state feedback

Observation
the last 10 meter readings, $o_t = (x_{t-9}, \ldots, x_t)$, with zeros before the first reading. The agent never sees the state, the photon losses or the photon number.

Action
one of 5 drive strengths, $\varepsilon \in \{0, 0.5, 1, 2, 4\}\,\kappa$, applied for the next interval Δt_c.

Reward
the probability of one photon after the interval, $r_{t+1} = |\langle 1|\psi_{t+1}\rangle|^2$, computed by the simulation from the full state. The return of an episode is $G = \sum_{t=0}^{T-1} r_{t+1}$, whose expectation is $\mathbb{E}[G] = T\bar{P}_1$, without discount.

Policy
a softmax policy (10.6) from a network with 10 inputs, two hidden layers of 64 units with the hyperbolic tangent and 5 outputs, 5189 parameters in total (Figure 60).

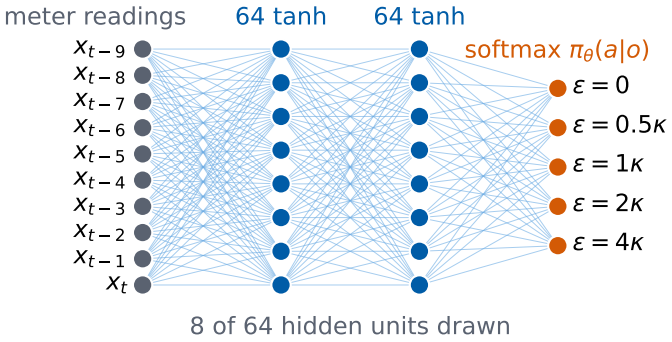


Figure 60: The policy network of the cavity agent. The ten most recent meter readings enter on the left, followed by two hidden layers of 64 tanh units, of which a few are drawn. The softmax of the five outputs gives the probabilities of the five drive strengths.

The observation is a window of past readings because a single reading is not a Markov state for the agent. The cavity state depends on the whole history of readings, drives and unobserved losses. The window is an enlarged observation that carries the most recent part of this memory, the situation of Section 3. The reward uses the full simulated state, including losses that nobody observes. Averaged over these unobserved losses it equals $\langle 1|\hat{\rho}_{t+1}|1\rangle$ of the state conditioned on the readings, so the expected return is the one an experimenter would assign from the readings. In an experiment

this reward would have to come from the device itself. One option is a projective photon-number measurement at the end of an episode. Its outcome, one photon or none, is an unbiased sample of $P(1)$ at that time and would reward the final $P(1)$ in place of its average. Another is an estimate of $\hat{\rho}$ computed from the readings with a model of the cavity. Training by the policy gradient is model-free in the sense of Part I, since the gradient uses only sampled histories and their rewards and no equation of the cavity.

The probability of a history τ of readings, losses and actions is a product of physics probabilities, for the readings and the losses, and of policy probabilities. Only the policy depends on θ. By the policy gradient theorem (10.5) the physics therefore drops out of the gradient, which is estimated from simulated or measured histories without a model of the cavity. We use REINFORCE with the reward-to-go and, as baseline, the batch average of G_t at the same time step, without discount. This average includes the trajectory itself. By equation (F.4) the expected update is then the policy gradient scaled by $1 - 1/B$ for a batch of B trajectories, a change of step size only. Each training epoch simulates 256 trajectories with the current policy and makes one step of the Adam optimiser. Three networks with different seeds are trained for 150 epochs each, a few minutes on an ordinary multi-core processor.

16.7 The trained agent

The untrained network chooses the five drives with roughly equal probability and reaches $\bar{P}_1 = 0.247$, about the value of the best constant drive. Within a few tens of epochs the batch average of $P(1)$ rises above e^{-1}, which no drive without feedback can reach. After about 100 epochs it levels off, with nearly the same learning curve for the three seeds (Figure 61). Tested on new trajectories, the three trained agents reach episode averages between 0.557 and 0.566, with standard errors of at most 0.003. These averages are 2.16 to 2.19 times the best constant drive. In the second half of the episode, once the cavity is filled, the agents hold $P(1)$ between 0.582 and 0.595, against 0.271 for the constant drive (Figure 62).

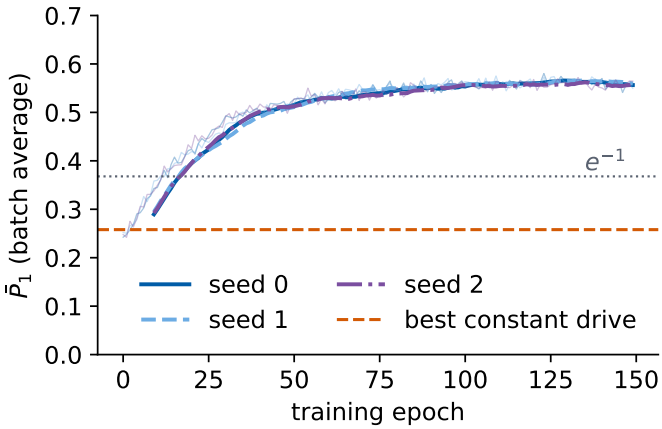


Figure 61: Training with REINFORCE. The batch average of the probability of one photon against the training epoch for three seeds, thin lines for every epoch and thick lines for the running mean over 10 epochs. The best constant drive and the coherent-state limit e^{-1} are drawn as references.

In one test trajectory (Figure 63) the agent drives hard at the start, when the readings are close to zero, and switches the drive off when the readings suggest one photon. After a loss the readings drift back towards zero, and the agent drives again. Since the readings are noisy, the agent acts on an average over several of them. A rule of this kind can be written down directly, "strongest drive if the mean of the last m readings is below x_{th}, otherwise no drive". Its best version found by a scan, $m = 4$ and $x_{\mathrm{th}} = 0.4$, reaches 0.549 to 0.550 on the same test sets, about 0.01 below the trained networks. Feedback exceeds the constant drive, and the network finds a rule of this quality from rewards alone, without being told what the readings mean. An optimal feedback policy for this model is unknown, so neither number is a proven optimum.

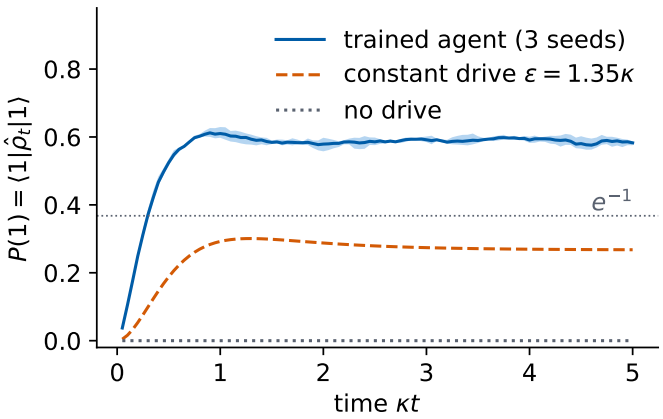


Figure 62: The probability of one photon $P(1) = \langle 1|\hat{\rho}_t|1\rangle$ against time for the trained agents (band of the three seeds), the best constant drive and no drive. $P(1)$ is the average of $|\langle 1|\psi_t\rangle|^2$ over test trajectories, the probability that a photon-number measurement at time t finds one photon.

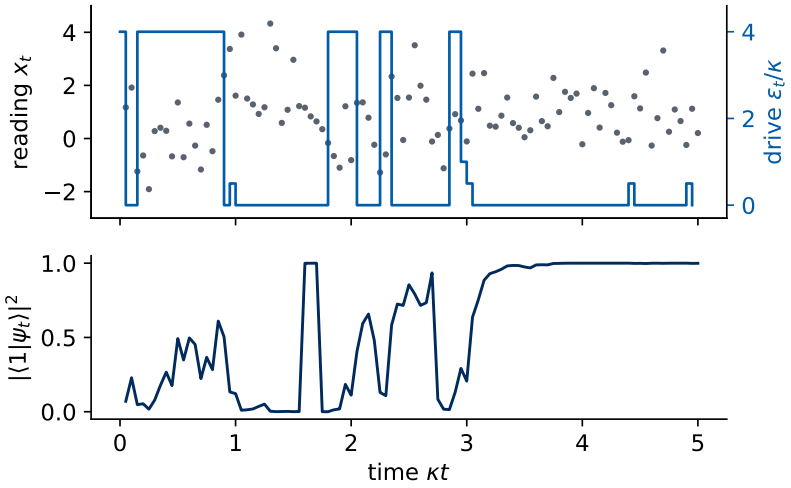


Figure 63: One test trajectory of a trained agent. Top, the meter readings x_t (dots) and the drive ε_t/κ chosen after each reading (line). Bottom, the probability of one photon $|\langle 1|\psi_t\rangle|^2$ in this trajectory, which the agent does not see.

At the end of the episode the cavity of the agent has the photon-number distribution $P(0) = 0.29$, $P(1) = 0.59$, $P(2) = 0.11$, against 0.41, 0.27 and 0.16 for the best constant drive (Figure 64). The Mandel parameter of this distribution is $Q_{\mathrm{M}} = -0.51$. Its negative value means that the photon number is spread below the Poisson value, which no coherent state or mixture of coherent states achieves. The constant drive with the meter on has $Q_{\mathrm{M}} = 0.65$, broader than Poisson, because the meter turns the coherent build-up into a random walk over photon numbers.

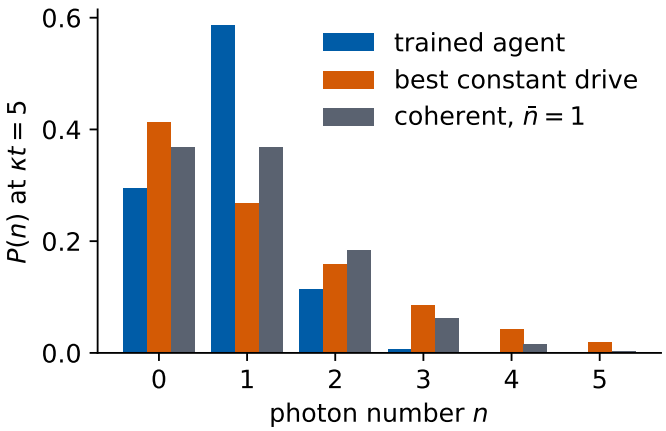


Figure 64: The photon-number distribution $P(n)$ at $t = 5/\kappa$ for a trained agent, for the best constant drive, and for the coherent state with $\bar{n} = 1$.

16.8 Discussion

We simulate pure states with random photon losses in place of the density matrices of Ref. [47]. Our model keeps the single decaying mode, the linear drive with discrete amplitudes as actions, and the weak photon-number measurement as the only information. It also keeps a feedforward network fed with the most recent readings, the reward $\langle 1|\hat{\rho}_t|1\rangle$, and policy-gradient training on batches of simulated trajectories. The measurement model differs, with discrete readings with Gaussian noise and a Bayes update in place of the continuous stochastic master equation of the source, stated in Appendix E. The discrete readings become a continuous measurement in the limit $\Delta t_c \to 0$ at fixed $\sigma_m^2 \Delta t_c$ [50]. The time step, the noise, the episode length, the drive strengths, the network and the training settings are our choices, since the source does not give all of them. The numbers are therefore not expected to agree with its figures, and we train for 150 epochs with three seeds.

The references without reinforcement learning are the best constant drive and the threshold rule. The agent exceeds the constant drive by a factor of about two, and the threshold rule comes within about 0.01 of the agent. On this task reinforcement learning found a good feedback rule without any model of the measurement. A physicist who looks at the agent's behaviour can write down a rule that is nearly as good. Reinforcement learning is worth more where no such rule is evident, with several modes, nonlinear elements or hardware delays, and where the model of the device is uncertain. Feedback of this kind has been run on hardware [51].

Part IV
Closing

17 Reinforcement learning and physics

Several objects of reinforcement learning are known objects of physics (Table 8). An exact correspondence is a mathematical identity, valid in the stated limit, and an analogy is a structural similarity that helps intuition and carries no proof.

In every example the reward defines the task, and its design decides the behaviour. A penalty per gate or a discount makes the agent prefer short circuits, and a cost on the CNOT gate changes which circuit is best. A reward paid only for the final fidelity left the discrete agent without a learning signal where the fidelity gain guided it. Value-based methods learn action values and act greedily on them, which suits a discrete and moderate set of actions. Policy-gradient methods learn the policy itself and handle continuous rotation angles in the same framework.

The studies use Bellman solutions, small-system searches, analytic circuits and numerical optimi-

reinforcement learning	physics	status
value as a sum over paths, eq. (A.1)	Neumann series of a resolvent, path-sum propagator	exact for the series, analogy for the propagator
Bellman equation of a random walker, eq. (A.2)	screened Poisson (Yukawa) equation, reward as charge	exact
Bellman optimality equation, $\Delta t \to 0$	Hamilton–Jacobi–Bellman equation (A.7)	exact for a smooth value function
policy gradient $\nabla_\theta \mathcal{J} = \mathbb{E}[G\,\nabla_\theta \ln p_\theta]$	fluctuation identity $\partial_\beta \langle E \rangle = -\operatorname{Var} E$	exact
softmax policy (10.6)	Boltzmann distribution	exact form
policy acting on the current state	closed-loop feedback control	exact

Table 8: Objects of reinforcement learning, their physical counterparts and the status of each correspondence.

sation as references. Beyond these references reinforcement learning adds the closed loop. A trained policy chooses each action from the state it observes and can react to errors that a fixed protocol cannot correct. In our simulations the agents read the exact state vector. On a device the state is known only through measurements, as for the cavity agent, which learns from noisy meter readings alone.

A The value as a physical potential

Several of the objects of reinforcement learning are known equations of electrostatics, mechanics and control theory in another notation. A correspondence called exact here is a mathematical identity, and an analogy is labelled as such.

A.1 A sum over paths

The solution of the linear system (6.7) can be expanded in powers of γ, since all eigenvalues of γP^π lie inside the unit circle,

$$V_\pi = \left(\mathbb{1} - \gamma P^\pi\right)^{-1} r_\pi = \sum_{k=0}^{\infty} \gamma^k \left(P^\pi\right)^k r_\pi. \tag{A.1}$$

The entry $[(P^\pi)^k]_{ss'}$ is the probability to be in s' after k steps when starting in s, a sum over all paths of length k. The vector $(P^\pi)^k r_\pi$ is therefore the expected reward r_{k+1} of the step after k transitions. The value is this reward summed over all times with the weights γ^k. The expansion is the Neumann series of the matrix $(\mathbb{1} - \gamma P^\pi)^{-1}$, the matrix analogue of $1/(1-x) = \sum_k x^k$, which physicists call a resolvent. It has the structure of a propagator of quantum mechanics written as a sum over all paths between two points. The analogy is structural, since the propagator sums complex amplitudes that interfere, while here nonnegative probabilities add.

A.2 The screened Poisson equation

For an agent that walks at random the Bellman equation becomes an equation of electrostatics. We take a chain of sites j, a policy that steps to the left or right with probability $1/2$ each, and a reward r collected on the single site j_0. Then $(P^\pi V)(j) = \frac{1}{2}[V(j-1) + V(j+1)]$, and the Bellman equation $V = r_\pi + \gamma P^\pi V$ reads $V(j) = r\,\delta_{j,j_0} + \frac{\gamma}{2}[V(j-1) + V(j+1)]$. Adding and subtracting $\gamma V(j)$ on the

right-hand side and rearranging gives

$$(1-\gamma)\,V(j) - \frac{\gamma}{2}\left[V(j+1) - 2V(j) + V(j-1)\right] = r\,\delta_{j,j_0}. \tag{A.2}$$

The bracket is the discrete second derivative, the lattice Laplacian. Equation (A.2) is the discrete screened Poisson equation, the equation of the Yukawa potential, with the reward as a point charge and $1-\gamma$ as a mass term that screens it.

Away from the source we try $V(j) \propto \lambda^{|j-j_0|}$. For $j \neq j_0$ equation (A.2) divided by $\lambda^{|j-j_0|}$ gives $1 = \frac{\gamma}{2}(\lambda + 1/\lambda)$, a quadratic equation $\lambda^2 - \frac{2}{\gamma}\lambda + 1 = 0$ whose root inside the unit interval is

$$\lambda = \frac{1-\sqrt{1-\gamma^2}}{\gamma}, \qquad V(j) \propto \lambda^{|j-j_0|} = e^{-|j-j_0|/\xi}, \qquad \xi := -\frac{1}{\ln\lambda}. \tag{A.3}$$

The value decays exponentially with the distance from the reward, with the screening length ξ. For a discount close to one, $1-\gamma^2 = (1-\gamma)(1+\gamma) \approx 2(1-\gamma)$. To leading order in the small quantity $\sqrt{1-\gamma}$ this gives $\lambda \approx 1 - \sqrt{2(1-\gamma)}$, $\ln\lambda \approx -\sqrt{2(1-\gamma)}$, and

$$\xi \approx \frac{1}{\sqrt{2(1-\gamma)}} \qquad (\gamma \to 1). \tag{A.4}$$

The same result follows from the continuum form of equation (A.2), $(1-\gamma)V - \frac{\gamma}{2}V'' = 0$ away from the source, whose solutions $e^{\pm x/\xi}$ have $\xi^2 = \gamma/[2(1-\gamma)]$. A discount close to one makes the value reach far, and a small discount keeps it local. Equation (A.3) gives the screening lengths $\xi = 2.14$, 7.04 and 22.35 sites for $\gamma = 0.9$, 0.99 and 0.999. On a two-dimensional lattice a rewarded and a penalised site act as a positive and a negative charge, and the value of a random walker is their screened potential.

The same picture describes the solvers. Jacobi relaxation solves a discretised Poisson-type equation in sweeps. Every sweep replaces the value at each site by the value that the equation demands given the current values of its neighbours. The iteration $V_{k+1} = r_\pi + \gamma P^\pi V_k$ does this for equation (A.2), and its contraction factor γ is the convergence rate of the relaxation. The learning methods of Section 8 have analogues as well. Monte Carlo evaluation estimates the value from sampled paths, the path-sum form (A.1). Temporal-difference learning enforces the local equation (A.2) at the states that a walker happens to visit, a stochastic relaxation.

A.3 Hamilton–Jacobi mechanics and the value function

The value function has a counterpart in the Hamilton–Jacobi theory of classical mechanics [21,22]. This optional subsection supplies no new algorithm. A value function, a local equation for it and a policy read off from its slope already appear in mechanics, where they describe all classical paths at once. We first state the mechanics, then derive the continuous-time Bellman equation, and finally show that the two coincide for a mechanical cost.

The word action has two meanings. In mechanics the *action* is the time integral of the Lagrangian, a number attached to a path. In reinforcement learning an action is a choice of the agent. In this subsection we write *mechanical action* for the first meaning, and the agent's action is the velocity it chooses.

Newton's equations follow one trajectory from its initial conditions. The Hamilton–Jacobi formulation assigns a number to every point of space and time, the mechanical action of the classical path that reaches it. The motion follows from how this number changes from point to point. As an analogy, a hiker reads the way downhill from the contour lines of a map.

We consider a particle of mass m at position x in a potential $U(x)$, with the Lagrangian $L = \frac{m}{2}\dot{x}^2 - U(x)$. Let $S(x,t)$ be the action $\int L\,dt'$ of the classical path that starts at a fixed point x_0 at the time t_0 and arrives at x at the time t. Its gradient is the momentum with which that path arrives, $p = \nabla S$, and S obeys the Hamilton–Jacobi equation

$$\partial_t S + \mathcal{H}(x, \nabla S) = 0, \qquad \mathcal{H}(x,p) = \frac{p^2}{2m} + U(x), \tag{A.5}$$

with the mechanical Hamiltonian $\mathcal{H}$, a classical function of position and momentum. For a free particle the classical path is a straight line, and $S = m(x-x_0)^2/[2(t-t_0)]$. Then $\partial_t S = -m(x-x_0)^2/[2(t-t_0)^2]$ and $(\nabla S)^2/2m = m(x-x_0)^2/[2(t-t_0)^2]$, so equation (A.5) holds. The gradient $\nabla S = m(x-x_0)/(t-t_0)$ is the momentum of the straight-line motion. Once S is known, the momentum at every point follows from its gradient without integrating a trajectory. The same function is the phase of the semiclassical wavefunction. We write $\psi = A\, e^{iS/\hbar}$ with a real amplitude $A(x,t)$ and a real phase $S(x,t)$. The real part of the Schrödinger equation $i\hbar\partial_t\psi = -\frac{\hbar^2}{2m}\partial_x^2\psi + U\psi$ then gives

$$\partial_t S + \frac{(\partial_x S)^2}{2m} + U = \frac{\hbar^2}{2m}\,\frac{\partial_x^2 A}{A},$$

and the imaginary part gives the conservation of the probability density A^2. The left-hand side is equation (A.5), and classical mechanics is the limit in which the right-hand side, of order $\hbar^2$, is negligible [23].

The value function of reinforcement learning is a map of the same kind over all states. The best action follows from how this map changes in the directions in which the actions move the state. The map obeys a local consistency condition, the Bellman equation, in place of equation (A.5). Value iteration carries information backwards from the reward and builds the map from its values at the end. The Hamilton–Jacobi equation is a differential equation in t and x, so the Bellman equation has to be written in continuous time [24,20].

We let the time step Δt between decisions shrink. The dynamics $\dot{x} = f(x,a)$ of a state $x \in \mathbb{R}^d$ is deterministic, a reward rate $r(x,a)$ pays $r\,\Delta t$ per step, and the discount is $\gamma = e^{-\eta\Delta t}$ with a rate η. The Bellman optimality equation (7.4) reads

$$V(x) = \max_a \Big[r(x,a)\,\Delta t + e^{-\eta\Delta t}\, V\big(x + f(x,a)\,\Delta t\big)\Big]. \tag{A.6}$$

For a differentiable V, with ∇V its gradient with respect to the state, we expand to first order in Δt, $e^{-\eta\Delta t}V(x + f\Delta t) = V(x) + [f\cdot\nabla V(x) - \eta V(x)]\,\Delta t + O(\Delta t^2)$. The term $V(x)$ cancels on both sides, and dividing by Δt and letting $\Delta t \to 0$ gives the Hamilton–Jacobi–Bellman (HJB) equation [20],

$$\eta\, V(x) = \max_a \big[r(x,a) + f(x,a)\cdot\nabla V(x)\big]. \tag{A.7}$$

Along the motion $\dot{x} = f$, the product $f\cdot\nabla V$ is the rate dV/dt at which the value changes. For the best action the equation therefore says $\eta V = r + dV/dt$. The value that discounting removes per unit time, ηV, is supplied by the reward collected per unit time and by the change of value along the motion. As an analogy, interest on a capital equals income plus the growth of the capital. The maximum selects the action whose direction f climbs the value most steeply after its reward is counted, the continuous counterpart of the greedy policy. The limit holds wherever the value function is smooth.

For a mechanical problem the HJB equation is the Hamilton–Jacobi equation. We take a finite horizon T_f, no discount, the velocity as the action, $\dot{x} = a$, and the reward rate $r = -L(x,a)$ with $L = \frac{m}{2}a^2 - U(x)$. The value now depends on the remaining time, $V(x,t)$, and the same expansion as above gives $0 = \partial_t V + \max_a[-L + a\cdot\nabla V]$. Mechanics speaks of costs, so we write $W = -V$ for the smallest action $W(x,t) = \min \int_t^{T_f} L\,dt'$ of a path that starts at x at the time t and ends anywhere at T_f. The equation becomes $0 = \partial_t W + \min_a[\frac{m}{2}a^2 - U + a\cdot\nabla W]$. The bracket is smallest where its derivative $ma + \nabla W$ vanishes, at the velocity

$$a^* = -\nabla W/m, \tag{A.8}$$

and inserting a^* gives the value $-(\nabla W)^2/2m - U = -\mathcal{H}(x,\nabla W)$, so that $\partial_t W - \mathcal{H}(x,\nabla W) = 0$. In the remaining time $\tau = T_f - t$ the equation reads $\partial_\tau W + \mathcal{H}(x,\nabla_x W) = 0$, with the condition $W(x,T_f) = 0$ at the end. It is equation (A.5) with time running backwards from the end, as value iteration runs backwards from the reward. The optimal motion (A.8) runs downhill on the cost-to-go, and its momentum $ma^* = -\nabla W = \nabla V$ is the gradient of the value. The sign is opposite to that of $p = \nabla S$ because S is the mechanical action as a function of the end point of a path with a fixed start.

classical mechanics	reinforcement learning
Lagrangian L	reward rate $r = -L$
mechanical action of the optimal path, W	value, $V = -W$
momentum $p = m\dot{x} = -\nabla W$	optimal action, $m\,a^* = \nabla V$
Hamilton–Jacobi equation (A.5)	Bellman equation for $\Delta t \to 0$, equation (A.7)

Table 9: Correspondence between Hamilton–Jacobi mechanics and the value function, for velocity as the action and the reward rate $-L$.

The cost-to-go W is the mechanical action as a function of the start point of a path with a free end. Moving the start forward along the path shortens the remaining path, so the two gradients enter with opposite signs. Table 9 collects the correspondence.

The correspondence holds where the value function is smooth. For the harmonic potential $U = \frac{1}{2}m\omega^2 x^2$ the solution is $W = -\frac{1}{2}m\omega\, x^2 \tan(\omega\tau)$, which diverges when the remaining time reaches $\tau = \pi/(2\omega)$. In the Lagrangian $L = \frac{m}{2}\dot{x}^2 - U$ a large displacement lowers the mechanical action through the potential term while it costs kinetic energy. We deform the optimal path by $\epsilon \sin[\pi(t' - t)/(2\tau)]$, which vanishes at the fixed start and is free at the end. The mechanical action changes at second order by $\epsilon^2 \frac{m\tau}{4}[(\pi/2\tau)^2 - \omega^2]$. For $\tau < \pi/(2\omega)$ this change is positive and the path is a minimum. For longer horizons it is negative, the mechanical action decreases without bound along this deformation, and W describes only a stationary path. The time $\pi/(2\omega)$ is a quarter of the oscillation period. With noise in the dynamics the HJB equation gains a term with second derivatives of V. Reinforcement learning approximates the solution of the Bellman equation from samples where the dynamics is unknown or the state space is too large to solve such an equation directly.

A.4 The linear quadratic regulator

Linear dynamics with a quadratic cost, the linear quadratic regulator (LQR), reduces optimal control to a matrix equation. In this case the Bellman equation of a continuous control problem is solved without any learning, and it is the standard reference for testing reinforcement learning in continuous control [25]. The dynamics is $\dot{x} = A_\mathrm{d} x + B_\mathrm{d} a$ with constant matrices A_d and B_d. The reward is $r = -x^\top C_x x - a^\top C_a a$ with positive definite matrices C_x and C_a that weigh the deviation of the state and the strength of the control. We try the quadratic value $V(x) = -x^\top K x$ with a symmetric matrix K, so $\nabla V = -2Kx$, and insert it into equation (A.7),

$$-\eta\, x^\top K x = \max_a \big[-x^\top C_x x - a^\top C_a a - 2x^\top K(A_\mathrm{d} x + B_\mathrm{d} a)\big]. \tag{A.9}$$

The bracket is a concave quadratic function of a, whose gradient $-2C_a a - 2B_\mathrm{d}^\top K x$ vanishes at

$$a^* = -C_a^{-1} B_\mathrm{d}^\top K\, x, \tag{A.10}$$

a linear feedback law. Inserting a^*, the two terms that contain a add up to $x^\top K B_\mathrm{d} C_a^{-1} B_\mathrm{d}^\top K x$. Writing $2x^\top K A_\mathrm{d} x = x^\top (K A_\mathrm{d} + A_\mathrm{d}^\top K) x$ we obtain an identity between quadratic forms that holds for every x, hence for the matrices,

$$K A_\mathrm{d} + A_\mathrm{d}^\top K - K B_\mathrm{d} C_a^{-1} B_\mathrm{d}^\top K + C_x - \eta K = 0. \tag{A.11}$$

For $\eta = 0$ the condition becomes the algebraic Riccati equation. It has several solutions, and the value function is given by the symmetric positive semidefinite solution for which the feedback (A.10) stabilises the dynamics. The optimal controller of the LQR is the linear feedback (A.10), computed from the solution of a matrix equation. Reinforcement learning is needed where such a solution is unavailable, because the dynamics is nonlinear, unknown or only sampled.

B Notation

symbol	meaning
s_t, a_t, r_{t+1}	state and action at step t, and the reward that follows them, always indexed $t+1$
T	final step of an episode, in Section 15 the duration of a protocol, and in Section 14 also the name of the $\hat{T}$ gate
γ, G_t, τ, B	discount factor, return from step t, trajectory $(s_0, a_0, r_1, s_1, \dots)$, number of episodes or transitions in a batch
π, π_θ, V_π, Q_π, A_π	policy, parametrised policy, state value, action value, advantage
V_*, Q_*, π^*	optimal values and optimal policy
$p(s', r\|s, a)$, P^π, r_π, Λ	dynamics, transition matrix of a policy, expected reward per step, largest eigenvalue of P^{π^*} on the non-terminal states
α, ε	learning rate, exploration rate, while in Section 16 α is the amplitude of a coherent state and ε the drive strength
θ, θ^-	parameters of a network or of a parametrised policy, and of the target network
$\mathcal{J}(\theta)$, $z(s,a)$, $\theta_{s,a}$, b	expected return of a parametrised policy, logits of a softmax policy, logits of the tabular softmax policy, baseline
μ, $\sigma = e^\nu$	mean and width of a Gaussian policy, with the log-width ν
$\tilde{g}$, $\widetilde{\nabla_\theta \mathcal{J}}$, $\tilde{A}_t$, $\tilde{F}$	sampled estimates of the policy gradient and of the advantage, fidelity estimated from measurement shots
$r_t(\theta)$, $\epsilon_{\mathrm{B}}(V)$	probability ratio of proximal policy optimisation (Section 12), Bellman residual of a value estimate
$\hat{\sigma}_x, \hat{\sigma}_y, \hat{\sigma}_z$, σ_{e}, σ_F, σ_{m}, σ_{tilt}	Pauli operators, noise of estimated values, noise of a measured fidelity, noise of one meter reading, scale of random tilts
λ, ξ, η, $W(x,t)$	decay factor and screening length of the value of a random walker, discount rate in continuous time, cost-to-go (Section A)
$\hat{\mathcal{H}}$	Hamiltonian operator, written without hat for the classical mechanical Hamiltonian, while $\hat{H}$, $\hat{S}$, $\hat{T}$ are gates
$\hat{X}_i$, $\hat{Y}_i$, $\hat{Z}_i$	Pauli operators $\hat{\sigma}_x$, $\hat{\sigma}_y$, $\hat{\sigma}_z$ acting on qubit or spin i
N, J, h, g	number of qubits or spins, coupling, transverse field, longitudinal field of the Ising chain (14.2)
F, ζ, D, n_{ang}, n	fidelity, cost per gate, number of entangling layers, number of angles of a template, number of gates of a circuit, in Section 16 the photon number
μ_k, $S_{N/2}$	Schmidt coefficients, half-chain entanglement entropy in bits
q, N_T, $\hat{U}^{\pm}_{\hat{O}}$	qubit index, in Section 15 the order parameter $q(T)$, number of intervals of a protocol, evolutions under Hamiltonian terms
κ, x_t, $P(n)$	in Section 16 the loss rate, the meter reading, the photon-number distribution
n_s, $n_{\max}$, D^*, $e_{t,q}$	shots per fidelity estimate, largest number of gates, depth found by gradient ascent, gate type on bond q at step t
T_c, T_{QSL}, $\bar{P}_1$, $\hat{\rho}$	end of the overconstrained phase, quantum speed limit, episode average of $P(1)$, density matrix
exceptions	in Section 14 θ denotes the parameters of the Q-network and of the policy network and the rotation angles $\theta_{t,q}$ of the circuit template, and μ the Schmidt coefficients μ_k and the Gaussian means μ_q, never in the same formula
$F_*(n)$, G_*, F_1^*, F_2^*	best fidelity with n gates and best return from the exhaustive searches, fidelity thresholds of the two ground-state agents
$t_\pm$, Δt_{B}	durations of the actions $\hat{U}^{\pm}_{\hat{O}}$, of one interval of a bang-bang protocol
Δt, Δt_{c}, Δt_{s}	time step of the continuous-time limit (Section A), decision interval and simulation step of the cavity
p_{g}, c_X, c_Z, s_{start}, s_{detour}	error probability of a gate, coefficients of the longitudinal-field chain, the two states of the detour task
ζ_a, ζ_{CX}, β, $\mathcal{E}(s)$	cost of the gate a, cost of a CNOT, entropy coefficient, entropy of the policy in the state s (Section 14), while in Section 10 β is an inverse temperature
A, B, $\mathbf{b}$	the two halves of a chain in the Schmidt decomposition, the magnetic field of Section 15, while B is also the batch size
w_T, $c(s)$, Φ, α_b	cost of a $\hat{T}$ gate, cheapest cost of a circuit from s, shaping potential, step size of the running baseline
d_i, Q_{M}	terminal flag in deep Q-learning, Mandel parameter of Section 16
K, C_x, C_a, A_{d}, B_{d}	Riccati matrix, cost matrices and dynamics matrices of the linear quadratic regulator, while in Section 14 K counts the values of a discretised angle

Table 10: Notation. A symbol keeps one meaning within a section, and the few that change meaning between sections are listed with the section where the second meaning applies. A hat marks a quantum operator and a tilde a sampled estimate of a classical quantity.

C Settings of the ground-state agents

Both agents of the ground-state preparation are multilayer perceptrons with 2^{N+1} inputs, 2 hidden layers of 128 rectified linear units and one output per action. The first agent has 12 outputs and the second 24, which for six spins gives 36 120 parameters. They are trained with the Double DQN loss (9.2) in its Huber form, the discount $\gamma = 0.98$ and the Adam optimiser with the learning rate 5×10^{-4}. Each run keeps 48 episodes going at the same time. In every iteration each episode takes one ε-greedy action, and the new transitions enter a replay memory of 20 000 transitions. Once the memory holds 1000 transitions, one gradient step on a random batch of 64 stored transitions updates the network. The target network is refreshed every 100 iterations, and ε decreases linearly from one to 0.02 over the first half of the training. The first agent trains for 4000 iterations. The second agent trains for 20 000 iterations for four spins and 30 000 for six. It starts from the states in which the trained first agent reaches F_1^* from 512 new tilted all-up states. Every target and chain length is trained with 3 seeds. Table 11 lists the settings that differ from Ref. [44].

item	Metz and Bukov	our reimplementation
chain length	4, 16 and 32 spins	4 and 6 spins
state	compressed tensor-network representation	exact vector of 2^N amplitudes
Q-network	trainable tensor network and a network with two hidden layers of 100 to 200 tanh units	two hidden layers of 128 rectified linear units
loss	squared error	Huber function
initial states, four spins	random product states and random reflection-symmetric states	all-up state with small random tilts
actions of the second agent	12 fine actions, steps $\pi/16$ and $\pi/21$	24 actions, the 12 coarse ones and 12 fine ones with steps $\pi/16$ and $\pi/26$
reward of the second agent	$N^{-1} \ln F$	$\ln F / \lvert \ln F_1^* \rvert$
replay memory, batch	8000 transitions, 32 to 64	20 000 transitions, 64
target network copy	every 10 steps	every 100 iterations
exploration	ε decays exponentially to 0.01 over 40 000 to 80 000 episodes	ε decays linearly to 0.02 over half of the training

Table 11: Settings of the ground-state agents that differ from those of Metz and Bukov.

D Exact preparation of the W state

The gates $\hat{H}$, $\hat{S}$, $\hat{T}$ and CNOT applied to $|0 \ldots 0\rangle$ produce amplitudes of a special form. The CNOT only permutes amplitudes, $\hat{S}$ and $\hat{T}$ multiply them by powers of $\omega = e^{i\pi/4}$, and $\hat{H}$ forms sums and differences divided by $\sqrt{2}$. Every amplitude of a state reached after any number of gates is therefore $b/\sqrt{2}^m$. Here m is an integer and b is a number in the set $\mathbb{Z}[\omega]$ of integer combinations of $1, \omega, \omega^2, \omega^3$. The set is closed under addition and multiplication and contains $\sqrt{2} = \omega + \omega^7$. Suppose a circuit prepared the W state with unit fidelity, i.e. up to a global phase. Then every nonzero amplitude has modulus $1/\sqrt{N}$, so $|b|^2 = b\bar{b} = 2^m/N$. The number $b\bar{b}$ is a product of elements of $\mathbb{Z}[\omega]$, since the complex conjugate of ω is ω^7, and it is a rational number. An element of $\mathbb{Z}[\omega]$ that is rational is an ordinary integer, because the four numbers $1, \omega, \omega^2, \omega^3$ are linearly independent over the rationals. So $2^m/N$ must be an integer, and N must be a power of two. For $N = 3$ and $N = 6$ the W state cannot be prepared with unit fidelity by these gates, at any circuit length.

E Cavity derivations and the continuous-measurement model

E.1 The coherent state under loss and drive

Without a jump the state changes in one simulation step by the operator

$$\sqrt{1-\kappa\Delta t_{\mathrm{s}}\hat{n}} \approx e^{-\kappa\Delta t_{\mathrm{s}}\hat{n}/2}, \tag{E.1}$$

to first order in Δt_{s}. Acting on a coherent state, and with $\hat{n}|n\rangle = n|n\rangle$,

$$\begin{aligned} e^{-\kappa\Delta t_{\mathrm{s}}\hat{n}/2}|\alpha\rangle &= e^{-|\alpha|^2/2}\sum_n \frac{\left(\alpha e^{-\kappa\Delta t_{\mathrm{s}}/2}\right)^n}{\sqrt{n!}}|n\rangle \\ &\propto |\alpha e^{-\kappa\Delta t_{\mathrm{s}}/2}\rangle. \end{aligned} \tag{E.2}$$

After normalisation the state is again coherent, with the amplitude reduced by the factor $e^{-\kappa\Delta t_{\mathrm{s}}/2} \approx 1-\kappa\Delta t_{\mathrm{s}}/2$. A jump does not change a coherent state, because $\hat{a}|\alpha\rangle = \alpha|\alpha\rangle$. The drive (16.3) acting for a time t gives

$$\exp\left[\varepsilon t\,(\hat{a}^\dagger - \hat{a})\right]|0\rangle = |\varepsilon t\rangle, \tag{E.3}$$

so in one step it displaces the amplitude by $\varepsilon\,\Delta t_{\mathrm{s}}$. Equations (E.2) and (E.3) together give

$$\alpha \to \alpha + \Big(\varepsilon - \frac{\kappa}{2}\alpha\Big)\Delta t_{\mathrm{s}}, \qquad \text{that is} \qquad \dot{\alpha} = \varepsilon - \frac{\kappa}{2}\alpha, \tag{E.4}$$

whatever the record of clicks.

E.2 The averaged meter reading

A reading x multiplies ψ_n by $\sqrt{L_n(x)}$, with the likelihood

$$L_n(x) = \frac{1}{\sqrt{2\pi\sigma_{\mathrm{m}}^2}}\exp\Big[-\frac{(x-n)^2}{2\sigma_{\mathrm{m}}^2}\Big], \tag{E.5}$$

and occurs with the probability that normalises the result. Averaged over the readings, the unnormalised updates combine with these probabilities so that the density matrix becomes

$$\rho_{nm} \;\to\; \rho_{nm}\int dx\,\sqrt{L_n(x)L_m(x)}. \tag{E.6}$$

The exponents in the integrand combine as

$$(x-n)^2 + (x-m)^2 = 2\Big(x - \frac{n+m}{2}\Big)^2 + \frac{1}{2}(n-m)^2, \tag{E.7}$$

and the remaining Gaussian integral over x equals one,

$$\begin{aligned} \int dx\,\sqrt{L_n(x)L_m(x)} &= \frac{1}{\sqrt{2\pi\sigma_{\mathrm{m}}^2}}\int dx\,\exp\Big[-\frac{(x-\frac{n+m}{2})^2}{2\sigma_{\mathrm{m}}^2}\Big]\,e^{-(n-m)^2/8\sigma_{\mathrm{m}}^2} \\ &= e^{-(n-m)^2/8\sigma_{\mathrm{m}}^2}. \end{aligned} \tag{E.8}$$

The diagonal elements, $n = m$, are unchanged, so the meter does not change the photon-number probabilities on average, and the coherences between different photon numbers are suppressed.

E.3 The continuous-measurement model

Our simulation uses discrete Gaussian photon-number readings and a likelihood update. The continuous-measurement equations connect it to the source model. For a prescribed drive and weak photon-number measurement, the average state obeys

$$\frac{d\hat{\rho}}{dt} = -i[\hat{\mathcal{H}}_{\mathrm{d}}, \hat{\rho}] + \kappa \mathcal{D}[\hat{a}]\hat{\rho} + \Gamma_{\mathrm{m}} \mathcal{D}[\hat{n}]\hat{\rho}, \qquad \mathcal{D}[\hat{L}]\hat{\rho} = \hat{L}\hat{\rho}\hat{L}^{\dagger} - \tfrac{1}{2}\{\hat{L}^{\dagger}\hat{L}, \hat{\rho}\}.$$

The three terms describe the drive, photon loss and measurement dephasing. For ideal measurement the conditional state and measurement record obey

$$\begin{aligned} d\hat{\rho}_{\mathrm{c}} &= \big(-i[\hat{\mathcal{H}}_{\mathrm{d}}, \hat{\rho}_{\mathrm{c}}] + \kappa \mathcal{D}[\hat{a}]\hat{\rho}_{\mathrm{c}} + \Gamma_{\mathrm{m}} \mathcal{D}[\hat{n}]\hat{\rho}_{\mathrm{c}} \big) dt + \sqrt{\Gamma_{\mathrm{m}}}\, \mathcal{M}[\hat{n}]\hat{\rho}_{\mathrm{c}}\, dW, \\ dY &= 2\sqrt{\Gamma_{\mathrm{m}}}\, \mathrm{Tr}(\hat{n}\hat{\rho}_{\mathrm{c}}) dt + dW, \\ \mathcal{M}[\hat{n}]\hat{\rho}_{\mathrm{c}} &= \hat{n}\hat{\rho}_{\mathrm{c}} + \hat{\rho}_{\mathrm{c}}\hat{n} - 2\,\mathrm{Tr}(\hat{n}\hat{\rho}_{\mathrm{c}})\hat{\rho}_{\mathrm{c}}. \end{aligned}$$

Here Γ_{m} is the measurement strength and dW is a Wiener increment of mean zero and variance dt. The stochastic term updates the state conditioned on the noisy record. A feedback policy makes the drive depend on the past record.

F The variance-minimising constant baseline

The baseline of equation (10.12) leaves the expected gradient unchanged and can be chosen to minimise its variance [1,2]. We take one component k of the gradient and a constant baseline b_k, and write $z_k = \partial_{\theta_k} \ln \pi_\theta(a|s)$ for the score and G for the return that multiplies it. The variance of the single-sample estimate $(G - b_k)z_k$ is

$$\mathrm{Var}\left[(G - b_k)z_k\right] = \mathbb{E}\big[(G - b_k)^2 z_k^2\big] - \big(\mathbb{E}[G z_k]\big)^2, \tag{F.1}$$

where the second term does not depend on b_k because $\mathbb{E}[b_k z_k] = 0$. The derivative with respect to b_k is $-2\mathbb{E}[(G - b_k)z_k^2]$, and setting it to zero gives

$$b_k^* = \frac{\mathbb{E}[G\, z_k^2]}{\mathbb{E}[z_k^2]}, \tag{F.2}$$

the average return weighted with the squared score. When the return and the squared score are uncorrelated, b_k^* is the mean return, the batch-mean baseline used for the cavity and for the continuous compilation. The variance-minimising baseline and the value baseline can differ.

F.1 Estimated baselines and finite batches

The baseline identity assumes that, given the information available before the current action, the baseline is independent of that sampled action. A baseline fitted from past data can satisfy this condition, while a batch mean that includes the current trajectory needs a separate calculation.

We consider independent trajectories $i = 1, \ldots, B$, a trajectory return $G_i = G(\tau_i)$, and its score $S_i = \nabla_\theta \ln p_\theta(\tau_i)$. Let $g = \mathbb{E}[G_i S_i]$ and $\bar{G} = B^{-1} \sum_j G_j$. Since $\mathbb{E}S_i = 0$ and different trajectories are independent,

$$\mathbb{E}[\bar{G} S_i] = \frac{1}{B}\mathbb{E}[G_i S_i] + \frac{1}{B}\sum_{j \neq i} \mathbb{E}[G_j]\mathbb{E}[S_i] = \frac{g}{B}, \tag{F.3}$$

$$\mathbb{E}\left[\frac{1}{B}\sum_i (G_i - \bar{G}) S_i\right] = \left(1 - \frac{1}{B}\right) g. \tag{F.4}$$

For this fixed-size independent batch, self-inclusion rescales the mean gradient.

The leave-one-out baseline excludes the current trajectory,

$$b_{-i} = \frac{1}{B-1}\sum_{j\neq i} G_j, \qquad B > 1, \tag{F.5}$$

$$\mathbb{E}[(G_i - b_{-i})S_i] = g. \tag{F.6}$$

For the reward-to-go at a fixed time, the baseline uses the other independent episodes at that time. A variable-length batch also needs a mask of the active episodes, so that the baseline excludes the current episode and uses only eligible other episodes. A fixed or previously fitted baseline serves as a fallback when none remain.

Dividing advantages by the return standard deviation of the same batch introduces a random denominator correlated with the gradient estimate. This operation changes the estimator. It can help numerically, while unbiasedness then no longer follows from the baseline identity.

For a vector score the total variance is minimised by the scalar $b_{\text{opt}} = \mathbb{E}[G\|S\|^2]/\mathbb{E}[\|S\|^2]$, the component formula above with z_k^2 replaced by $\|S\|^2$.

F.2 A value baseline can increase variance

We consider one binary action $a \in \{0, 1\}$ sampled with probability $\Pr(a = 1) = p$ and the reward $R = a$. For a logistic policy parameter the score is $S = a - p$. At $p = 0.9$ the expected gradient is $p(1 - p) = 0.09$. Evaluating the two possible outcomes gives

$$\operatorname{Var}(RS) = 0.0009, \tag{F.7}$$

$$\operatorname{Var}[(R - 0.9)S] = 0.0576, \tag{F.8}$$

$$b_{\text{opt}} = \frac{\mathbb{E}[RS^2]}{\mathbb{E}[S^2]} = 0.1. \tag{F.9}$$

The exact value baseline is $\mathbb{E}R = 0.9$, and it increases variance in this example. With $b_{\text{opt}} = 0.1$, both actions give the same score-weighted estimate, 0.09, so its variance is zero. A baseline that estimates the value accurately therefore need not minimise gradient variance.

G Convergence of value updates

G.1 Fixed points, relaxation and stochastic updates

If the complete Bellman average were available, moving every entry a fraction α towards it would apply

$$\mathcal{U}_\alpha V = (1 - \alpha)V + \alpha\mathcal{T}_\pi V. \tag{G.1}$$

The fixed point is unchanged. For $0 < \alpha \leq 1$ and $\gamma < 1$, the Bellman contraction gives

$$\|\mathcal{U}_\alpha V - \mathcal{U}_\alpha W\|_\infty \leq (1 - \alpha)\|V - W\|_\infty + \alpha\|\mathcal{T}_\pi V - \mathcal{T}_\pi W\|_\infty \tag{G.2}$$

$$\leq [1 - \alpha(1 - \gamma)]\|V - W\|_\infty. \tag{G.3}$$

This deterministic relaxation approaches the Bellman fixed point. Temporal-difference learning replaces the average by samples and updates only visited entries, and its convergence needs a stochastic-approximation argument in addition to contraction. In a finite table, bounded rewards, repeated visits and suitably decreasing learning rates control the sampling noise. A fixed positive learning rate generally leaves continuing fluctuations around the solution.

G.2 A stopping criterion for value iteration

For a discounted continuing task, let $\epsilon_{\mathrm{B}}(V) = \|\mathcal{T}V - V\|_\infty$ be the Bellman residual. Adding and subtracting $\mathcal{T}V$, and using the contraction (7.6), gives

$$\|V_* - V\|_\infty \le \|\mathcal{T}V_* - \mathcal{T}V\|_\infty + \epsilon_{\mathrm{B}}(V) \tag{G.4}$$

$$\le \gamma\|V_* - V\|_\infty + \epsilon_{\mathrm{B}}(V), \tag{G.5}$$

$$\|V_* - V\|_\infty \le \frac{\epsilon_{\mathrm{B}}(V)}{1-\gamma}. \tag{G.6}$$

The residual can be measured during planning without knowing V_*. Because of the denominator, a small residual can still leave an appreciable value error when the discount is close to one. The bound concerns an exact Bellman operator, and an approximate transition model adds another source of error.

H Small compilation experiments

Three small experiments complement the studies of the main text. They use restricted tasks and short training, and their measurements are separate from the recorded results of the main text.

H.1 A finite-budget GHZ compiler

We start in $|000\rangle$ and seek $|\mathrm{GHZ}_3\rangle$ with the restricted action set $\hat{H}_0$, $\widehat{\mathrm{CNOT}}_{0\to1}$ and $\widehat{\mathrm{CNOT}}_{1\to2}$ and a budget of 6 gates. The observation includes the physical state and the remaining budget, and the reward is the fidelity gain minus the gate cost used above, with the same discount. The task ends on success or when the budget is exhausted. Because the restriction makes the set of reachable physical states finite, backward induction gives a reference value for every remaining budget.

The reference circuit has fidelities 1/2, 1/4, 1/4 and 1 along its three gates. Its discounted return is

$$G = -0.26 + 0.95(-0.01) + 0.95^2(0.74) = 0.39835. \tag{H.1}$$

All 4 independently trained tabular agents reached this reference return. The learning curves describe this restricted action set only.

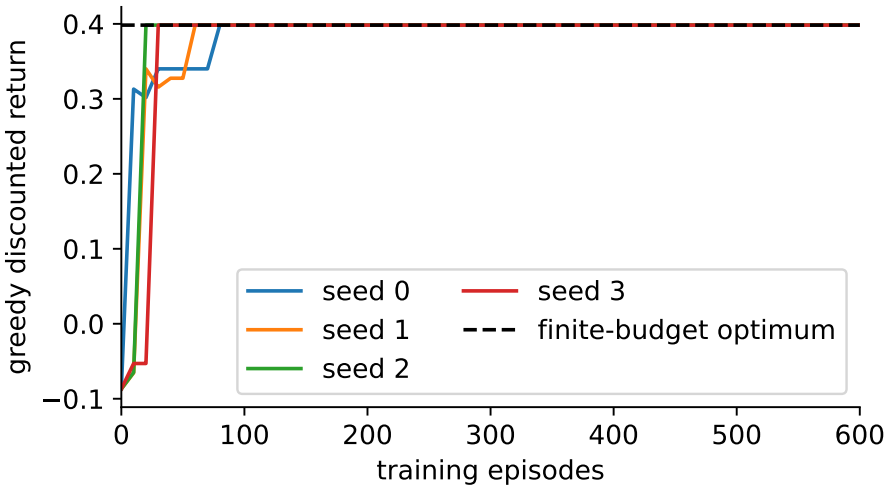


Figure 65: Greedy-policy return during tabular learning in the restricted finite-budget GHZ task. Each curve is an independent training seed. The dashed line is the backward-induction reference. Evaluation starts from the all-zero state with the full budget.

H.2 A baseline calculation with an exact gradient

We keep the three gates $\hat{H}_0$, $\widehat{\mathrm{CNOT}}_{0\to1}$ and $\widehat{\mathrm{CNOT}}_{1\to2}$ and the target $|\mathrm{GHZ}_3\rangle$ of the finite-budget compiler. The policy has one softmax distribution over the three gates at each of three times. We enumerate the 27 circuit sequences and their final fidelities. With a terminal-fidelity objective and discount one, the expected fidelity and its derivative are finite weighted sums. Independent sampled batches then estimate that gradient with different baselines. The self-inclusive mean follows the

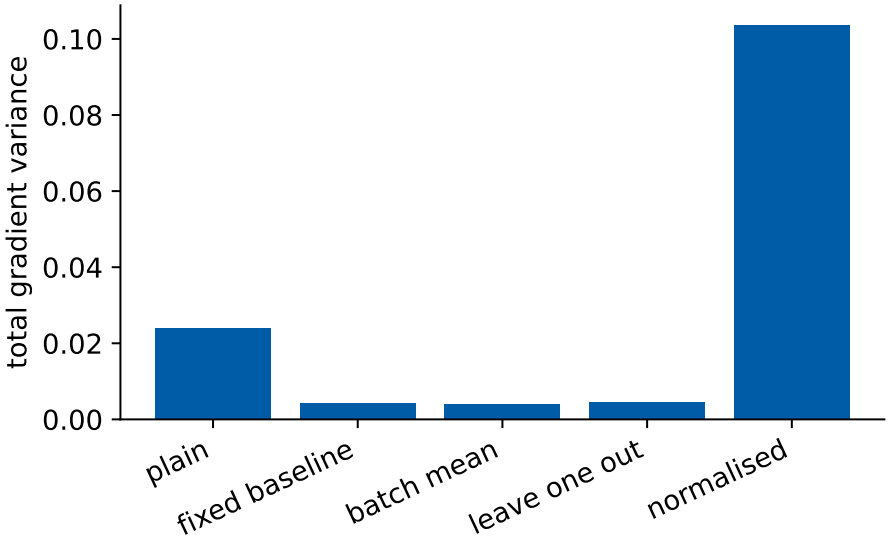


Figure 66: Sum of component variances of sampled gradient estimates in the fixed-depth circuit calculation. The normalised-return estimator has a different scale and expectation, so its bar shows the change of estimator and is read together with the reported component means.

factor derived in equation (F.4), and the leave-one-out mean preserves the expected gradient. For one gradient component the exact value is 0.00947. We use batches of 16 episodes and 5000 independent batches. The sum of the component variances is 0.0241 for plain returns and 0.00429 for the fixed baseline equal to the exact expected fidelity. It is 0.00408 for the self-inclusive batch mean and 0.00464 for the leave-one-out mean. The leave-one-out mean is unbiased and here has a slightly larger variance than the biased self-inclusive mean. The predicted 6% bias of the self-inclusive mean is comparable to the statistical resolution of this sample. Normalised returns divide by the batch standard deviation and estimate a rescaled gradient.

H.3 A short continuous-angle compiler

We prepare $|\mathrm{W}_2\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$ from $|00\rangle$. For two qubits the W state is a Bell state, related to $|\Phi^+\rangle$ by local gates, and the difference between GHZ and W entanglement appears only from three qubits on. Three layers apply one $\hat{R}_y$ rotation to each qubit, and the first two layers also choose no gate, CNOT or controlled-$\hat{Z}$ on the bond. A time-dependent policy learns independent Gaussian angles and categorical gates for each layer, with a leave-one-out baseline, terminal fidelity as reward and discount one. Its parameters are directly trained means, log widths and gate preferences, and this smaller policy is separate from the state-dependent network of the full study.

Each of the 3 seeds uses 64000 training fidelity evaluations. The final deterministic circuits have fidelities between 0.9914 and 0.9996, all above the success threshold $F > 0.99$, and every seed chooses CZ on both bonds. A direct optimisation shows that one CNOT or one CZ already reaches $F = 1$ in this template, whereas without an entangling gate the best fidelity is 1/2. Since the reward has no gate cost, the second entangling gate is not penalised. Figure 67 shows the deterministic-circuit learning curves. A numerical reference optimises the angles with both entangling operations fixed to CNOT, finding high-fidelity candidates with parameter-shift gradients and recording their evaluation

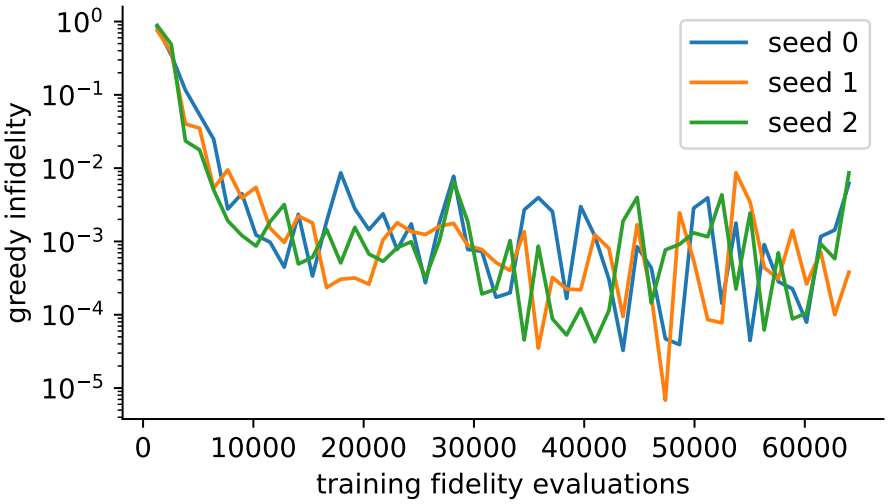


Figure 67: Greedy-circuit infidelity in the short continuous-angle experiment, for individual seeds against the number of training fidelity evaluations. The policy samples angles and gate types during training and is evaluated with mean angles and most probable gates.

costs. That reference uses exact model-based gradients on a fixed template, while the policy sees only sampled terminal rewards and can change gate types. A separate finite-shot run with one seed demonstrates training with noisy rewards and gives limited evidence about reliability.

References

1. A. Dawid, J. Arnold, B. Requena, A. Gresch, M. Płodzień, K. Donatella et al., Modern applications of machine learning in quantum sciences, arXiv:2204.04198, chapter Reinforcement learning, published as Machine Learning in Quantum Sciences (Cambridge University Press, 2025), doi:10.1017/9781009504942
2. R. S. Sutton, A. G. Barto, Reinforcement Learning, An Introduction, 2nd ed. (MIT Press, 2018), http://incompleteideas.net/book/the-book-2nd.html
3. D. Silver et al., Mastering the game of Go without human knowledge, Nature **550**, 354 (2017), doi:10.1038/nature24270
4. D. Silver et al., Mastering the game of Go with deep neural networks and tree search, Nature **529**, 484 (2016), doi:10.1038/nature16961
5. J. Tromp, The number of legal Go positions, in Computers and Games (CG 2016), Lecture Notes in Computer Science **10068**, 183 (Springer, 2016), doi:10.1007/978-3-319-50935-8_17
6. J. Schrittwieser et al., Mastering Atari, Go, chess and shogi by planning with a learned model, Nature **588**, 604 (2020), doi:10.1038/s41586-020-03051-4
7. V. Mnih et al., Human-level control through deep reinforcement learning, Nature **518**, 529 (2015), doi:10.1038/nature14236
8. O. Vinyals et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning, Nature **575**, 350 (2019), doi:10.1038/s41586-019-1724-z
9. B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, I. Mordatch, Emergent tool use from multi-agent autocurricula, International Conference on Learning Representations (2020), arXiv:1909.07528, videos at https://openai.com/index/emergent-tool-use/
10. Boston Dynamics, Starting on the right foot with reinforcement learning, blog post, https://bostondynamics.com/blog/starting-on-the-right-foot-with-reinforcement-learning/
11. J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, Learning agile and dynamic motor skills for legged robots, Sci. Robot. **4**, eaau5872 (2019), doi:10.1126/scirobotics.aau5872
12. A. A. Melnikov et al., Active learning machine learns to create new quantum experiments, Proc. Natl. Acad. Sci. USA **115**, 1221 (2018), doi:10.1073/pnas.1714936115

13. A. Fawzi et al., Discovering faster matrix multiplication algorithms with reinforcement learning, Nature **610**, 47 (2022), doi:10.1038/s41586-022-05172-4

14. D. J. Mankowitz et al., Faster sorting algorithms discovered using deep reinforcement learning, Nature **618**, 257 (2023), doi:10.1038/s41586-023-06004-9

15. J. Degrave et al., Magnetic control of tokamak plasmas through deep reinforcement learning, Nature **602**, 414 (2022), doi:10.1038/s41586-021-04301-9

16. V. Kain et al., Sample-efficient reinforcement learning for CERN accelerator control, Phys. Rev. Accel. Beams **23**, 124801 (2020), doi:10.1103/PhysRevAccelBeams.23.124801

17. I.-J. Chen et al., Precise atom manipulation through deep reinforcement learning, Nat. Commun. **13**, 7499 (2022), doi:10.1038/s41467-022-35149-w

18. M. A. Nielsen, I. L. Chuang, Quantum Computation and Quantum Information (Cambridge University Press, 2010), doi:10.1017/CBO9780511976667

19. R. Bellman, Dynamic Programming (Princeton University Press, 1957, reissued 2010), doi:10.1515/9781400835386

20. K. Doya, Reinforcement learning in continuous time and space, Neural Comput. **12**, 219 (2000), doi:10.1162/089976600300015961

21. H. Goldstein, C. Poole, J. Safko, Classical Mechanics, 3rd ed. (Addison-Wesley, 2002), Chapter 10, Hamilton–Jacobi theory

22. L. D. Landau, E. M. Lifshitz, Mechanics, 3rd ed. (Pergamon, 1976), §47, the Hamilton–Jacobi equation

23. L. D. Landau, E. M. Lifshitz, Quantum Mechanics, Non-relativistic Theory, 3rd ed. (Pergamon, 1977), the quasi-classical limit

24. D. P. Bertsekas, Dynamic Programming and Optimal Control, Vol. I, 4th ed. (Athena Scientific, 2017)

25. B. Recht, A tour of reinforcement learning, The view from continuous control, Annu. Rev. Control Robot. Auton. Syst. **2**, 253 (2019), doi:10.1146/annurev-control-053018-023825

26. C. J. C. H. Watkins, P. Dayan, Q-learning, Mach. Learn. **8**, 279 (1992), doi:10.1007/BF00992698

27. J. E. Smith, R. L. Winkler, The optimizer's curse, Skepticism and postdecision surprise in decision analysis, Manag. Sci. **52**, 311 (2006), doi:10.1287/mnsc.1050.0451

28. H. van Hasselt, Double Q-learning, Adv. Neural Inf. Process. Syst. **23** (2010), https://papers.nips.cc/paper_files/paper/2010/hash/091d584fced301b442654dd8c23b3fc9-Abstract.html

29. S. Bravyi, A. Kitaev, Universal quantum computation with ideal Clifford gates and noisy ancillas, Phys. Rev. A **71**, 022316 (2005), doi:10.1103/PhysRevA.71.022316

30. A. Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations, Theory and application to reward shaping, in Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999), p. 278, https://mlanthology.org/icml/1999/ng1999icml-policy/

31. R. S. Sutton, D. McAllester, S. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, Adv. Neural Inf. Process. Syst. **12** (2000), https://proceedings.neurips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html

32. R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, Mach. Learn. **8**, 229 (1992), doi:10.1007/BF00992696

33. R. J. Williams, J. Peng, Function optimization using connectionist reinforcement learning algorithms, Connect. Sci. **3**, 241 (1991), doi:10.1080/09540099108946587

34. L.-J. Lin, Self-improving reactive agents based on reinforcement learning, planning and teaching, Mach. Learn. **8**, 293 (1992), doi:10.1007/BF00992699

35. H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, Proc. AAAI Conf. Artif. Intell. **30** (2016), doi:10.1609/aaai.v30i1.10295

36. D. M. Greenberger, M. A. Horne, A. Shimony, A. Zeilinger, Bell's theorem without inequalities, Am. J. Phys. **58**, 1131 (1990), doi:10.1119/1.16243

37. W. Dür, G. Vidal, J. I. Cirac, Three qubits can be entangled in two inequivalent ways, Phys. Rev. A **62**, 062314 (2000), doi:10.1103/PhysRevA.62.062314

38. P. Pfeuty, The one-dimensional Ising model with a transverse field, Ann. Phys. **57**, 79 (1970), doi:10.1016/0003-4916(70)90270-8

39. H. Kim, D. A. Huse, Ballistic spreading of entanglement in a diffusive nonintegrable system, Phys. Rev. Lett. **111**, 127205 (2013), doi:10.1103/PhysRevLett.111.127205

40. K. Mitarai, M. Negoro, M. Kitagawa, K. Fujii, Quantum circuit learning, Phys. Rev. A **98**, 032309 (2018), doi:10.1103/PhysRevA.98.032309

41. M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, N. Killoran, Evaluating analytic gradients on quantum hardware, Phys. Rev. A **99**, 032331 (2019), doi:10.1103/PhysRevA.99.032331

42. Y.-H. Zhang, P.-L. Zheng, Y. Zhang, D.-L. Deng, Topological quantum compiling with reinforcement learning, Phys. Rev. Lett. **125**, 170501 (2020), doi:10.1103/PhysRevLett.125.170501

43. L. Moro, M. G. A. Paris, M. Restelli, E. Prati, Quantum compiling by deep reinforcement learning, Commun. Phys. **4**, 178 (2021), doi:10.1038/s42005-021-00684-3

44. F. Metz, M. Bukov, Self-correcting quantum many-body control using reinforcement learning with tensor networks, Nat. Mach. Intell. **5**, 780 (2023), doi:10.1038/s42256-023-00687-5, arXiv:2201.11790

45. L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, E. F. Mishchenko, The Mathematical Theory of Optimal Processes (Routledge reprint, 2018), doi:10.1201/9780203749319

46. G. C. Hegerfeldt, Driving at the quantum speed limit, Optimal control of a two-level system, Phys. Rev. Lett. **111**, 260501 (2013), doi:10.1103/PhysRevLett.111.260501

47. F. Marquardt, Machine learning and quantum devices, SciPost Phys. Lect. Notes **29** (2021), doi:10.21468/SciPostPhysLectNotes.29, arXiv:2101.01759

48. L. Mandel, Sub-Poissonian photon statistics in resonance fluorescence, Opt. Lett. **4**, 205 (1979), doi:10.1364/OL.4.000205

49. H. M. Wiseman, G. J. Milburn, Quantum Measurement and Control (Cambridge University Press, 2009), doi:10.1017/CBO9780511813948

50. K. Jacobs, D. A. Steck, A straightforward introduction to continuous quantum measurement, Contemp. Phys. **47**, 279 (2006), doi:10.1080/00107510601101934

51. K. Reuer et al., Realizing a deep reinforcement learning agent for real-time quantum feedback, Nat. Commun. **14**, 7138 (2023), doi:10.1038/s41467-023-42901-3

52. Farama Foundation, Gymnasium CartPole documentation, https://gymnasium.farama.org/environments/classic_control/cart_pole/.

53. Farama Foundation, Gymnasium LunarLander documentation, https://gymnasium.farama.org/environments/box2d/lunar_lander/.

54. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms (2017), https://arxiv.org/abs/1707.06347.